



Ain Shams University
Faculty of Engineering
Computer Engineernig and Software Systems

CFI - Care: Collaborative Flow of Intelligence for Care

Graduation Project Thesis

Author: Mohamed Salah Fathy 21P0117
Youssef Tamer Hossam 21P0138
Salma Hisham Wagdy 21P0124
Salma Mohamed Yousef 21P0148
Belal Anas Awad 21P0072
Supervisors: Prof Dr Haitham Azmi

Submission Date: 27 January, 2026

Mohamed Salah Fathy 21P0117
Youssef Tamer Hossam 21P0138
Salma Hisham Wagdy 21P0124
Salma Mohamed Yousef 21P0148
Belal Anas Awad 21P0072
Date: 27 January, 2026

Contents

Acknowledgments	X
1 Introduction	1
1.1 The Global Digital Health Revolution	1
1.2 The Situation in Egypt	1
1.3 Problem Statement	1
1.4 The Proposed Solution: CFI Care	2
1.5 Thesis Objectives	2
1.6 Thesis Structure	3
2 Literature Review	4
2.1 Background and Context	4
2.1.1 State of Health Data in Egypt	4
2.1.2 Challenges to Implementation	4
2.2 Theoretical Framework	5
2.2.1 Patient-Centricity in Healthcare	5
2.2.2 Data Governance and Interoperability	5
2.2.3 Security: Role-Based Access Control	5
2.2.4 Artificial Intelligence: Retrieval-Augmented Generation	5
2.2.5 Version Control for Health Records	5
2.3 HL7 - Health Level Seven	5
2.3.1 Evolution of Standards	6
2.4 Fast Healthcare Interoperability Resources (FHIR)	6
2.4.1 Core Architecture	7
2.4.2 Key Technical Advantages	7
2.4.3 Comparison of Standards	7
2.5 Authentication and Authorization Standards	9
2.5.1 OAuth! (OAuth!) 2.1 and OpenID Connect (OIDC)	9
2.5.2 JSON Web Tokens (JWT! (JWT!))	9
2.5.3 PKCE! (PKCE!)	9
2.6 Healthcare Security and Integration Standards	9
2.6.1 FHIR! (FHIR!) and SMART! (SMART!) on FHIR!	9
2.7 Token-Based Architecture and Session Management	10
2.7.1 The Backend-for-Frontend (BFF) Pattern	10
2.7.2 Distributed Session State Cache	10
2.8 Edge Security and Infrastructure Defense	10
2.8.1 Perimeter Protection and Rate Limiting	10

2.8.2	Mobile Application Isolation	10
2.9	Audit Logging, Diagnostics, and Forensic Compliance	10
2.9.1	Security Event Auditing	10
3	Proposed Design Architecture	12
3.1	Backend	12
3.1.1	System Architecture	12
3.1.2	High-Level Architecture	12
3.1.3	Node.js Application Server	13
3.1.4	Data Persistence and Schema	14
3.1.5	Request/Response Flow and Data Transformation Pipeline . . .	16
3.1.6	Error Handling and Fault Tolerance	20
3.1.7	Security and Infrastructure Layer	21
3.1.8	HAPI FHIR Server Implementation	21
3.2	API Design and Endpoints	22
3.2.1	RESTful Structure and Formats	22
3.2.2	FHIR-Compliant Endpoints	22
3.2.3	Custom History Graph Endpoints	22
3.3	Patient History Graph Implementation	23
3.3.1	Motivation: Beyond Linear Medical Records	23
3.3.2	Graph Data Model	24
3.3.3	Temporal Branching and Concurrent Episodes	25
3.3.4	Relationship Classification	26
3.3.5	Handling Edge Cases	27
3.3.6	Query Patterns and Optimization	27
3.4	Performance Optimization	28
3.4.1	Tier 1: Standard Resource Cache	29
3.4.2	Tier 2: Specialized Graph Cache	30
3.5	Frontend	33
3.5.1	Overview and Design Objectives	33
3.5.2	Technology Stack	34
3.5.3	System Architecture	34
3.5.4	Styling and UX Philosophy	35
3.5.5	Authentication and Navigation Shell	37
3.5.6	Patients Dashboard	40
3.5.7	Patient Journey Dashboard	44
3.5.8	Patient Journey Dashboard — Categorical View	46
3.5.9	Patient Journey Dashboard — Chronological View	48
3.5.10	Complementary Roles of the Two Views	51
3.5.11	Interactive Tutorial System	51
3.5.12	AI-Assisted Clinical Chat	53
3.5.13	Doctor Profile and Schedule Management	56
3.5.14	Domain Models and State Services	57
3.5.15	Error Handling and Resilience	58
3.5.16	Sequence Diagram	58
3.5.17	Component Diagram	60
3.5.18	Data Flow Diagram	60

3.5.19	Architectural Rationale	62
3.5.20	Graph Logic Test Suite	62
3.5.21	Summary	64
3.6	Mobile	1
3.7	Mobile Application Objectives & UI	1
3.7.1	Authentication Flow & Account Security	1
3.7.2	Episode of Care	8
3.7.3	Digital Document Management & Elimination of Physical Records	9
3.7.4	Doctor-Patient Data Sharing	16
3.7.5	Caregiver and Family Involvement	18
3.7.6	Appointment Management	22
3.7.7	Wearable Device Integration	28
3.8	Flutter and Dart	34
3.8.1	Clean Architecture in the Mobile Application	34
3.8.2	Layered Structure	35
3.8.3	Centralized Runtime Permission Handling	36
3.9	FCM Push Notifications	39
3.9.1	How It Works	39
3.10	Requirements Engineering	42

List of Figures

2.1	Evolution of HL7 Standards from Messaging to RESTful Resources. . .	6
2.2	Visual Comparison of Data structure: HL7 v2 Message vs. FHIR JSON Resource.	8
3.1	High-Level Container Architecture showing the interaction between the Nginx proxy, application services, and data persistence layers.	13
3.2	Internal layered architecture of the Node.js Application Server, detailing the middleware stack and service distribution.	14
3.3	Entity-Relationship Diagram (ERD) illustrating the integration of custom graph tables with the standard FHIR resource table.	15
3.4	Request/Response flow for patient data retrieval, showing cache hit and miss scenarios with complete middleware pipeline.	17
3.5	Complete encounter creation workflow showing FHIR resource persistence, graph construction, and cache invalidation cascade.	19
3.6	Example Patient History Graph showing a diabetes patient with concurrent hypertension, demonstrating temporal branching and multiple relationship types.	24
3.7	Two-Tier Caching Architecture showing distinct strategies for standard FHIR resources (Tier 1) and complex graph structures (Tier 2).	28
3.8	Registration form enforcing field completeness. The form cannot be submitted until all required fields are populated, preventing incomplete account creation.	37
3.9	Email verification step requiring entry of a six-digit code sent to the clinician's registered address, confirming ownership before account activation.	38
3.10	Recovery code generation screen presented at account activation. The clinician must copy or download the codes as a text file before proceeding, establishing a secure fallback authentication method.	38
3.11	Multi-factor authentication method selection screen, offering the clinician a choice between a TOTP authenticator application and a recovery authentication code.	39
3.12	Password reset interface allowing clinicians to securely regain access to their account through a guided credential recovery flow.	39
3.13	Patients Dashboard showing the master-detail layout. The left sidebar presents the patient navigator with search, filter, and episode management controls. The main panel displays the full clinical summary for the selected patient, including active conditions, medications, recent laboratory results, procedures, and a scoped encounter history.	40

3.14	Patient access grant modal presenting the six-cell OTP input field. The clinician enters the code generated on the patient’s device to initiate the access request handshake.	42
3.15	Categorical Patient Journey view representing medical events as a hierarchical left-to-right graph. High-priority events display animated priority rings, sibling relationships are labelled on each node, and the detail sidebar is visible on the right.	47
3.16	Chronological Patient Journey view displaying medical events along a horizontal D3.js timeline with pan and zoom. Parallel sub-branches represent alternative diagnostic hypotheses, each on its own horizontal track. The completed branch is merged back into the main timeline via a green arc, and the discontinued branch is blocked with a red wall marker. A “CASE CLOSED” flag is rendered at the timeline terminus when the main timeline is set to the Finished state.	50
3.17	Zoomed view of a completed branch in the chronological timeline. The green curved merge arc connects the branch’s final node back to the main timeline, with a “MERGED” label rendered above the arc. The branch line is rendered in green to indicate the validated state.	50
3.18	Zoomed view of a discontinued branch in the chronological timeline. A pulsing red wall marker is rendered at the branch terminus with a “DISCONTINUED” label. The branch line is rendered in red, and all add, edit, and delete operations on nodes within this branch are blocked. . .	50
3.19	Tutorial Step 4 of 18: Expand a branch. The categorical view shows first-level nodes in their collapsed default state. The + button on the CBC Blood Test node is highlighted with a pulse animation, and the tutorial card instructs the clinician to click it to expand the sub-tree. The step advances automatically upon interaction.	52
3.20	Tutorial Step 10 of 18: Switch to Chronological view. The view toggle in the header is highlighted, and the tutorial card instructs the clinician to click “Chronological” to transition the layout from the hierarchical tree to the D3.js timeline. The step advances automatically once the toggle is activated.	53
3.21	Tutorial Step 16 of 18: Main timeline state. The chronological view is shown with the branch state menu open on the main timeline line, presenting the In Progress and Finish Case options. The tutorial card instructs the clinician to click the main timeline to open this menu, advancing automatically once the interaction occurs.	53
3.22	AI-assisted clinical chat in its compact panel view, rendered alongside the Patient Journey Dashboard. The session list, message history, response mode selector, and composer are all visible within the panel.	55
3.23	AI-assisted clinical chat in maximised mode, expanding to fill the full viewport. This mode is intended for extended clinical reasoning sessions and is also the default view for conversations opened via the Fork action.	56
3.24	Doctor Profile page showing the practitioner information panel on the left and the Schedule Manager on the right, with slot creation form, status filters, and a today’s summary sidebar.	57

3.25	Sequence diagram illustrating the full MedGraph interaction lifecycle: iframe initialisation, postMessage-based node operations, backend confirmation, optimistic rollback on failure, and stateful mode switching. . .	59
3.26	Component diagram illustrating the architecture of the dashboard, MedGraph iframe boundary, AI chat module, application services, and backend integration points.	60
3.27	Data flow diagram for a MedGraph node CRUD operation. The flow begins with a user interaction in the graph engine (iframe), passes through the postMessage boundary into the Angular host, is forwarded to the FHIR backend, and returns via a confirmation or error path. On success, the temporary node ID is replaced with the persistent UUID. On failure, the pending operation is rolled back from the corresponding state map.	61
3.28	CFI-Care Graph Logic Test Suite dashboard after a full run. All 62 tests across five suites pass with zero failures and zero warnings in 22ms total. The scoreboard, per-suite result panels, performance bar chart, and log console are visible.	64
3.29	Login with KeyCloak	1
3.30	Register with KeyCloak	2
3.31	Verification Code	3
3.32	Recovery Codes	4
3.33	Enter Recovery Code #1	4
3.34	Authenticator Setup	5
3.35	Unauthenticated Reset Password	6
3.36	Verify your Identity	7
3.37	Enter & Confirm New Password	8
3.38	Episode of Care	8
3.39	Documents Screen	10
3.40	Add Document Form	11
3.41	trapezoid-shaped Image	13
3.42	Green Spirit Level because parallel to the surface	13
3.43	Red Spirit Level because tilted to the surface	14
3.44	Text Extraction Failed	15
3.45	OCR processing	15
3.46	Text Extract successfully and synced with backend	16
3.47	Notification with doctor requesting access to your data	17
3.48	Doctor appearing in Doctors that can access your data list and user can revoke access	18
3.49	Patient Generating Code	19
3.50	Caregiver entering Patient's code	19
3.51	Caregiver request access (Patient view)	20
3.52	Caregiver is notified and their list updates	21
3.53	Patient sees Caregiver in their family accessors list and ability to revoke access	21
3.54	Caregiver updating Patient data	23
3.55	Patient receiving notification that Caregiver updated Patient data . . .	23
3.56	Patient update their own data	24
3.57	Caregiver receiving notification that Patient updated their own data . .	24

3.58	list of doctor speciality	25
3.59	List of doctors with the speciality choosen (in this case Dermatology) .	26
3.60	Doctor Profile with slots available	27
3.61	Thank You Screen	28
3.62	Health_Connect Diagram	29
3.63	Granular Permissions	30
3.64	Data Sources & Priority	31
3.65	Health_Connect with GalaxyWatch & AndroidSmartphone	32
3.66	OMRON Connect requesting to read & write Samsung Health data . .	33
3.67	Withings requesting access to Health Connect	34
3.68	Layered Clean Architecture adopted in the mobile application	35
3.69	using handlePermission For Camera Access In code	37
3.70	Best Practices for Handling Permissions in Flutter	38
3.71	Dialog to allow the app to access the camera	39

Acknowledgments

We would like to extend our deepest gratitude to our esteemed supervisor, Prof Dr. Haitham Azmi, whose guidance, support, and invaluable insights have been instrumental in the successful completion of our senior project. His unwavering dedication and expertise have enhanced our understanding of the complexities involved in building an AI-Powered, Patient-Centric Healthcare Platform. Prof Dr. Azmi provided us with the intellectual stimulation and encouragement necessary to navigate the challenges we encountered throughout this research. His constructive feedback and meticulous mindfulness have significantly improved the quality of our work.

We are also grateful to Ain Shams University (ASU) for providing us with the resources and environment necessary to pursue this project. Special thanks to the Faculty of Engineering and the Computer and Systems Engineering Department for their continuous support and encouragement throughout our academic journey.

We would like to acknowledge the contributions of our colleagues and friends who provided valuable feedback and assistance during the development of this project. Their support and collaboration have been invaluable in overcoming technical challenges and refining the final product.

Abstract

Healthcare today is characterized by fragmented experiences where patient medical information is often dispersed across various institutions and providers. This project proposes a comprehensive, patient-centric healthcare application designed to consolidate all medical reports into a single, secure, and unified platform. The system employs **RBAC!** (**RBAC!**) to ensure privacy and controlled information sharing. To enhance clinical decision-making, the platform features a medical history visualization graph for tracking health trends and integrates a Retrieval-Augmented Generation (RAG)-enabled AI model to generate diagnostic reports and insights. By transforming isolated data into an interconnected ecosystem, this project aims to establish meaningful relationships between patients, caregivers, and doctors for personalized healthcare delivery.

Chapter 1

Introduction

1.1 The Global Digital Health Revolution

Healthcare is shifting from reactive treatment toward proactive, data-driven prevention. The global digital health market was valued at roughly USD 288 billion in 2024 and is projected to approach USD 946 billion by 2030, growing at more than 22% annually. This acceleration is powered by mobile health, telemedicine, and Artificial Intelligence (AI), which promise cheaper, faster, and more effective care. Yet adoption is uneven: while leading systems deploy AI for personalized care, many Low- and Middle-Income Countries (LMICs) still struggle with basic record keeping because of weak infrastructure, unclear policy, and fragmented data.

1.2 The Situation in Egypt

Egypt, home to over 106 million people and the largest healthcare market in MENA, has improved outcomes but faces a persistent data void. Records in many Public Health Centers (PHCs) remain paper-based, locked in filing rooms, and rarely interoperable across facilities. Digital systems that do exist often operate in silos, so a patient moving between providers loses continuity of care. Data quality is frequently incomplete or error-prone, and patients seldom provide formal written consent, reflecting limited trust and awareness of data value. This fragmentation prevents AI from learning accurate patterns and delivering benefits to Egyptian patients.

1.3 Problem Statement

When a patient sees multiple specialists, their medical records scatter across disconnected systems. Radiology images sit at one hospital, lab results at another, prescriptions at a third. Clinicians piece together incomplete histories from memory or phone calls. Patients have no way to track their own health trends over time. Care coordination breaks down because nobody has the full picture. Egypt needs modern, AI-enabled healthcare but lacks the foundational data ecosystem. The core challenges are:

- **Fragmentation:** Records scattered across clinics with no single longitudinal home.
- **Analog Dominance:** Paper-first workflows impede longitudinal tracking and lossless sharing.
- **Lack of Control:** Patients cannot easily access or share their own histories.
- **Privacy and Consent Gaps:** No standardized, auditable consent flows.
- **Missed AI Opportunities:** Absent or inaccessible digital data blocks safe clinical decision support such as Retrieval-Augmented Generation (RAG).

1.4 The Proposed Solution: CFI Care

CFI Care is an AI-powered, collaborative, patient-centric health record platform that unifies medical reports in one secure ecosystem while keeping patients in control.

- **Unified Records:** Consolidates data from disparate sources into a single, secure repository with timeline visualization.
- **Git-Inspired Version Control:** Tracks evolving treatment plans as a medical history graph to reveal branches, merges, and longitudinal trends.
- **Role-Based Access Control (RBAC):** Lets patients grant time-bound, role-specific access to clinicians or caregivers, creating a clear consent audit trail.
- **AI Integration (RAG):** Grounds generative summaries and answers in the patient's own records to support safer diagnostics and triage.
- **Mobile Digitization:** A camera scanner digitizes legacy paper reports, crowd-sourcing data ingestion despite PHC device constraints.

1.5 Thesis Objectives

This thesis aims to design and develop CFI Care to modernize health data management in Egypt by:

- Documenting digital health barriers in Egypt, including infrastructure gaps and consent practices.
- Designing a secure, scalable, and usable architecture for patient-centric records and clinician workflows.
- Implementing robust RBAC and consent enforcement to safeguard privacy while enabling necessary sharing.
- Engineering AI (RAG) services that can interpret medical records and deliver actionable insights.
- Empowering patients with tools for data ownership, appointments, caregiver sharing, and mobile digitization.

1.6 Thesis Structure

- Chapter 1: Introduction to the global digital health revolution, Egypt's healthcare challenges, and the CFI Care solution.
- Chapter 2: Standards (HL7/FHIR), governance, authentication, and security foundations.
- Chapter 3: CFI Care system architecture including backend, frontend, and mobile components.
- Chapter ??: Defense-in-depth security architecture with OAuth 2.0, JWT Tokens, RBAC, Rate Limiting on API endpoints, and audit logging.
- Chapter ??: AI integration using Retrieval-Augmented Generation for clinical decision support.
- Chapter ??: Testing methodology, implementation milestones, and validation.
- Chapter ??: Conclusion, expected impact, and future enhancements.

Chapter 2

Literature Review

2.1 Background and Context

2.1.1 State of Health Data in Egypt

Egypt’s largest-in-MENA healthcare market still operates with a persistent data void. Many Public Health Centers (PHCs) rely on paper records stored in locked rooms, producing data silos that cannot be queried or shared across providers. Even when digital systems exist, they often function in isolation, so patients effectively lose their history when moving between facilities. Data quality is frequently incomplete, error-prone, and difficult to audit, and formal written consent is rarely collected, reflecting limited trust and awareness of data value. These factors prevent AI systems from learning reliable patterns and delivering safe support to clinicians.

2.1.2 Challenges to Implementation

Key barriers identified in the Egyptian context include infrastructure limits, low digital literacy among frontline workers, organizational fragmentation, constrained funding, and cultural hesitancy around data sharing. Table 2.1 summarizes these obstacles.

Table 2.1: Implementation Barriers in Egypt’s Health Data Landscape

Barrier	Description
Infrastructure	Limited connectivity and device availability in PHCs.
Digital literacy	Nurses and clerks often lack training on digital tools.
Fragmentation	Providers operate in silos and resist data sharing.
Funding	Budgets for devices and training are inadequate.
Culture and trust	Patients may hesitate to share data or perceive little value in doing so.

2.2 Theoretical Framework

2.2.1 Patient-Centricity in Healthcare

Traditional institution-centric models keep data under provider control. A patient-centric approach shifts ownership to patients, ensuring portability and empowerment. When patients upload documents, manage appointments, and share records, adherence and outcomes improve.

2.2.2 Data Governance and Interoperability

High-quality, ethical data use depends on governance principles that prioritize protection, value, and equity. Interoperability—particularly via FHIR—enables systems to exchange and reuse structured health information, avoiding vendor lock-in and enabling national-scale insights.

2.2.3 Security: Role-Based Access Control

Role-Based Access Control (RBAC) restricts access by clearly defined roles. Patients can grant time-bound access to doctors or caregivers, creating auditable consent and aligning with privacy requirements.

2.2.4 Artificial Intelligence: Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) grounds AI outputs in a patient’s actual records. By retrieving relevant documents before generation, it reduces hallucinations and offers safer clinical decision support for summarization and question answering.

2.2.5 Version Control for Health Records

A Git-inspired model captures branching and merging in treatment histories, forming a medical history graph. This temporal view helps clinicians understand forks in care plans, compare outcomes, and track longitudinal trends.

2.3 HL7 - Health Level Seven

Health Level Seven (HL7) International is a standards development organization that has defined the framework for the exchange, integration, sharing, and retrieval of electronic health information for decades [?]. Understanding the evolution of these standards is crucial to appreciating the architectural shifts in modern healthcare applications.

2.3.1 Evolution of Standards

- **HL7 v2 (The Workhorse):** Established in 1989, HL7 v2 remains the most widely implemented standard for hospital workflows (e.g., ADT - Admission, Discharge, Transfer messages). It utilizes a non-XML, pipe-delimited syntax (e.g., MSH|^~\&|...). While highly efficient for legacy backend systems, its lack of semantic strictness leads to significant implementation variances, posing challenges for modern web and mobile application development [?].
- **HL7 v3 (The Theoretical Model):** Introduced in the 1990s to address the vagueness of v2, HL7 v3 was built on a rigorous Reference Information Model (RIM). It enforced strict semantic correctness using complex XML structures. However, this theoretical purity resulted in a steep learning curve and bloated message sizes, leading to poor adoption rates and a lack of backward compatibility with v2 [?].
- **HL7 CDA (Clinical Document Architecture):** A component of v3, CDA focuses on the exchange of clinical documents (such as discharge summaries). While effective for static snapshots of care, it is document-centric rather than data-centric, making it difficult to extract granular data (like a single heart rate reading) required for real-time interactive applications [?].

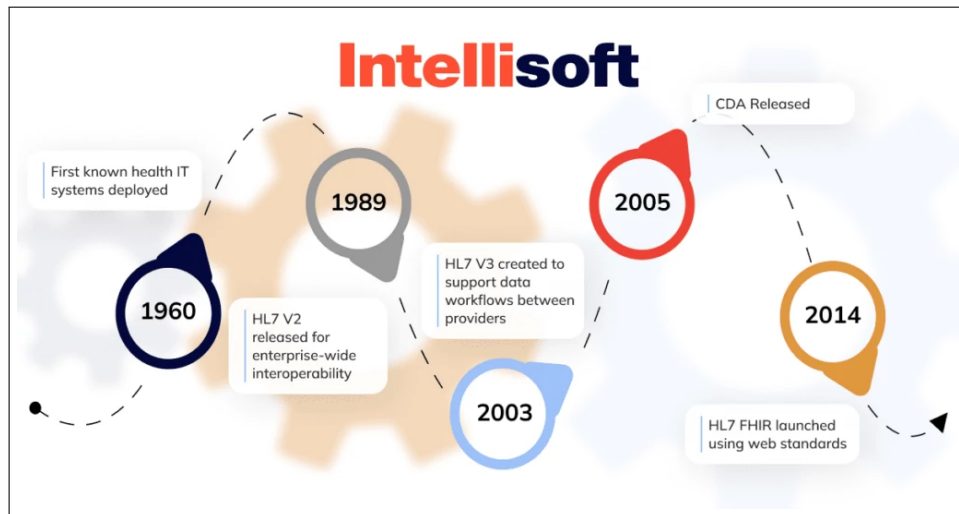


Figure 2.1: Evolution of HL7 Standards from Messaging to RESTful Resources.

2.4 Fast Healthcare Interoperability Resources (FHIR)

In response to the complexity of HL7 v3 and the limitations of v2, HL7 introduced **FHIR** (Fast Healthcare Interoperability Resources) in 2012. FHIR represents a paradigm shift from complex, custom healthcare protocols to modern web standards.

2.4.1 Core Architecture

Unlike its predecessors, FHIR is built on the RESTful API approach widely used in modern IT industries (e.g., by Google, Facebook, Twitter). It treats data elements as independent "Resources"—modular components like *Patient*, *Observation*, or *Medication*—that can be accessed and manipulated via standard HTTP methods (GET, POST, PUT, DELETE) [?].

2.4.2 Key Technical Advantages

- **Granularity via Resources:** FHIR allows systems to access specific data elements using unique URLs (e.g., `GET /Patient/123`), rather than exchanging entire heavy documents. This makes it ideal for mobile devices with limited bandwidth [?].
- **Modern Formats (JSON & XML):** While previous standards relied on complex XML or pipe-delimited text, FHIR natively supports JSON (JavaScript Object Notation), which is lightweight and directly parsable by web browsers and mobile apps [?].
- **The 80/20 Rule:** FHIR focuses on defining the 20% of requirements that satisfy 80% of common healthcare use cases. For the remaining complex or local requirements, it uses a built-in extension mechanism, ensuring the core standard remains simple and implementable [?].

2.4.3 Comparison of Standards

Table 2.2 highlights the fundamental differences between the major HL7 standards, illustrating why FHIR was chosen as the interoperability backbone for this project.

Table 2.2: Comparison of HL7 v2, v3, CDA, and FHIR Standards

Feature	HL7 v2	HL7 v3 / CDA	FHIR
Primary Paradigm	Event-Driven Messaging	Reference Model (RIM) / Documents	RESTful API & Resources
Data Format	Pipe-delimited text	Complex XML	JSON, XML, RDF
Transport	LLP / MLLP (TCP/IP)	SOAP / Web Services	HTTP / REST
Complexity	Moderate (Vague semantics)	High (Steep learning curve)	Low (Web-standard based)
Use Case	Hospital Backend (ADT, Orders)	Clinical Documents (Summaries)	Mobile, Web, Cloud, & Backend

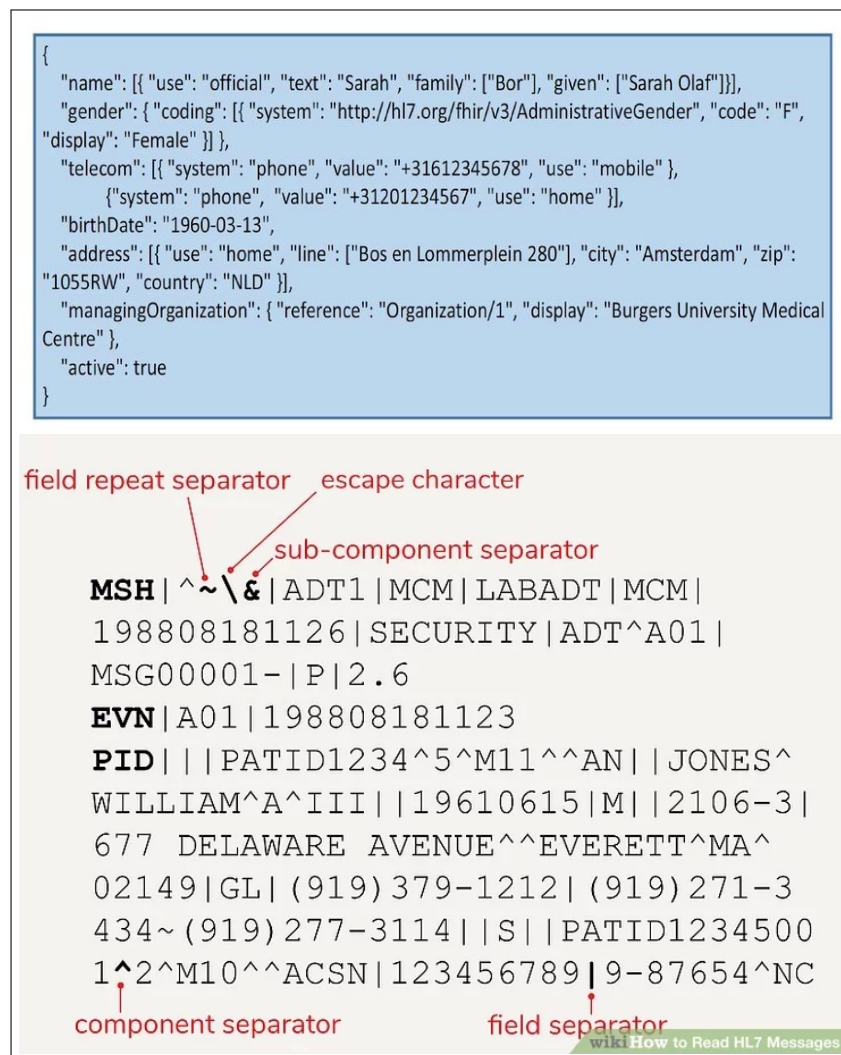


Figure 2.2: Visual Comparison of Data structure: HL7 v2 Message vs. FHIR JSON Resource.

2.5 Authentication and Authorization Standards

2.5.1 OAuth! 2.1 and OpenID Connect (OIDC)

Modern web and mobile security paradigms have transitioned toward the **OAuth! 2.1** consolidation framework, which deprecates inherently insecure vectors -such as the Resource Owner Password Credentials grant- due to the risks associated with client-side credential handling [?]. The standard explicitly mandates the Authorization Code Flow combined with dynamic cryptographic extensions for native applications, and delegates browser session management to secure edge boundaries. OpenID Connect (OIDC) [?] extends this framework by introducing an identity layer via ID tokens, enabling robust Single Sign-On (SSO) capabilities. Within decoupled health architectures, this project leverages gatekeeper abstractions to securely manage token translation scopes out-of-band.

2.5.2 JSON Web Tokens (JWT!)

JWT!s [?] are cryptographically signed structural tokens that encode user identity claims, access scopes, and session longevity attributes (e.g **iss**, **aud**, **exp**). Utilizing asymmetric signing algorithms (such as RS256) allows distributed microservices and backend API workers to perform stateless signature validation against an Identity Provider's JSON Web Key Sets (JWKS), removing the performance overhead of database lookups.

2.5.3 PKCE!

PKCE! [?] mitigates authorization code interception attacks on public and native clients by introducing a dynamic cryptographic secret exchange loop. By validating a high-entropy **code_verifier** against a pre-computed **code_challenge**, the protocol ensures that malicious actors tracking custom OS deep links cannot intercept or exchange leaked authorization codes for active token clusters [?].

2.6 Healthcare Security and Integration Standards

2.6.1 FHIR! and SMART! on FHIR!

The **FHIR!** specification [?] provides structural consistency for healthcare models, but relies on external security profiles for transactional protection. The **SMART! on FHIR!** framework [?] addresses this by mapping **OAuth! 2.0** access scopes directly to granular medical resources (e.g **patient/Observation.read**, **user/Condition.write**). This allows patient-controlled capability policies to interoperate seamlessly with standardized **FHIR!** Consent resource models [?], enforcing time-bound, actor-specific boundaries directly over **PHI!** (**PHI!**).

2.7 Token-Based Architecture and Session Management

2.7.1 The Backend-for-Frontend (BFF) Pattern

To guard web clients against **XSS!** (**XSS!**) token exfiltration, modern architectures employ the **BFF!** (**BFF!**) pattern [?]. By containing OAuth token loops entirely behind an edge gateway, access and refresh tokens are kept out of reach of browser storage layers. The frontend-to-backend connection is secured via encrypted HTTP-only, Secure, and SameSite-restricted session cookies, preventing both script-based extraction and **CSRF!** (**CSRF!**) vectors.

2.7.2 Distributed Session State Cache

While stateless JWT tokens simplify transaction execution, immediate security invalidation (e.g during logout) requires backchannel tracking. Utilizing low-latency, in-memory data structures (such as Redis) allows systems to maintain a real-time blacklist or session map, striking a secure balance between stateless processing speed and immediate session revocation capabilities.

2.8 Edge Security and Infrastructure Defense

2.8.1 Perimeter Protection and Rate Limiting

Securing electronic health records requires robust defenses at the ingestion boundary. Reverse proxies and API gateways (such as Nginx) isolate internal application nodes by terminating TLS profiles, dropping non-conforming HTTP payloads, and establishing strict rate-limiting boundaries backed by sliding-window token buckets to block distributed brute-force attempts and **DoS!** (**DoS!**) vectors [?].

2.8.2 Mobile Application Isolation

Native environments require a completely different security profile than web applications. Lacking standard cookie containment scopes, mobile applications mitigate malicious data access by isolating long-lived refresh tokens and active access JWTs inside hardware-backed storage containers, such as the Android Keystore [?], keeping secrets cryptographically secure even if the host operating system is compromised.

2.9 Audit Logging, Diagnostics, and Forensic Compliance

2.9.1 Security Event Auditing

Security tracing requires structured log recording for authentication lifecycles, user state transitions, data access denials, and cryptographic key rotations. To preserve forensic

utility, processing engines must isolate audit trails in real time to automated storage pipelines to prevent accidental downstream data exposure [?].

Chapter 3

Proposed Design Architecture

3.1 Backend

3.1.1 System Architecture

The CFI-Care system is built upon a robust three-tier architecture designed to facilitate secure and interoperable healthcare data management. The system integrates a modern frontend, a logic-heavy middleware layer, and a standards-compliant backend to ensure data consistency and security. The architecture adheres to the FHIR (Fast Healthcare Interoperability Resources) standard for all medical data exchange and utilizes OAuth 2.0 via Keycloak for identity management.

3.1.2 High-Level Architecture

The application operates through a strict separation of concerns. The Presentation Layer is an Angular-based Single Page Application (SPA) that serves as the user interface. It does not communicate directly with the data stores. Instead, all client requests are intercepted by an Nginx Reverse Proxy, which handles SSL/TLS termination and routes traffic to the appropriate internal services.

The core of the backend processing is handled by the Node.js Application Server. This server acts as the primary API gateway, implementing business logic, enforcing security policies, and managing user sessions. It communicates with the HAPI FHIR Server, a specialized Java-based component responsible for persisting clinical data in compliance with the FHIR R5 standard. Data storage is consolidated in PostgreSQL databases, while high-speed data access and session management are handled by Redis instances.

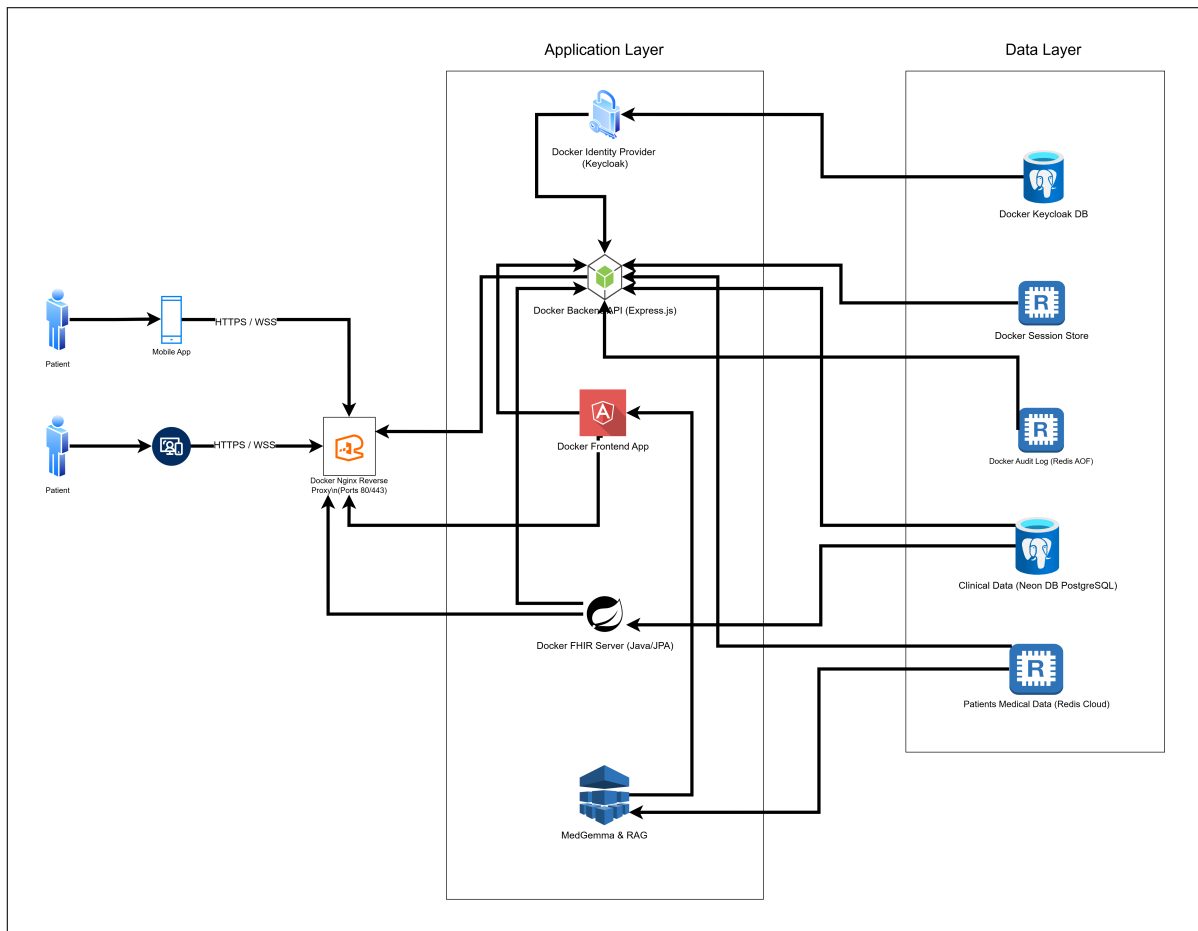


Figure 3.1: High-Level Container Architecture showing the interaction between the Nginx proxy, application services, and data persistence layers.

3.1.3 Node.js Application Server

The Node.js server, built with the Express.js framework, serves as the middleware layer that bridges the gap between the user interface and the standardized medical records. It is designed with a modular layered architecture to ensure scalability and maintainability.

Middleware and Security

Upon receiving a request, the server passes it through a comprehensive middleware stack. Security headers are applied using Helmet to prevent common web vulnerabilities. Cross-Origin Resource Sharing (CORS) is configured to restrict access to authorized domains. Crucially, the server implements rate limiting to protect API endpoints from abuse and utilizes a Redis-backed session store to manage user authentication states efficiently.

Routing and Services

The application is divided into specific route modules corresponding to healthcare domains. Dedicated endpoints manage Patients, Practitioners, Encounters, and Episodes

of Care. A specialized "History Graph" module handles the retrieval and processing of graph visualization data.

Beneath the routing layer lies the service layer, which contains the core business logic. These services act as proxies to the FHIR server, transforming raw FHIR JSON into formats optimized for the frontend, such as the D3.js visualizations. They also handle document processing tasks, such as converting base64 strings to PDF files for binary storage.

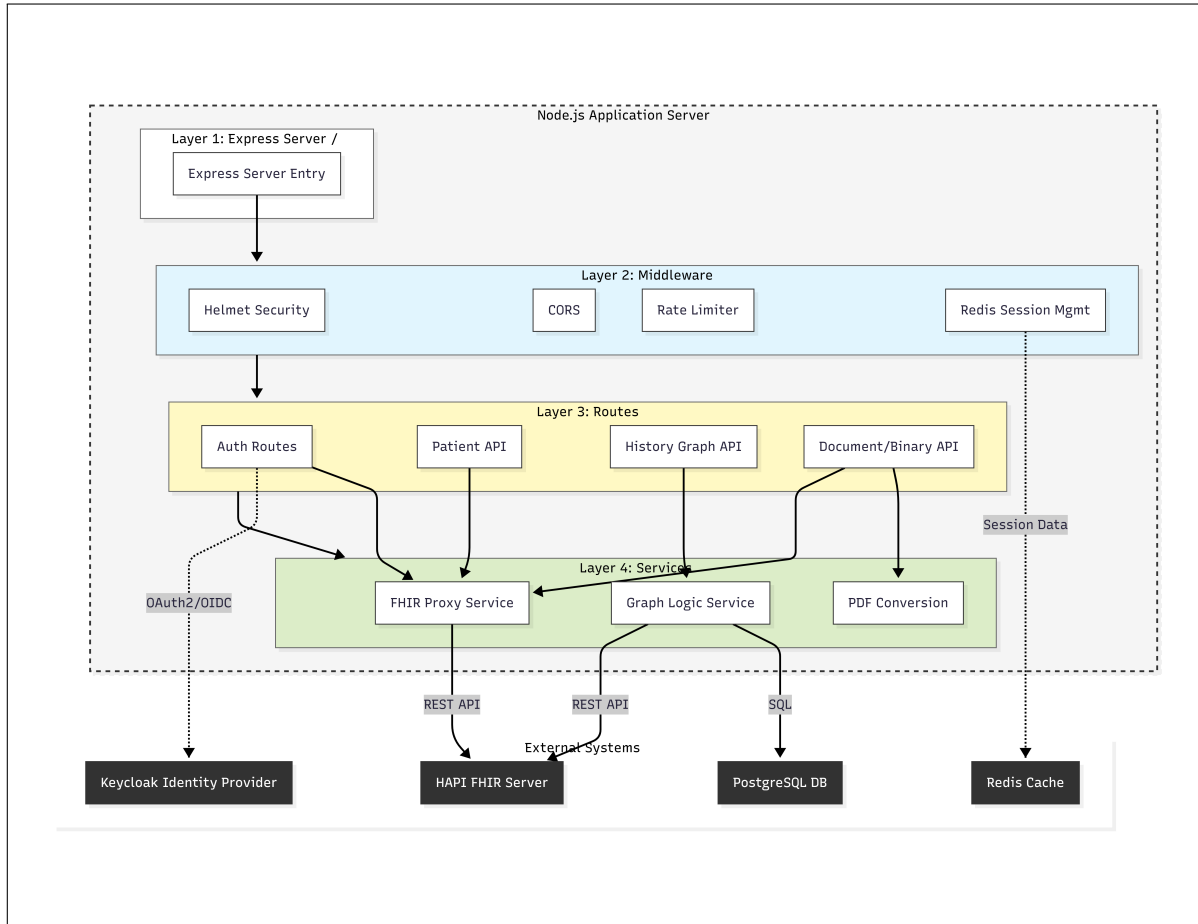


Figure 3.2: Internal layered architecture of the Node.js Application Server, detailing the middleware stack and service distribution.

3.1.4 Data Persistence and Schema

The system utilizes a polyglot persistence strategy, employing different database technologies for specific use cases to maximize performance.

Relational Data

The primary storage engine is PostgreSQL. It is divided into two logical segments. The first segment supports the HAPI FHIR server, storing all standard clinical resources

(like Patients and Observations) in a highly structured format managed by Hibernate. The second segment supports the Keycloak identity provider, storing user credentials and realm configurations.

Custom Graph Schema

To support the unique "Medical History Graph" feature, a custom schema was designed alongside the standard FHIR tables. This schema consists of `encounter_nodes` and `node_relations`. These tables allow the system to map standard FHIR Encounters into a Directed Acyclic Graph (DAG), enabling the visualization of a patient's journey through time with causal links between medical events.

Caching and Auditing

Redis is utilized for ephemeral data storage. One instance is dedicated to storing user sessions, ensuring fast authentication checks. A second Redis instance operates in Append-Only File (AOF) mode to store audit logs, providing a persistent and high-speed trail of system activities.

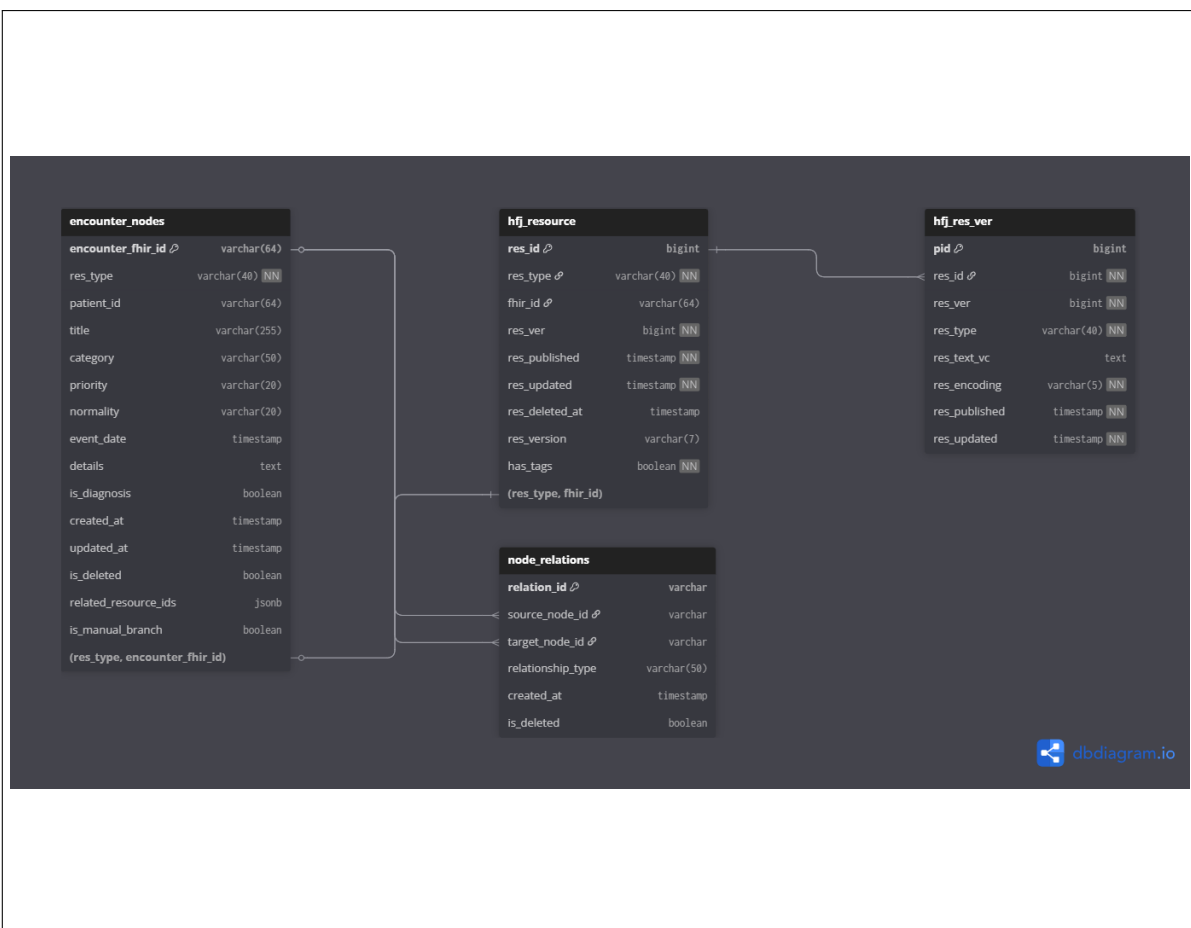


Figure 3.3: Entity-Relationship Diagram (ERD) illustrating the integration of custom graph tables with the standard FHIR resource table.

3.1.5 Request/Response Flow and Data Transformation Pipeline

Understanding how requests traverse the CFI-Care system is essential for comprehending the architecture’s design decisions. A typical request from the frontend follows a well-defined path through multiple layers, each contributing specific functionality: authentication verification, input validation, business logic execution, data transformation, and response formatting.

Request Ingress and Authentication

A request originates from the Angular frontend as an HTTPS request, typically including a Bearer token in the Authorization header. The request first reaches the Nginx reverse proxy, which acts as the external-facing gateway. Nginx terminates the TLS/SSL connection, decrypts the HTTPS payload, and forwards the unencrypted request to the Node.js server over the private Docker network. Before the request reaches application logic, it passes through the Helmet middleware, which applies security headers to prevent common web vulnerabilities such as XSS (Cross-Site Scripting), CSRF (Cross-Site Request Forgery), and clickjacking. The CORS middleware then validates that the request originates from an authorized frontend domain; requests from unauthorized origins are rejected immediately.

The next critical middleware validates the JWT token embedded in the Authorization header. The Node.js server maintains a cached set of public keys (JWKS) from Keycloak, updated periodically to handle key rotations. The token is cryptographically verified against these keys. If verification succeeds, the token claims (user ID, roles, permissions) are extracted and attached to the request context. If verification fails—due to an invalid signature, expired token, or missing token—the middleware returns a 401 Unauthorized response, and the request is rejected before reaching downstream services.

Concrete Example: Patient Data Retrieval

Consider a clinician’s request to retrieve a patient’s full medical history. The frontend issues a GET request to `/api/patients/:patientId`. After passing authentication and security middleware, the request reaches the routing layer, which dispatches it to the PatientService controller. The controller immediately implements input validation: the `patientId` parameter is checked to ensure it conforms to expected format (numeric ID or UUID, depending on the system design), and the authenticated user’s roles are verified to confirm they have permission to access that patient’s data (enforcing **HIPAA!** (HIPAA!) privacy principles).

Before querying the database, the controller checks the Tier 1 cache (managed by `cacheHelper`). The middleware generates a deterministic cache key from the request URL: `cache:patients:patientId:{id}`. If a valid cache entry exists, the cached patient data is immediately returned with a 200 OK response, bypassing all downstream processing. This cache hit path typically completes in 20-50 milliseconds.

If a cache miss occurs, the request proceeds to the PatientService layer. The service constructs a FHIR query and forwards it to the HAPI FHIR server via an HTTP call:

GET /fhir/Patient/{patientId}. The FHIR server retrieves the patient resource from PostgreSQL, performs any necessary transformations or computations required by the FHIR R5 standard, and returns the response in FHIR JSON format.

The PatientService receives the FHIR response and performs transformation operations: extracting relevant fields, formatting dates to match frontend expectations, computing derived values (e.g., age from birthdate), and sanitizing sensitive information if necessary. The transformed data is then stored in the Redis cache with a 1-hour TTL for future requests. Finally, the response is serialized to JSON and returned to the frontend with a 200 OK status code and appropriate cache headers to guide browser-side caching.

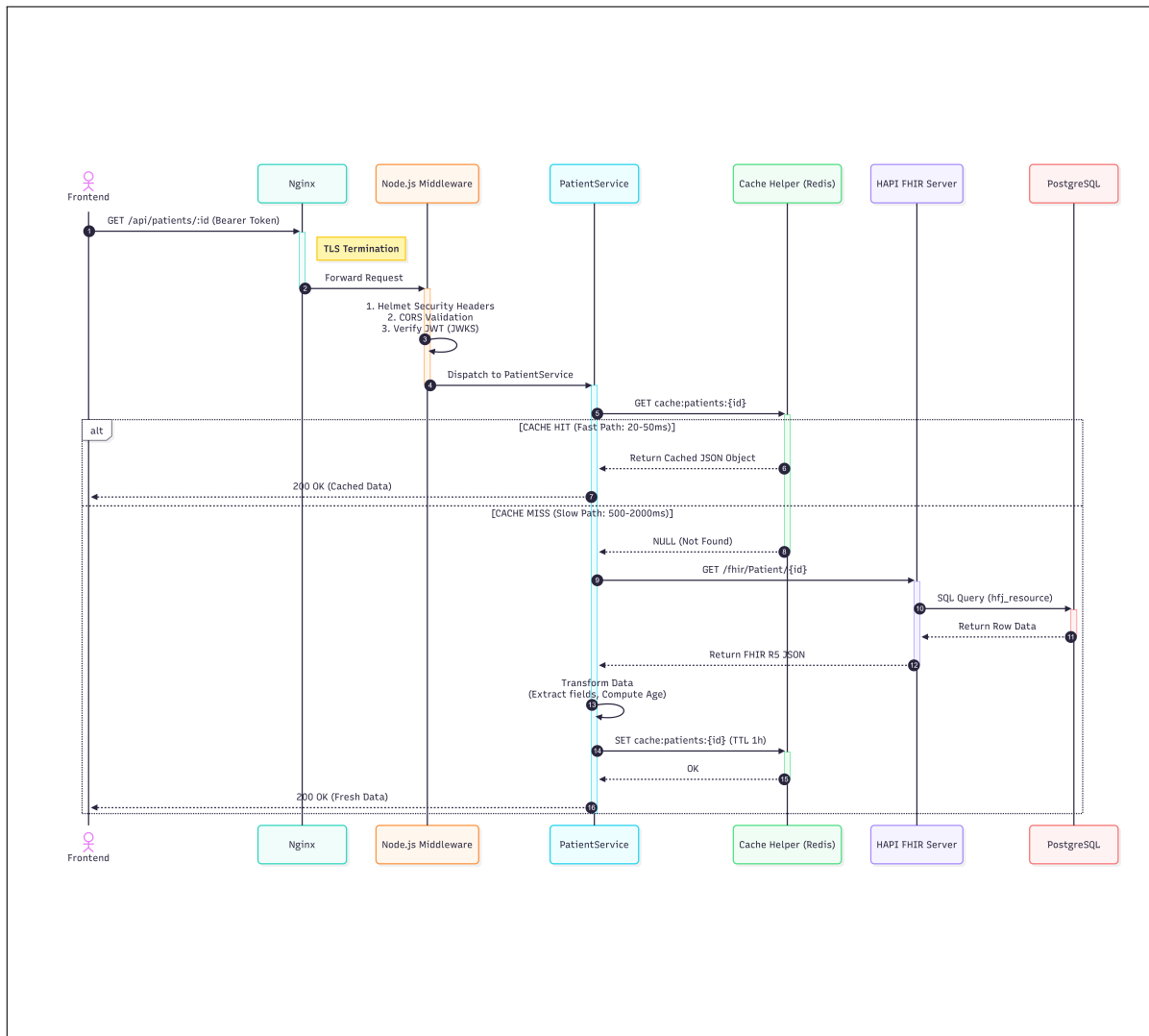


Figure 3.4: Request/Response flow for patient data retrieval, showing cache hit and miss scenarios with complete middleware pipeline.

Concrete Example: Encounter Creation and Graph Update

The encounter creation flow demonstrates more complex interactions across multiple system layers. When a clinician records a new consultation, the frontend issues a POST

request to `/api/encounters` with a JSON payload containing encounter details (patient ID, encounter type, date, notes, etc.).

The request undergoes the same initial security and authentication pipeline as before. Upon reaching the `EncounterService` controller, the request enters the input validation phase. The encounter data is validated against a schema ensuring that all required fields are present and correctly formatted. The encounter date is verified to be a valid date and logically consistent. The patient ID is validated to ensure the patient exists in the system.

Next, the `EncounterService` constructs a FHIR Encounter resource conforming to the FHIR R5 standard and submits a POST request to the HAPI FHIR server: `POST /fhir/Encounter`. The FHIR server validates the resource, generates a unique FHIR resource ID, persists the resource to PostgreSQL, and returns the created resource with its new ID.

The encounter creation triggers a cascade of downstream operations. The `EncounterService` receives the FHIR response and extracts the newly created encounter's ID. It then invokes the `historyGraphService`, passing the new encounter data. The `historyGraphService` performs a temporal scan of existing nodes for the patient, identifies potential parent nodes based on clinical rules and temporal windows, and creates appropriate edges in the `node_relations` table. This graph construction may take several hundred milliseconds to several seconds depending on the patient's history size.

Concurrently, multiple cache entries are invalidated. The patient-specific Tier 2 graph cache is cleared immediately, ensuring that the next graph visualization request triggers a fresh reconstruction. The Tier 1 cache entry for the `/api/patients/{patientId}` endpoint is also invalidated, forcing a fresh FHIR server query for updated patient data.

Finally, the `EncounterService` returns the newly created encounter resource to the frontend with a 201 Created status code. The response includes the encounter's FHIR ID, allowing the frontend to reference this specific encounter in subsequent requests.

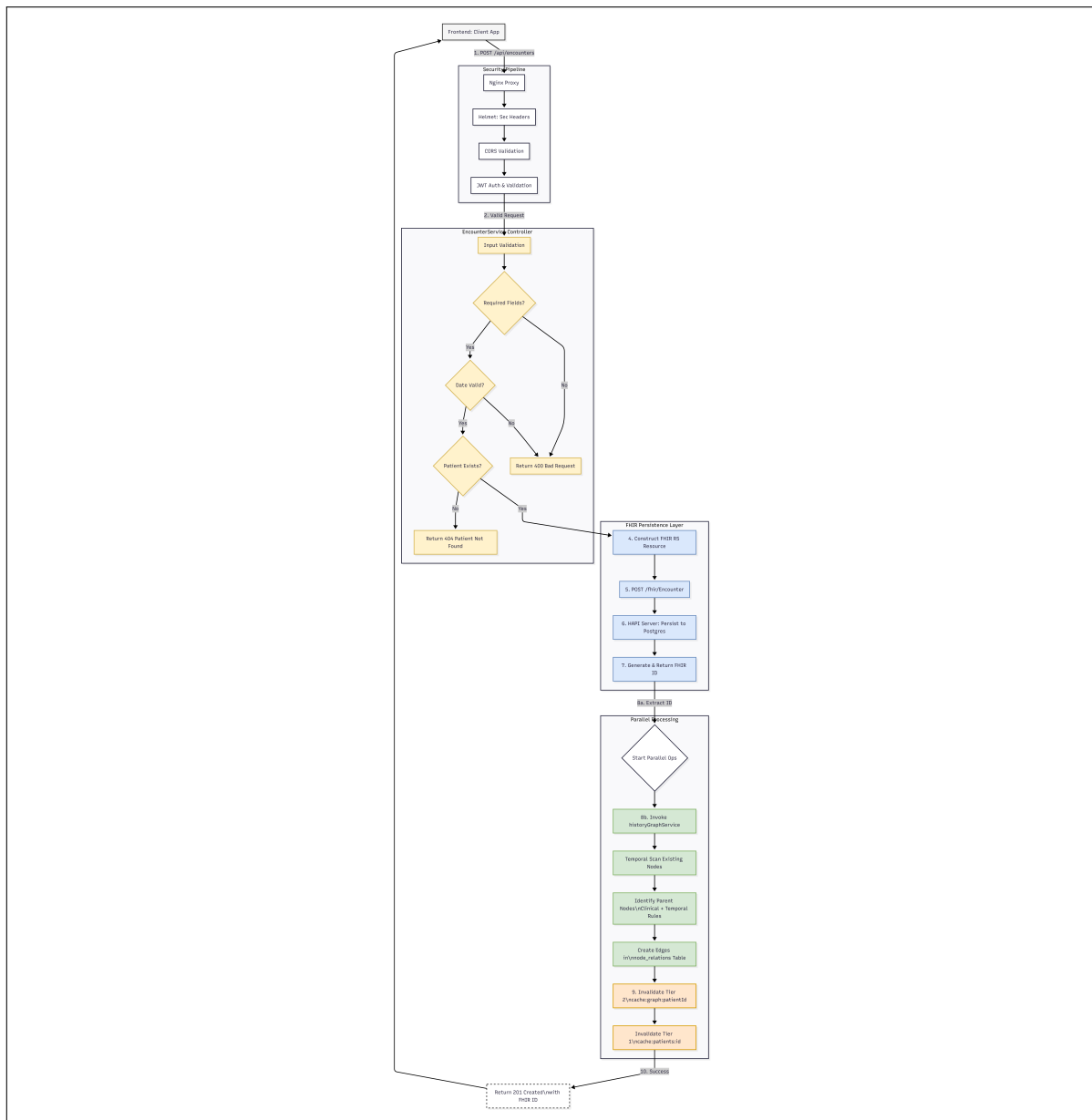


Figure 3.5: Complete encounter creation workflow showing FHIR resource persistence, graph construction, and cache invalidation cascade.

Validation Points and Error Conditions

Throughout this flow, validation occurs at multiple critical junctures. Input validation at the controller layer ensures that user-supplied data conforms to expected types and ranges. Business logic validation in the service layer verifies that operations are consistent with clinical rules. FHIR resource validation at the HAPI FHIR server ensures that resources conform to the FHIR R5 specification, preventing invalid data from being persisted.

Error conditions at any stage result in appropriate HTTP status codes and error messages: 400 Bad Request for input validation failures, 403 Forbidden for authorization

violations, 404 Not Found for missing resources, 409 Conflict for data consistency violations, and 500 Internal Server Error for unexpected server failures. Each error response includes a structured JSON object with a unique error code, human-readable message, and optionally, detailed information about the nature of the failure, enabling clinicians and developers to understand and address issues.

3.1.6 Error Handling and Fault Tolerance

The CFI-Care architecture recognizes that healthcare systems cannot tolerate downtime or data loss. Error handling and fault tolerance are designed into every layer to ensure reliable operation and graceful degradation under failure conditions.

Fallback Mechanisms and Graceful Degradation

When the FHIR server becomes temporarily unavailable, the CFI-Care system implements fallback strategies to maintain partial functionality. The caching layer becomes the primary data source: clinicians can still access patient records that were recently viewed, as these records remain in the Redis cache. While cached data may not reflect the absolute latest updates, it permits continued operation during brief FHIR server outages.

For the Patient History Graph, if the graph reconstruction fails due to FHIR server unavailability, the system returns a cached version of the graph if one exists, even if it is slightly stale. This allows clinicians to view previously computed graphs rather than receiving a complete failure. A notification is displayed to indicate that the graph may not reflect recent changes.

For new operations that require FHIR server access (creating encounters, adding diagnoses), if the FHIR server is unavailable, the system cannot proceed and returns a 503 Service Unavailable response, informing the clinician that the action cannot be completed at this moment. Rather than accepting incomplete or invalid data, the system fails safely, requiring the user to retry when the service recovers.

Logging and Observability for Error Diagnosis

All errors are logged with structured logging that includes timestamp, error level (ERROR, WARN, INFO), service component generating the error, error code and message, request context (user ID, patient ID, operation being performed), and stack trace for unexpected exceptions. This structured logging enables rapid diagnosis of issues: operators can search logs by error code to identify all instances of a particular failure, by user ID to understand a user's experience during an outage, or by component to focus on specific services.

Critical errors are propagated to alerting systems, enabling operators to be notified of failures in real-time. For instance, if the FHIR server becomes unavailable, an alert is triggered within seconds. If error rates exceed thresholds, alerts notify operators that the system is degrading. This proactive monitoring enables rapid response to emerging issues before they impact clinicians' ability to deliver care.

3.1.7 Security and Infrastructure Layer

The infrastructure is secured through a centralized Identity and Access Management (IAM) system using Keycloak. Keycloak handles all OAuth 2.0 and OpenID Connect flows, issuing JSON Web Tokens (JWT) that the Node.js backend validates against a public key set (JWKS).

All components are containerized using Docker and orchestrated via Docker Compose. The environment is isolated within a private bridge network, with Nginx acting as the sole entry point. Nginx enforces SSL/TLS encryption using locally generated certificates, ensuring that all data in transit is encrypted. Containers are configured with read-only file system mounts and strict privilege limitations to minimize the security attack surface.

3.1.8 HAPI FHIR Server Implementation

The core component responsible for standardizing healthcare data within the CFI-Care system is the FHIR Server. This server is implemented using the HAPI FHIR JPA Server Starter framework. It is built upon the Java Spring Boot ecosystem and provides a fully compliant implementation of the FHIR Release 5 (R5) standard. This choice ensures that all clinical data, regardless of its source, adheres to international interoperability protocols and persists reliably within the database.

To tailor the server for the specific needs of this project, several important configuration changes were applied to the default setup. To maintain a lightweight and efficient runtime, advanced features that were not immediately required were explicitly disabled. This includes the Clinical Reasoning module and the CDS Hooks service. Similarly, bulk export and import operations were deactivated to prevent unnecessary resource consumption. Conversely, the OpenAPI documentation endpoints were enabled. This configuration allows developers to inspect the API structure and test endpoints directly through a web interface.

The persistence layer is powered by a PostgreSQL database connected via the Hibernate JPA engine. Specific configurations were applied to the Hibernate settings to optimize performance. The system uses the dedicated `HapiFhirPostgresDialect` to ensure efficient SQL generation. Additionally, the database schema management is configured to automatically update the table structures upon server startup, ensuring that the database remains synchronized with the code.

The server is responsible for managing a wide array of clinical resources necessary for the application. These include foundational entities such as Patient and Practitioner, as well as clinical events like Encounters, Procedures, and Observations. It also handles complex data types, including MedicationRequests for prescriptions and Binary resources for storing documents like PDF files.

For deployment, the server utilizes a secure multi-stage Docker build process. The first stage uses Maven to compile the application code and generate the necessary artifacts. The final runtime image is built upon a "distroless" Java 17 base image. This approach significantly reduces the final container size and improves security by removing unnecessary operating system tools and shells from the production environment.

3.2 API Design and Endpoints

The communication between the frontend client and the backend server is established through a structured RESTful API. This design ensures that the system remains stateless and scalable, utilizing standard HTTP methods to perform operations on resources. The Node.js application server acts as the centralized gateway, exposing a unified set of endpoints that handle both standard clinical data and custom graph logic.

3.2.1 RESTful Structure and Formats

All API interactions strictly adhere to the Javascript Object Notation (JSON) format for both request bodies and response payloads. The system utilizes standard HTTP verbs to define the nature of the operation. GET requests are used exclusively for retrieving data without side effects, while POST requests handle the creation of new resources. PUT methods are employed for updating existing records, and DELETE methods remove specific resources when permitted.

Responses from the server are standardized to ensure predictability for the client application. Successful operations return a 200 OK status code accompanied by the requested data object. In cases of resource creation, a 201 Created status is returned. Error handling is centralized, ensuring that any failure creates a consistent JSON error object containing a human-readable message and a specific error code, rather than exposing raw server stack traces.

3.2.2 FHIR-Compliant Endpoints

A significant portion of the API is dedicated to proxying standard FHIR resources. Endpoints such as `/api/patients` and `/api/encounters` act as a validation and caching layer sitting in front of the HAPI FHIR server. When the frontend requests patient details, the Node.js server intercepts this call, validates the parameters, and then forwards the request to the underlying Java server. The response is strictly compliant with the FHIR R5 standard, ensuring that the data structure remains interoperable with other external healthcare systems.

3.2.3 Custom History Graph Endpoints

Beyond standard FHIR resources, the API exposes specialized endpoints designed to support the Patient History Graph visualization. The primary endpoint, `/api/historyGraph`, does not return a simple list of records. Instead, it aggregates data from multiple FHIR resources and transforms them into a graph structure containing nodes and relational edges. This allows the frontend to render the complex temporal relationships between medical events without needing to perform heavy data processing in the browser.

Method	Endpoint	Description
GET	/api/patients	Retrieve list of patients with pagination support
POST	/api/patients	Register a new patient with FHIR validation
GET	/api/encounters/:id	Get specific encounter details including clinical observations
GET	/api/historyGraph/:id	Get patient history graph nodes and causal relationships
POST	/auth/login	Authenticate user and obtain JWT token via Keycloak

Table 3.1: Key REST API Endpoints provided by the Node.js Backend.

3.3 Patient History Graph Implementation

A core distinguishing feature of the CFI-Care backend is the Patient History Graph, which visualizes a patient’s medical journey as a Directed Acyclic Graph (DAG). Unlike standard FHIR lists, which present data linearly, this graph models causal and temporal relationships between medical events. This representation provides clinicians with a sophisticated tool for understanding the evolution of a patient’s condition, particularly in complex cases involving multiple concurrent diagnoses or extended treatment histories.

3.3.1 Motivation: Beyond Linear Medical Records

Traditional Electronic Health Records (**EHR!** (**EHR!**)s) typically present patient data as chronologically ordered events: a consultation on January 1st, a lab test on January 3rd, a prescription on January 5th, and so forth. While this linear representation is straightforward to implement and understand, it obscures the causal relationships that are essential to clinical reasoning. Consider a patient with Type 2 Diabetes presenting with hypertension and elevated cholesterol levels. In a linear record, these appear as separate events without explicit connection. However, a clinician recognizes that these conditions are causally related—elevated cholesterol and hypertension may be comorbidities stemming from the underlying metabolic dysfunction of diabetes.

The Patient History Graph addresses this limitation by explicitly modeling causality and temporal dependencies. Each medical event is represented as a node in the graph, and directed edges represent causal or temporal relationships. This graphical representation allows clinicians to trace the progression of a specific condition across multiple encounters and interventions, identify temporal clusters of related events that may indicate disease progression or treatment response, visualize the impact of therapeutic interventions on subsequent clinical outcomes, and differentiate between independent medical episodes

and related sequences of events. By making these relationships explicit, the graph transforms raw temporal sequences into clinically meaningful narratives that support more effective decision-making.

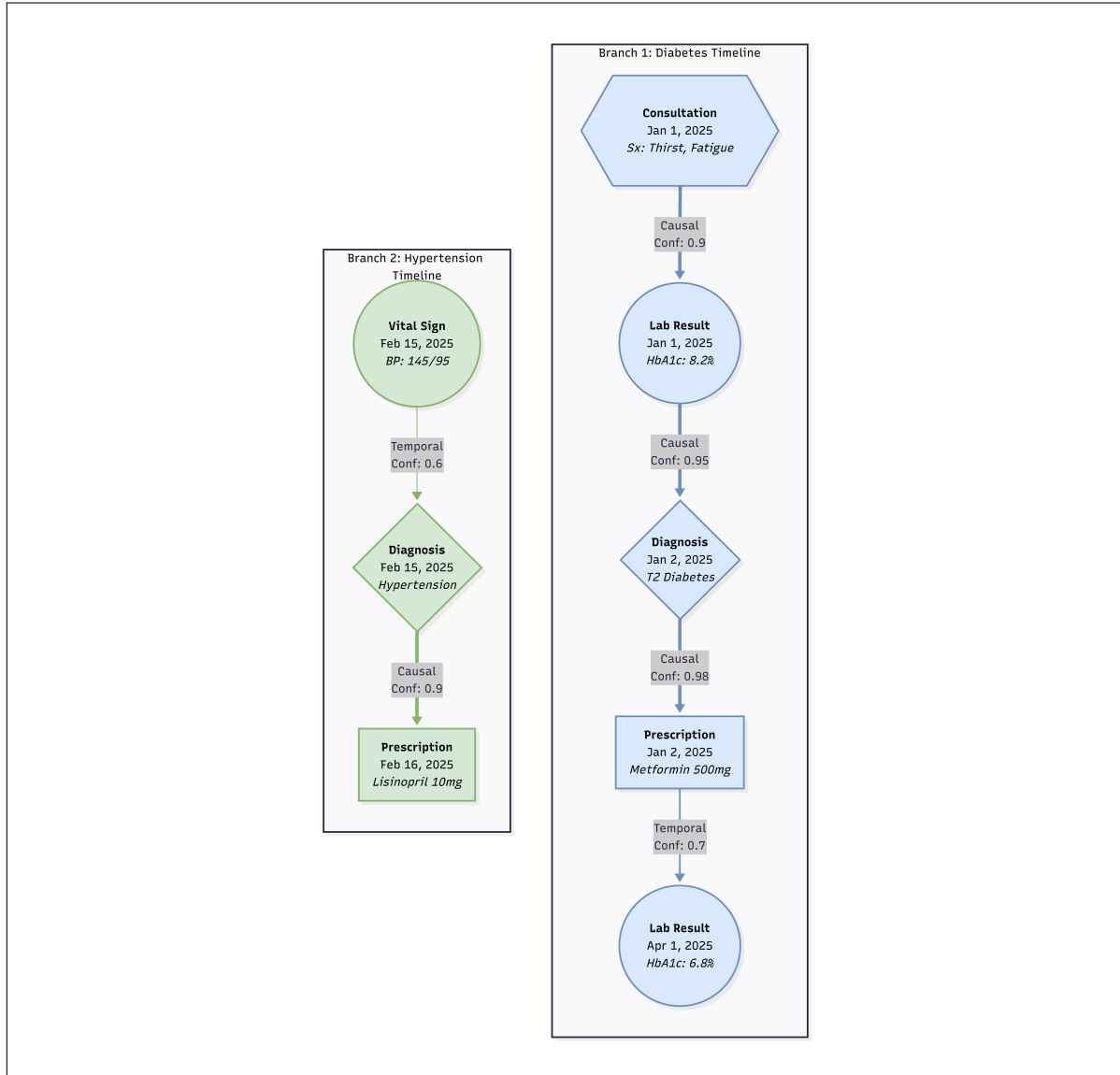


Figure 3.6: Example Patient History Graph showing a diabetes patient with concurrent hypertension, demonstrating temporal branching and multiple relationship types.

3.3.2 Graph Data Model

The Patient History Graph is built upon a relational foundation with two primary tables: `encounter_nodes` and `node_relations`.

Node Representation

Each node in the graph represents a distinct clinical event or observation. The system recognizes six primary node types: **Consultation** represents a clinical encounter between

patient and clinician; **Diagnosis** denotes a confirmed medical condition recorded during or after a consultation; **Prescription** indicates a medication prescribed to address a specific condition; **LabResult** captures laboratory or diagnostic test results; **Vital** encompasses blood pressure, temperature, and other vital signs; and **Symptom** represents patient-reported subjective findings.

Each node stores rich metadata including its creation timestamp, an associated FHIR resource ID for traceability, its clinical category, and the patient to which it belongs. A fundamental design principle of the Patient History Graph is immutability: nodes, once created, cannot be modified or deleted. This architectural choice ensures that the graph preserves a complete and auditable record of the patient's medical journey, enabling clinicians to understand not just what happened, but also when records were created and by whom.

Edge Representation

Edges in the graph represent causal or temporal relationships. An edge from node *A* to node *B* indicates that event *A* preceded and potentially caused or informed event *B*. The system maintains an explicit `node_relations` table storing parent-child pairs, enabling efficient graph traversal for visualization and analysis.

3.3.3 Temporal Branching and Concurrent Episodes

Real-world patient histories are rarely linear. A patient may have multiple concurrent health issues being managed in parallel. For example, a patient may simultaneously receive treatment for diabetes, hypertension, and arthritis. In a traditional linear **EHR!**, this complexity is flattened; all events appear chronologically without indication of which episode they belong to.

The CFI-Care graph model handles this through an explicit branching mechanism:

Head Node Concept

A "Head Node" is a node with no outgoing edges to other nodes in the current timestamp window. When a new clinical event arrives that does not fit within any existing causal chain (i.e., no valid parent is found), the system creates a new Head Node, effectively initiating a new branch in the history graph.

For example, consider a patient's history:

1. Consultation for chest pain (2025-01-01)
2. EKG test (2025-01-01)
3. Cardiology referral (2025-01-02)
4. New consultation for joint pain (2025-02-15) — **New Branch**

5. X-ray of knee (2025-02-15)
6. Rheumatology referral (2025-02-16)

The system recognizes that the joint pain consultation on 2025-02-15 cannot be causally linked to the earlier cardiology events (temporal gap exceeds threshold, clinical context is unrelated). Therefore, it creates a new Head Node for the rheumatology branch, allowing clinicians to visually separate the two episodes.

Visual Implications

From a visualization perspective, branching creates a multi-root forest structure (multiple separate trees within the graph). The frontend rendering engine positions these branches side-by-side or in hierarchical layout, enabling clinicians to track multiple concurrent conditions without visual clutter or confusion.

Clinical Advantages

Explicit branching provides significant clinical benefits that improve the utility of the Patient History Graph. Clinicians immediately recognize which events belong to which condition through visual separation, substantially reducing cognitive load compared to linearly ordered records. The system can provide condition-specific recommendations by analyzing the isolated branch rather than considering the entire patient history, enabling more targeted and relevant clinical decision support. Additionally, each branch can be summarized independently for specialist-to-specialist referrals, allowing specialists to focus on the specific problem domain without irrelevant context from other concurrent conditions.

3.3.4 Relationship Classification

Not all edges in the Patient History Graph carry the same semantic weight or clinical significance. The system classifies relationships into three distinct types to preserve this important distinction. Causal relationships represent situations where Node *A* directly caused or necessitated Node *B*. For example, a Diabetes Diagnosis directly necessitates a Metformin Prescription. These edges carry high confidence scores and strong clinical significance, indicating a clear cause-and-effect relationship. Temporal relationships occur when Node *A* preceded Node *B* within the same clinical episode, but causality is not explicitly established. For instance, a Consultation followed by a Blood Pressure Reading taken during the same visit represents a temporal relationship rather than a causal one. Associative relationships capture scenarios where Node *A* and Node *B* are related through shared clinical context—such as both addressing the same underlying condition—but neither directly causes the other. For example, a High Cholesterol Lab Result and a Diabetes Diagnosis both present in the same patient might be associated through their shared metabolic dysfunction.

Each relationship in the graph stores a confidence score ranging from 0.0 to 1.0, generated by the construction algorithm based on how well the relationship satisfies the various constraint criteria. High-confidence relationships with scores exceeding 0.8 are displayed prominently in the visualization, while lower-confidence relationships are shown with reduced visual emphasis or as optional links that the clinician can explore if desired.

3.3.5 Handling Edge Cases

The Patient History Graph implementation must gracefully handle several challenging scenarios that arise in real-world clinical practice. Retroactive diagnosis occurs when a patient receives a diagnosis that applies to earlier symptoms after those symptoms have already been recorded. For example, a patient might experience chest pain on January 1st and only receive a diagnosis of coronary artery disease on March 1st. The algorithm detects this temporal mismatch and retroactively relinks the symptom node to the new diagnosis, maintaining causal clarity and preventing the false impression that the symptom was unexplained.

Conflicting information presents another challenge. When new FHIR data contradicts previously recorded information—such as a diagnosis being changed, retracted, or corrected—the system must handle this gracefully. Rather than deleting nodes (which would violate the immutability principle and destroy the audit trail), the system creates a new node representing the correction and links it appropriately to existing nodes. This approach preserves the complete historical record while accurately reflecting the current clinical understanding.

Circular references represent a potential structural problem. Although directed acyclic graphs (DAGs) cannot, by definition, contain cycles, the algorithm must validate that no accidental cycles are introduced during edge creation. This validation is enforced through topological sorting checks performed before persisting relationships to the database, preventing data consistency issues that could arise from cycles.

3.3.6 Query Patterns and Optimization

The frontend visualization layer requires efficient access to the graph structure through several common query patterns, each optimized for specific clinical use cases. Full Graph Retrieval fetches all nodes and edges for a patient, which is necessary during the initial page load when clinicians first view a patient's complete medical history. This operation is optimized through caching strategies discussed in Section 3.4. Ancestor Traversal answers questions such as "What led to this prescription?" by fetching all ancestor nodes of a specific event. This pattern is implemented via recursive queries that are bounded by depth to prevent unbounded traversals. Branch Isolation allows clinicians to focus on specific medical episodes, such as "Show only cardiology-related events," by fetching all nodes within a specific branch. This operation uses topological partitioning to efficiently isolate branches without scanning the entire graph. Temporal Range Queries retrieve nodes within a specific date range, enabling clinicians to analyze disease progression

over a specific time period. These queries leverage indexed timestamp columns to achieve $O(\log n)$ lookup time complexity, ensuring responsiveness even for patients with extensive histories.

3.4 Performance Optimization

To ensure low-latency responses for the frontend visualizations, the backend implements a specialized Two-Tier Caching Strategy using Redis. This hierarchical caching approach balances data freshness with response times, recognizing that different types of medical data have different update frequencies and computational costs.

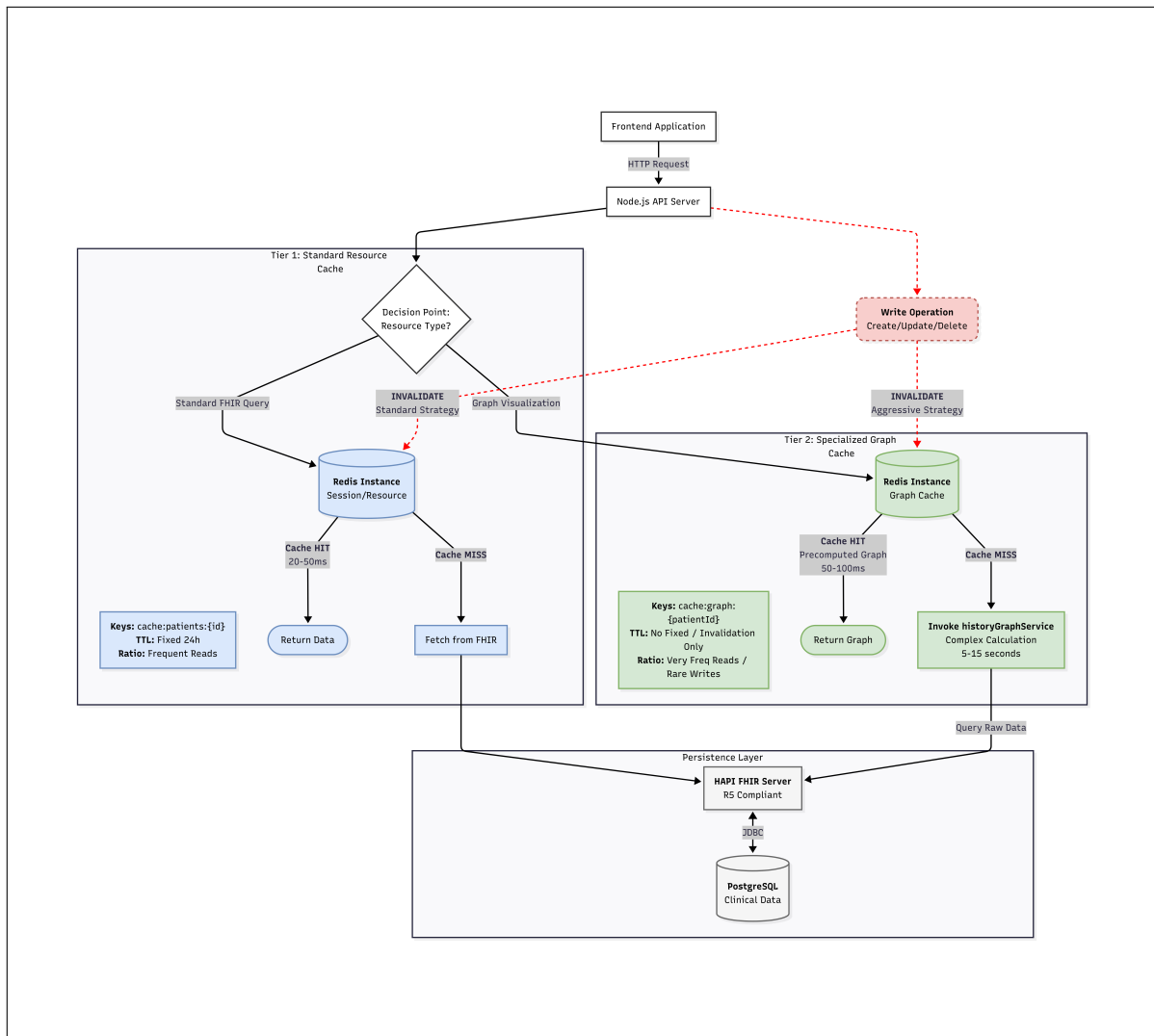


Figure 3.7: Two-Tier Caching Architecture showing distinct strategies for standard FHIR resources (Tier 1) and complex graph structures (Tier 2).

3.4.1 Tier 1: Standard Resource Cache

The first tier utilizes a generic `cacheHelper` middleware that intercepts HTTPS requests for standard FHIR resources. This middleware operates at the request level, functioning as a transparent cache layer between the frontend client and the backend processing pipeline.

Mechanism and Interception

When an HTTPS request arrives at the backend (such as a GET request to `/api/patients`), the `cacheHelper` middleware performs a sequence of operations to determine whether a cached response can be served. The middleware first generates a deterministic cache key from the request URL and query parameters—for example, a request to `/api/patients?page=1&limit=20` generates the key `cache:patients:page:1:limit:20`. The middleware then queries the Redis cache for an entry matching this key. If a valid cache entry exists, meaning the entry has not expired, the cached response is immediately returned to the client, entirely bypassing the FHIR server, database query execution, and all downstream processing. This direct cache-to-client path reduces latency from potentially 500-2000ms (database query time) to sub-50ms (Redis in-memory retrieval). If no entry exists or the entry has expired, the request proceeds through the normal processing pipeline: the backend queries the FHIR server, processes the response, and returns it to the client. Before returning, the response is stored in Redis for future requests, repopulating the cache.

Time-To-Live (TTL) Management

Cached FHIR resources are assigned a default Time-To-Live (TTL) of 1 hour, representing a clinically acceptable window where cached data is considered sufficiently fresh for clinical use. This 1-hour window is chosen based on several practical considerations. From a clinical workflow perspective, most healthcare workflows operate on new fresh data every hour. A patient's medication list, diagnosis history, and vital signs are statistically unlikely to change multiple times within a single hour in most clinical scenarios. From a system load perspective, a 1-hour TTL significantly reduces the query volume to the underlying FHIR server and database, minimizing computational load during peak usage hours when many clinicians are accessing patient data simultaneously. While the 1-hour window does introduce some degree of data staleness, this tradeoff is clinically acceptable for read-heavy operations on relatively stable data that changes infrequently.

However, the TTL is configurable on a per-resource-type basis to accommodate different data freshness requirements. Real-time vital signs or active medication adjustments might use a shorter TTL of one hour to ensure clinicians see recent changes, while historical diagnoses that rarely change might use a longer TTL of 12 hours to reduce computational overhead.

Cache Invalidation for Tier 1

Although Tier 1 uses a fixed TTL-based expiration, certain write operations trigger explicit cache invalidation to ensure data consistency. When a patient record is modified—such as contact information changes or new allergies being recorded—the corresponding cache entries for that patient are immediately removed, forcing a fresh fetch on the next request. Similarly, when a new encounter is recorded or an existing one is updated, the cache entries for `/api/encounters` are invalidated to reflect the changes. The system maintains a comprehensive mapping of dependencies between cache keys and FHIR resources. When a resource is modified, all dependent cache entries are invalidated simultaneously, preventing scenarios where multiple cache entries contain stale references to the same resource.

Practical Impact

The practical benefit of Tier 1 caching becomes apparent in typical clinic workflows where healthcare providers request the same patient’s record multiple times during a single day. Without caching, each request would trigger a database query taking 500-1000ms, and checking the same patient 5 times would incur 2500-5000ms of cumulative latency. With Tier 1 caching in place, the first request incurs the full latency of 500-1000ms, but subsequent requests within 24 hours are served from cache in approximately 20-50ms each. For five checks of the same patient, the total latency becomes 500-1000ms plus 80-200ms for the cached requests, totaling 580-1200ms. This represents a 75-80 percent reduction in latency, dramatically improving the responsiveness of the system and reducing clinician wait times during patient data retrieval.

3.4.2 Tier 2: Specialized Graph Cache

The second tier is specifically designed for the computationally intensive Patient History Graph. While Tier 1 caches simple resource retrieval, Tier 2 caches the results of complex graph construction and relationship mapping.

Computational Cost of Graph Construction

Constructing a Patient History Graph from raw FHIR resources is computationally expensive for several interconnected reasons. Data aggregation requires retrieving all FHIR resources for a patient, including consultations, diagnoses, prescriptions, lab results, vitals, and symptoms, potentially from multiple FHIR repositories. A patient with an extensive medical history might have hundreds of such resources, necessitating multiple server calls and database queries. Relationship inference constitutes the most computationally intensive phase. For each new node, the algorithm must scan all existing nodes for that patient, evaluate whether causal relationships should exist by checking temporal windows, clinical context compatibility, and event type rules, compute confidence scores for each potential relationship, and determine the most relevant parent. This represents an $O(n)$ operation where n is the number of existing nodes, and complex

patients with 500+ nodes can require significant computational resources. Graph formatting transforms the raw graph structure into a visualization-ready format suitable for the frontend rendering library. This transformation includes computing optimal node positions in the visual layout, determining visual attributes such as color, size, and emphasis, serializing the entire structure to JSON, and applying any additional formatting for the specific frontend framework. For a patient with 500 nodes, these cumulative operations can collectively require 5-15 seconds of computational time—far too slow for the responsive interactive experience that clinicians expect.

The Graph Cache Manager: `toonNodesCacheHelper`

The `toonNodesCacheHelper` service manages caching specifically for the Patient History Graph. Unlike the generic Tier 1 cache which uses simple TTL expiration, the graph cache is patient-specific and extensively optimized for the unique characteristics of graph data. Each patient has a dedicated cache entry with the key `cache:graph:{patientId}`, containing their complete precomputed graph structure. The cache value stores a comprehensive JSON object containing all nodes with their associated metadata (type, timestamp, clinical details), all edges with their relationship types and confidence scores, and precomputed visual attributes including node positions and branch assignments. Unlike Tier 1 caching which relies on fixed TTL expiration, the graph cache does not use time-based expiration at all. Instead, it relies entirely on explicit invalidation to ensure that cached data always reflects the current patient history.

Aggressive Invalidation Strategy

The graph cache employs an aggressive invalidation approach that prioritizes data consistency over computational efficiency. Any write operation to a patient’s medical history immediately invalidates their corresponding graph cache entry. This design reflects a fundamental tradeoff in the caching strategy: write operations are relatively rare compared to read operations in clinical workflows, while read operations dominate. A single patient might have their graph read hundreds of times per day across multiple clinicians viewing the same patient from different workstations or different clinical contexts, yet the patient’s medical history is modified only a few times per day. Given this asymmetry, it is computationally more efficient to invalidate the cache upon writes (an immediate operation) than to maintain cache staleness during reads or to implement complex update-in-place strategies. The invalidation is triggered by several categories of events: recording a new clinical event such as a consultation, diagnosis, prescription, or lab result; updating or correcting an existing node; modifying relationships between nodes; or retracting or changing a diagnosis or medication. Any of these operations immediately removes the cached graph, forcing reconstruction on the next request.

Cache Reconstruction Workflow

When a graph cache miss occurs, either due to invalidation from a write operation or because the cache has never been populated for that patient, a well-defined reconstruction workflow is executed. The frontend or backend first requests the graph

for the patient. The `toonNodesCacheHelper` detects the cache miss and invokes the `historyGraphService` to reconstruct the graph from scratch. The reconstruction process retrieves all FHIR resources for the patient, executes the complete graph construction algorithm including temporal scanning and parent identification, and computes visual attributes. Once complete, the entire graph structure is serialized to JSON and stored in Redis. The fully reconstructed cache is then returned to the requesting client. Subsequent requests within the same session or from other clinicians viewing the same patient retrieve the cached graph in sub-100ms latency, providing responsive interactive performance.

Hit Rate and Performance Impact

The effectiveness of the graph cache in improving system performance depends heavily on the read-to-write ratio, which varies depending on the clinical workflow pattern. In read-heavy workflows, which represent the typical clinical use case, a clinician views a patient's graph multiple times during a consultation, compares it with previous snapshots to track disease progression, and discusses findings with colleagues or consulting specialists. The same graph may be read 10-20 times before any modification to the patient record occurs. In these scenarios, the graph cache achieves hit rates of 85-95 percent, resulting in dramatic performance improvements. A graph that would normally require 15-20 seconds of computational time to construct is served from cache in 50-100ms, a 100-200x latency reduction. In contrast, write-heavy workflows are rare but do occur, such as when a clinician is entering comprehensive historical data for a newly-registered patient, recording 20-30 events that generate new medical events in quick succession. Each new event triggers cache invalidation, causing multiple cache misses and requiring full reconstructions. In these scenarios, the cache hit rate drops to 20-30 percent. The aggressive invalidation strategy in this case sacrifices caching efficiency to ensure that data consistency is always maintained. In typical clinical workflows, the combination of read-heavy access patterns and Tier 2 caching provides a 100-200x latency improvement for graph visualization, translating to substantially more responsive clinical interfaces and reduced clinician wait times.

Coordination Between Tier 1 and Tier 2

The two caching tiers work in close coordination to maximize performance across different types of requests. When the frontend requests standard FHIR resources such as a patient list or individual patient demographic data, Tier 1 cache is checked first, providing rapid responses for frequently accessed basic information. When the frontend requests the complex Patient History Graph visualization, Tier 2 cache is consulted, serving the precomputed graph structure with all relationships and visual attributes already prepared. Invalidation operations propagate across both tiers simultaneously. For example, recording a new prescription invalidates both the Tier 1 cache entry for `/api/prescriptions` (which would now be stale relative to the 24-hour TTL) and the Tier 2 cache for that patient's history graph (which must be reconstructed to include the new node and relationships). This coordinated invalidation ensures that regardless of which API endpoint a clinician accesses, the data reflects the most recent state of the patient's record.

Memory Management and Cache Eviction

Redis is configured with a maximum memory limit to prevent unbounded growth and to integrate into production server resource constraints. To manage memory efficiently, the CFI-Care backend employs an LRU (Least Recently Used) eviction policy. When Redis approaches its configured maximum memory capacity, the system automatically evicts the least recently accessed cache entries to make space for new data. This dynamic approach to cache management creates a natural working set where the cache automatically retains entries for patients with active clinical workflows (frequently accessed by multiple clinicians) while entries for inactive patients are evicted and subsequently reconstructed on-demand if and when those patients are accessed again. This ensures that memory usage remains bounded and predictable while simultaneously maximizing cache hit rates for the active subset of patients. In practice, this means that high-volume clinical workflows benefit from maximum cache coverage, while less frequently accessed patients gracefully degrade to on-demand reconstruction without requiring administrator intervention or manual cache management.

3.5 Frontend

3.5.1 Overview and Design Objectives

The frontend of *CFI-Care* was designed as an interactive medical information system aimed at enabling healthcare professionals to efficiently explore, manage, and reason about patient data. The primary objective was to transform fragmented medical records into a coherent, visual, and navigable representation of a patient’s medical journey, while maintaining clinical clarity, responsiveness, and usability across the full scope of the application.

The frontend follows a component-based architecture implemented using the Angular framework. Advanced data visualisation is achieved through the integration of D3.js and Treemap, while Bootstrap and Bootstrap Icons provide responsive layout, visual consistency, and accessibility across devices.

The main design goals of the frontend include:

- Clear separation of concerns between navigation, visualisation, and data manipulation.
- Visual prioritisation of clinically critical events through deliberate colour coding and animated indicators.
- Support for both categorical and temporal reasoning over patient medical data through two complementary visualisation modes.
- A patient-access grant workflow that balances security requirements with a smooth, low-friction clinical experience.

- A fully integrated AI-assisted chat interface that enables real-time clinical reasoning alongside patient data.

Component-based frontend architectures have been shown to improve scalability, maintainability, and development efficiency in large-scale systems [?].

3.5.2 Technology Stack

Table 3.2 summarises the main technologies used in the frontend implementation.

Table 3.2: Frontend Technology Stack

Technology	Purpose
Angular 17	Core frontend framework, routing, and state management
D3.js (v7)	Data-driven visualisation and interaction logic
Treeviz	Hierarchical graph layout abstraction
Bootstrap	Responsive layout and UI consistency
Bootstrap Icons	Visual indicators and interaction affordances
RxJS	Reactive streams for async data flow and polling
ng-otp-input	Six-digit OTP input component for grant handshake

Angular’s declarative templates and reactive change-detection model make it suitable for highly interactive, state-driven medical interfaces [?]. RxJS operators such as `interval`, `switchMap`, and `takeWhile` are used to implement the patient-access grant polling loop, avoiding manual timer management and ensuring automatic teardown on component destruction.

3.5.3 System Architecture

The CFI-Care frontend is structured as a modular, component-based single-page application (SPA) built with Angular. The system follows a layered frontend architecture that cleanly separates presentation, state management, domain models, and visualisation logic.

At a high level, the architecture consists of the following layers:

- **Application Shell:** Responsible for route registration, authentication guards, and top-level layout composition.
- **Feature Modules:** Self-contained UI modules grouped by clinical concern, covering the patient dashboard, doctor profile and schedule management, grant management screens, and the AI chat interface.
- **MedGraph Visualisation Module:** A dedicated visualisation boundary hosting both the categorical tree and the chronological timeline, isolated from Angular’s change-detection loop via an `iframe` boundary to prevent rendering interference.

- **Shared Domain Models and Services:** Centralised TypeScript interfaces and injectable Angular services that carry application state and mediate all HTTP communication with the backend.
- **Graph Rendering Engine:** A self-contained HTML file (loaded inside an iframe) that combines Treeviz, D3.js, and vanilla JavaScript to render interactive patient journey graphs, communicating with the Angular host via the `postMessage` API.

Route Structure and Navigation Guards

The application defines the following client-side routes:

Table 3.3: Application Route Configuration

Path	Component	Guard
/	Redirect to /dashboard	—
/dashboard	Dashboard	authGuard
/med-graph/:id	MedGraph	authGuard
/chat-section	ChatSection	authGuard
/doctor-profile	DoctorProfileComponent	authGuard
/grants/pending	GrantPendingComponent	authGuard
/grants/active	GrantActiveComponent	authGuard
/error/:status	GatewayErrorComponent	—

All clinically sensitive routes are protected by `authGuard`, an Angular route guard that verifies session validity before permitting navigation. Unauthenticated users are redirected to the login flow. The `/error/:status` route accepts a dynamic HTTP status code and renders a contextual error page, handling backend or gateway failures gracefully without exposing raw error traces to the user.

The `/med-graph/:id` route accepts a patient identifier as a path parameter and an optional `eocId` query parameter, enabling direct deep-linking to a specific episode of care’s visualisation.

Component Hierarchy

Figure 3.26 illustrates the full component tree. The `Dashboard` component acts as a layout compositor, combining three child components: `TopBar` (global navigation and account controls), `UsersPanel` (patient search, filter, and list), and `UserInfo` (detailed clinical summary for the selected patient). The `MedGraph` component operates independently, receiving its patient context from route parameters and communicating with the embedded iframe via `postMessage`.

3.5.4 Styling and UX Philosophy

The CFI-Care frontend adopts a *High-Contrast Clinical Design System*, intentionally crafted to balance institutional medical trust with modern, interactive sophistication. This design approach prioritises clarity, confidence, and rapid information recognition, all critical requirements in high-stakes clinical environments.

Colour Theory and Semantic Encoding

The visual palette is dominated by blue-scale tones combined with cyan and teal gradients. Deep blue hues are widely associated with trust, security, and professionalism in healthcare interfaces, while cyan gradients introduce a contemporary, technology-forward aesthetic suited to an AI-augmented system.

Colour is used not merely as decoration but as a primary semantic layer. Medical event categories such as laboratory results, consultations, imaging studies, AI-generated suggestions, prescriptions, and allergies are consistently colour-coded across every view of the system. This allows clinicians to identify event types instantly through pre-attentive visual processing, without requiring explicit text reading, a principle well-supported in visualisation research [?, ?].

The same colour assignments are maintained across the categorical tree, the chronological timeline, node detail panels, and the legend, ensuring that the visual vocabulary is learned once and remains coherent throughout the application.

Visual Hierarchy and Layout Strategy

The interface relies on a card-based information architecture layered over a neutral background. Glassmorphism and soft elevation effects are applied selectively to authentication cards and dashboard panels, creating visual depth without introducing clutter.

A master-detail layout is employed throughout the patient dashboard, with a persistent high-contrast sidebar acting as a stable navigational anchor. This structure reduces spatial disorientation and supports rapid context switching, which is essential in time-constrained clinical workflows [?].

Typography, Iconography, and Micro-Interactions

The Inter typeface is used consistently throughout the application to maximise legibility across varying screen sizes and lighting conditions. Minimalist line-based Bootstrap Icons provide language-independent visual cues, accelerating recognition and navigation.

Within the Patient Journey Dashboard, medical events are represented as stylised circular nodes. High-priority nodes carry red rings rendered through CSS, drawing clinical attention without requiring user interaction. Hover states on node circles include a smooth scale transform and an enhanced drop-shadow filter, reinforcing the interactive affordance of each element.

The application additionally provides a dark mode, toggled via a persistent button in the top-right corner of the graph view. The user's preference is persisted to `localStorage`, ensuring continuity across sessions. Dark mode applies a comprehensive set of CSS variable overrides across all panels, the SVG canvas, modal dialogs, and sidebar components.

3.5.5 Authentication and Navigation Shell

Login Interface

The Sign In page serves as the primary entry point to the system and was designed to establish trust and professionalism, which are critical factors in healthcare software adoption. The interface follows a split-view layout: the left pane reinforces the system's value proposition through concise messaging emphasising security and patient-centred care, while the right pane presents a modern authentication card. Authentication is handled via Keycloak and OAuth 2.0, providing industry-standard session management.

The registration flow enforces input completeness before submission, preventing partial account creation. As shown in Figure 3.8, all required fields must be populated for the form to proceed. Once a user registers, they are required to verify their email address using a six-digit code sent to their registered address, as illustrated in Figure 3.9. Upon first activation, the system also generates a set of recovery codes that the clinician must copy or download as a text file, providing a fallback authentication mechanism should primary credentials be lost (Figure 3.10).

Figure 3.8: Registration form enforcing field completeness. The form cannot be submitted until all required fields are populated, preventing incomplete account creation.

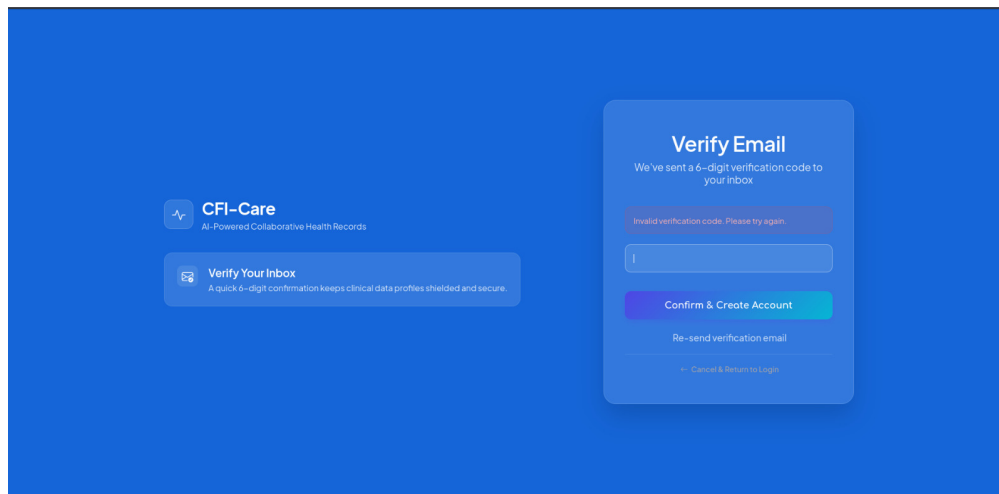


Figure 3.9: Email verification step requiring entry of a six-digit code sent to the clinician’s registered address, confirming ownership before account activation.

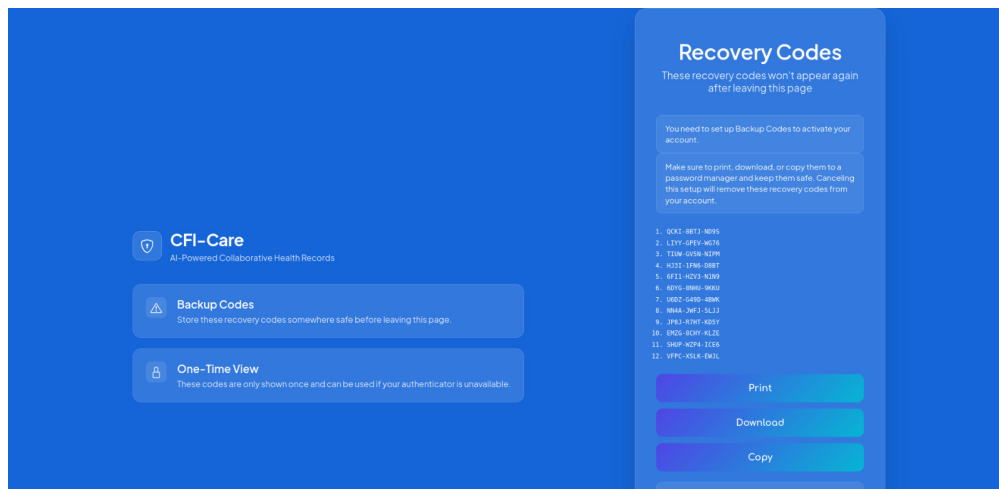


Figure 3.10: Recovery code generation screen presented at account activation. The clinician must copy or download the codes as a text file before proceeding, establishing a secure fallback authentication method.

For subsequent logins, the system provides two multi-factor authentication options: a time-based authenticator application (TOTP) or a recovery authentication code, as shown in Figure 3.11. Should a clinician need to recover access, a dedicated password reset flow is also available (Figure 3.12).

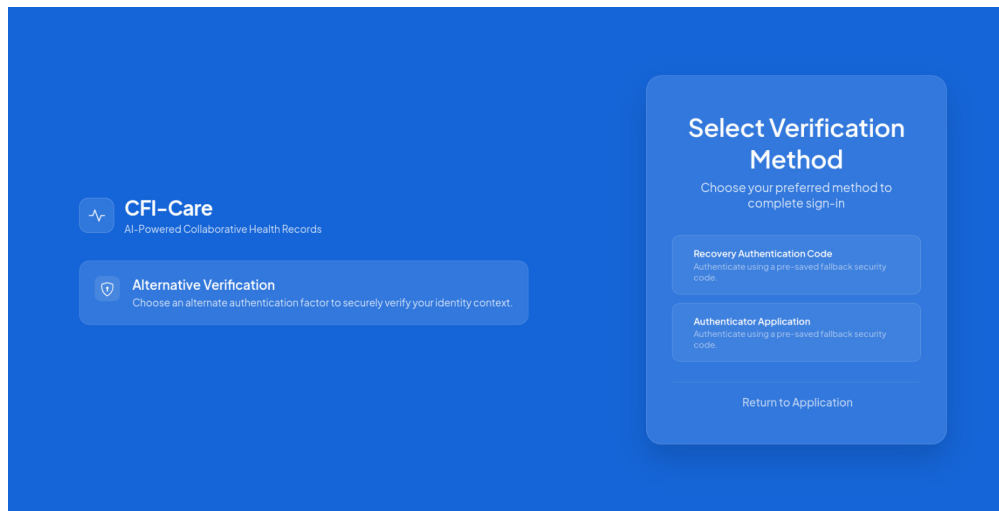


Figure 3.11: Multi-factor authentication method selection screen, offering the clinician a choice between a TOTP authenticator application and a recovery authentication code.

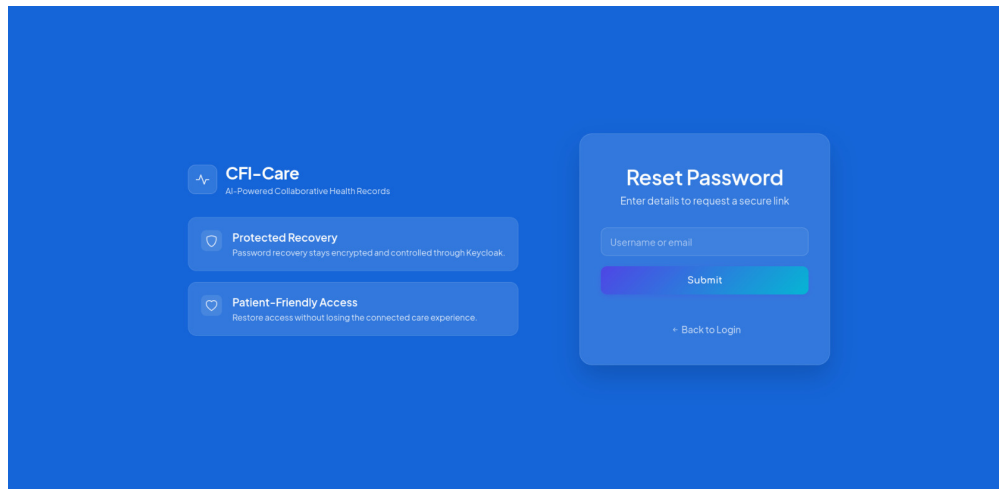


Figure 3.12: Password reset interface allowing clinicians to securely regain access to their account through a guided credential recovery flow.

Top Bar and Global Navigation

A persistent `TopBar` component is rendered across all authenticated views. It provides contextual navigation including a return link to the patient dashboard, access to the doctor's profile, account management options, and logout functionality. Within the `MedGraph` view specifically, the top bar includes a back-to-dashboard button rendered prominently to the left, ensuring the clinician is never more than one click from the patient list regardless of the depth of graph exploration.

The consistent presence of the top bar across all routes ensures spatial orientation is maintained throughout the application, following established guidelines for navigation in complex information systems.

3.5.6 Patients Dashboard

Overview

The Patients Dashboard, shown in Figure 3.13, functions as the primary data management hub of the system. It enables clinicians to search, select, and review patient profiles, manage episodes of care, and navigate to the patient journey visualisation. The dashboard is composed of three Angular components that operate in concert: `UsersPanel`, `Users`, and `UserInfo`.

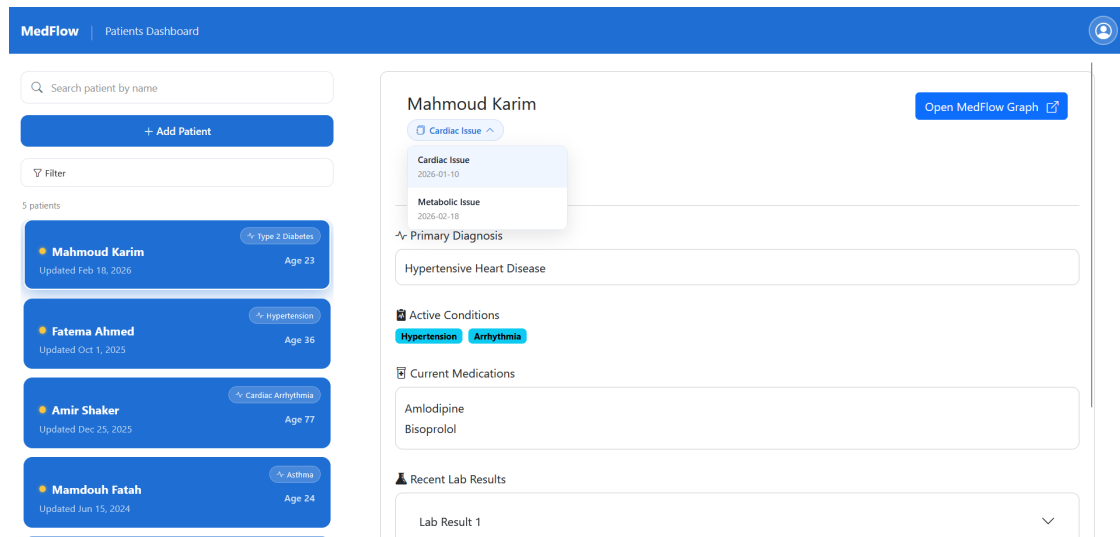


Figure 3.13: Patients Dashboard showing the master-detail layout. The left sidebar presents the patient navigator with search, filter, and episode management controls. The main panel displays the full clinical summary for the selected patient, including active conditions, medications, recent laboratory results, procedures, and a scoped encounter history.

Master-Detail Layout

The dashboard follows a master-detail layout, a design pattern widely adopted in medical information systems [?]. The layout consists of:

- **Patient Navigator (Left Sidebar):** Rendered by `UsersPanel`, this panel contains a search bar, a filter panel, an “Add Patient” action, and a scrollable list of patient cards rendered by the `Users` component.
- **Clinical Summary View (Main Panel):** Rendered by `UserInfo`, this panel displays structured patient information in a card-based layout, updating reactively when a new patient is selected.

Patient Navigator: Search, Filter, and Selection

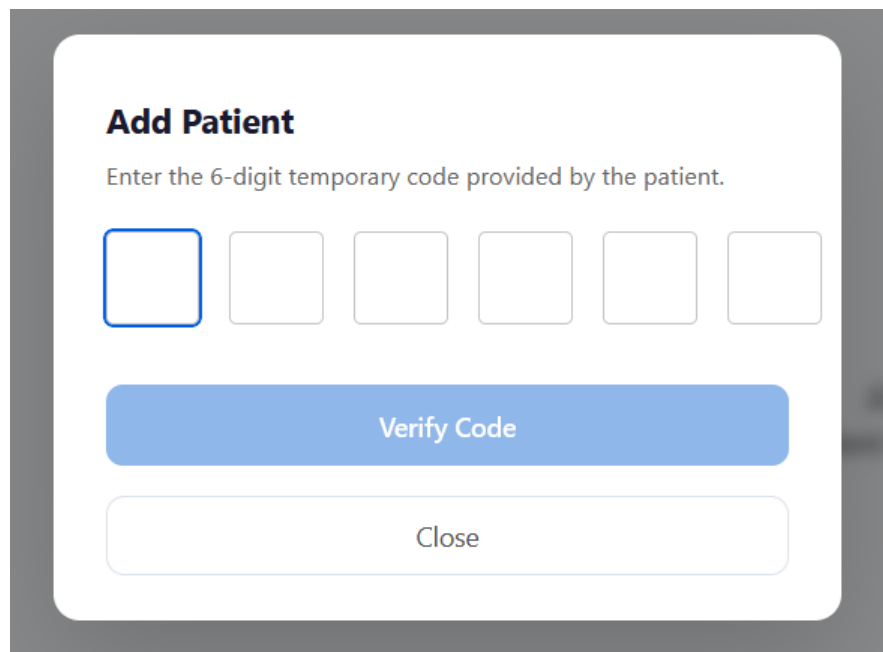
The `UsersPanel` component implements a reactive search mechanism that filters the patient list in real time as the clinician types. A filter panel toggles open to expose additional controls for sorting (by name, age, ID, or last-updated timestamp) and filtering by age range. The `applyFilter` method recombines all active criteria into a single filtered list, and the filter panel closes automatically upon application. A reset action restores all defaults without requiring manual field clearing.

Each patient card in the `Users` component displays the patient’s name, age, and last-updated timestamp. A colour-coded status dot provides an immediate freshness indicator: green indicates the record was updated within the past 30 days, and yellow indicates a stale record. This status is computed by the `getStatus` method using a precise day-difference calculation, providing a passive clinical alert without requiring explicit user interrogation of individual records.

Patient selection is mediated entirely through the `SelectedPatientService`, an injectable Angular service that exposes a reactive `selectedPatientId$` observable. Both the `Users` component (which sets the selected ID on click) and the `UserInfo` component (which subscribes to trigger a detail load) communicate through this shared service without any direct parent-to-child binding, exemplifying a clean observable-based state management pattern [?].

Patient Access Grant Workflow (OTP Handshake)

A key usability and security feature of the patient navigator is the patient access grant workflow, accessible via the “Add Patient” button at the top of the navigator. This workflow allows a clinician to request access to a patient’s data through a time-limited, patient-initiated one-time password (OTP). The six-digit OTP input modal is shown in Figure 3.14.



The image shows a modal dialog box with a dark gray border. Inside, the title "Add Patient" is in bold. Below it, a subtitle reads "Enter the 6-digit temporary code provided by the patient." There is a row of six square input boxes for the OTP code; the first box has a blue border. Below the input field is a prominent blue button labeled "Verify Code". At the bottom is a white button with a gray border labeled "Close".

Figure 3.14: Patient access grant modal presenting the six-cell OTP input field. The clinician enters the code generated on the patient’s device to initiate the access request handshake.

The process operates as follows:

1. The patient generates a six-digit OTP on their own device.
2. The clinician clicks “Add Patient” and a modal dialog opens presenting a six-cell OTP input field (powered by `ng-otp-input`) with confirm and cancel actions.
3. Upon submission, the frontend calls `handshake.verifyOtp()` and transitions to a “forwarded” state, displaying a confirmation that the request has been sent for the patient’s approval.
4. The component begins polling the backend via `handshake.pollStatus()` on a three-second interval, implemented using the RxJS `interval` and `switchMap` operators with `takeWhile` to automatically terminate polling when a terminal state is reached.
5. On patient approval, the modal transitions to a “success” state, the polling subscription is cleaned up, and the patient list refreshes to include the newly granted patient.
6. If the OTP is invalid, expired, or not found, distinct error states are rendered inline within the modal with specific, actionable messages.

This design keeps the entire workflow within a single modal without page reloads, minimising disruption to the clinician’s working context.

Clinical Summary View

The `UserInfo` component loads full patient details via a `forkJoin` of seven concurrent FHIR API calls, covering patient demographics, conditions, encounters, medications, procedures, document references, and episodes of care, ensuring that all data is fetched in parallel and the UI transitions to a loaded state only once all requests complete. The component maps raw FHIR resources to structured domain transfer objects using a set of dedicated transformation functions (`fhirPatientToDetailsDTO`, `fhirEOCToEpisode`, `extractConditionDisplay`, etc.), isolating FHIR-specific parsing logic from presentation concerns.

The clinical summary view organises data into clearly defined sections:

- Demographics: name, age, gender, and patient identifier.
- Primary diagnosis, active conditions with coloured pill badges, and current medications.
- Recent laboratory results with direct PDF viewing via inline `DocumentReference` binary resolution.
- Recent procedures.
- A full encounter history scoped to the selected episode, with each encounter typed and styled according to a comprehensive encounter type map spanning twelve clinical categories.

Episode of Care Management

The clinical summary panel includes an episode-of-care selector allowing the clinician to switch between multiple episodes for the same patient. The episode selector presents a dropdown of existing episodes, and each episode selection triggers a fresh encounter-scoped filter of conditions, medications, and procedures, ensuring that all displayed clinical data corresponds precisely to the chosen episode context.

New episodes can be created inline via a name input and a “+ Add Episode” button, which calls the FHIR `EpisodeOfCare` creation endpoint and immediately appends the result to the episode list. Fallback logic handles cases where FHIR filtering returns no data for a given episode, deriving medications from `Medication-type` encounters and diagnosing from `AI-Diagnosis-type` encounters as a secondary inference mechanism.

Patient-Scoped MedGraph Access

Rather than exposing the journey visualisation globally, access to the MedGraph is embedded within the patient details card. A prominent “Open History Graph” button navigates to `/med-graph/:id`, passing the currently selected episode’s ID as an `eocId` query parameter. This design ensures that any transition to the MedGraph view is explicitly scoped to the selected patient and episode, preventing context errors and reinforcing the principle that advanced visualisation actions should be clearly associated with their target data objects.

3.5.7 Patient Journey Dashboard

While traditional medical dashboards present patient data as isolated records, clinical decision-making often requires understanding how events evolve, relate, and unfold over time. To address this limitation, CFI-Care introduces the *Patient Journey Dashboard*, a dedicated interactive environment that provides a holistic, visual view of a patient's complete medical history.

The Patient Journey Dashboard enables clinicians to:

- Explore relationships between medical events through a hierarchical tree structure.
- Understand disease progression from initial presentation through to treatment and follow-up.
- Analyse alternative diagnostic or treatment pathways via independent branches.
- Identify clinically critical events at a glance without losing contextual awareness.
- Add, edit, and delete medical events in-place with full bidirectional backend synchronisation.

Rather than enforcing a single representation, the dashboard provides two complementary visualisation modes: *Categorical* and *Chronological*. Each mode offers a distinct analytical perspective over the same underlying patient data, allowing clinicians to switch between structural and temporal reasoning as needed.

Iframe Isolation Architecture

The graph engine is implemented as a self-contained HTML document loaded inside an iframe within the Angular **MedGraph** component. This isolation is a deliberate and well-reasoned architectural choice driven by three compounding concerns. First, D3.js and Treviz both perform direct, synchronous DOM mutations at high frequency during zoom, pan, and re-render cycles. Angular's Zone.js patches every asynchronous operation and triggers change detection on each one, so running D3 inside the Angular component tree would cause hundreds of unnecessary change-detection cycles per second during graph interaction, degrading performance meaningfully. Second, the iframe provides a complete DOM isolation boundary: the graph's global event listeners, CSS rules, and third-party library globals (D3 registers several of these) cannot leak into or conflict with the Angular host application. This goes beyond what `NgZone.runOutsideAngular()` would achieve, which suppresses Angular's change detection but still shares the same DOM, the same global CSS cascade, and the same JavaScript namespace. Third, full DOM isolation enables the visualisation layer to be developed, tested, and replaced independently, without destabilising the broader Angular application during iteration.

Communication between the Angular host and the iframe is implemented entirely via the `window.postMessage` API. The Angular host sends an `INIT_GRAPH` message on iframe load, carrying the full node list, patient ID, and episode of care ID. The iframe dispatches `NODE_UPDATE` messages for every add, edit, and delete operation,

which the Angular host intercepts and forwards to the corresponding backend API calls. Backend responses are relayed back to the iframe as `NODE_SAVED` or `NODE_ERROR` messages, enabling optimistic UI updates with rollback on failure. Figure 3.25 illustrates this message-passing sequence in full, and Figure 3.27 illustrates the complete data flow from user interaction through to backend persistence and rollback.

Node Data Model

Each node in the patient journey graph carries the following fields:

- **id**: Unique identifier (temporary client-side ID prefixed `temp-` until the backend assigns a persistent UUID).
- **text_1**: Display title of the medical event.
- **father**: Parent node ID (`null` for the root node).
- **category**: One of nine clinical categories, each mapped to a distinct colour.
- **priority**: Low, Medium, or High.
- **normality**: Normal, Abnormal, or Pending.
- **dateIssued**: ISO date string.
- **details**: Free-text clinical notes.
- **isDiagnosis** / **isManualBranch**: Flags marking branch-starter nodes in the chronological view.
- **branchState**: In-progress, discontinued, or completed, governing branch rendering and interaction permissions.

Global Controls and Interaction Modes

The graph header provides a consistent set of controls applicable to both views:

- **Add Node**: Activates a crosshair cursor and highlights all selectable nodes with a green ring. Clicking any node opens a modal form to specify the child event's title, date, category, priority, normality, and details. The new node is added optimistically to the data array and rendered immediately; the backend is notified asynchronously and the temporary ID is replaced with the real UUID upon confirmation.
- **Delete Node**: Activates a “not-allowed” cursor and highlights deletable nodes with a red ring. Selecting a node triggers a confirmation dialog and recursively removes the target node and all its descendants. If the backend delete fails, all removed nodes are restored from a `pendingDeletes` map.

- **Filters:** Toggles a filter panel allowing refinement by category, priority, normality, and date range. Filtering is applied reactively to both views with parent-node inclusion logic ensuring that matching nodes are never orphaned from the tree.
- **View Toggle:** Switches between Categorical and Chronological modes without page reload, preserving all filter and expansion state.

Guard logic prevents modifications within discontinued branches, completed branches, or a finished case, surfacing contextual toast messages that explain the restriction and describe how to lift it.

3.5.8 Patient Journey Dashboard — Categorical View

Purpose and Conceptual Model

The categorical view presents the patient journey as a hierarchical, relationship-oriented structure. Its primary purpose is to visualise **clinical relationships, causality, and dependency** between medical events. In this mode clinicians can clearly observe how diagnostic steps lead to subsequent decisions, which medical events are logically connected, and complete clinical paths from disease onset to treatment.

Hierarchical Graph Representation

Medical events are rendered as nodes within a directed node-link graph, laid out left-to-right using the Treviz library. A single root node represents the patient case origin, from which first-level category branches emerge. When more than one root node exists in the data, for example when a patient has multiple concurrent care pathways, the system automatically inserts a hidden virtual root node to produce a single connected tree, ensuring Treviz always receives a valid hierarchy without requiring manual data normalisation.

Expand and Collapse Semantics

First-level nodes directly connected to the root carry a small expand/collapse trigger button rendered in the top-right corner of their dot. Clicking this button expands or collapses the entire sub-tree rooted at that node, including all deeper descendants via the recursive `addAllDescendants` and `removeAllDescendants` functions. To contain visual complexity, a maximum of two first-level branches may be expanded simultaneously; activating a third branch automatically collapses the oldest expanded branch. This constraint preserves a coherent visual field without preventing access to any part of the tree.

High-Priority Propagation Mechanism

Clinically critical events are visually marked with an animated pulsing red ring rendered via a CSS `@keyframes pulse` animation. When a high-priority node exists within a collapsed branch, the priority indicator propagates upward to the corresponding first-level parent node. This ensures that urgent information is never visually hidden from a collapsed state. Upon expansion, the indicator is removed from the parent and appears solely on the originating node, preserving both visibility and clinical accuracy [?].

Node Relationship Indicators

Each non-root node carries a coloured sibling badge indicating whether it is the sole child of its parent (“SINGLE”, rendered in green) or one of multiple siblings (“SIBLING”, rendered in blue). These indicators allow clinicians to infer structural branching without expanding the full hierarchy, supporting rapid structural scanning.

Node Detail Sidebar

Clicking any node opens a slide-in detail sidebar on the right of the canvas, displaying the node’s title, normality status tag, priority tag, date, category with colour swatch, and full clinical detail text. The sidebar includes an “Edit Node” button that opens a draggable, resizable floating form prepopulated with the node’s current values. All edit operations are subject to the same branch-state guard logic as add and delete operations. The form itself is draggable via a `mousedown`-based drag handler and resizable via CSS `resize: both`, giving clinicians flexible screen real estate management during data entry.

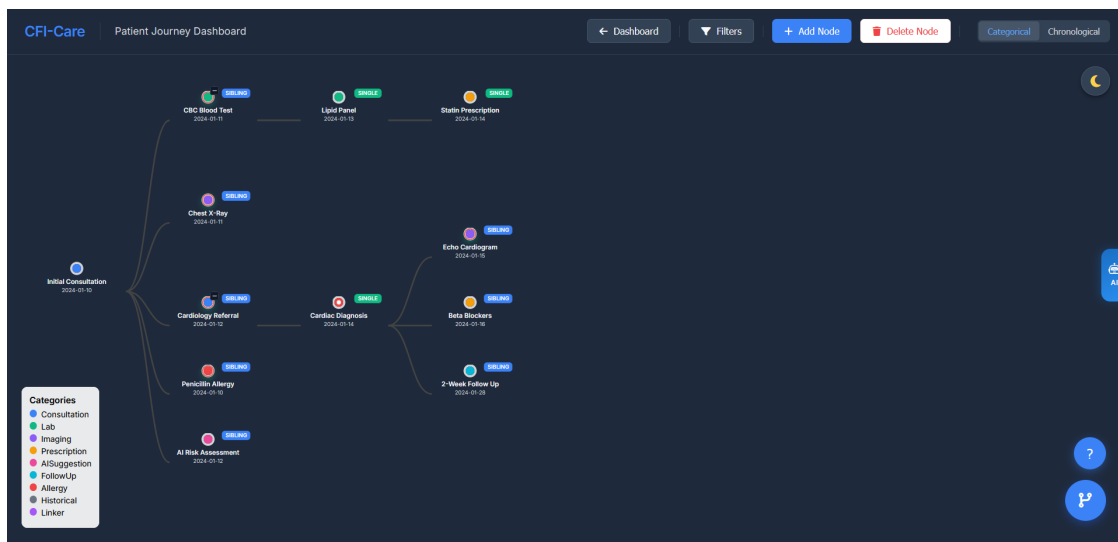


Figure 3.15: Categorical Patient Journey view representing medical events as a hierarchical left-to-right graph. High-priority events display animated priority rings, sibling relationships are labelled on each node, and the detail sidebar is visible on the right.

3.5.9 Patient Journey Dashboard — Chronological View

Purpose and Conceptual Model

The chronological view reorganises the same patient data into a time-oriented representation rendered using D3.js. Its purpose is to support **temporal reasoning** by allowing clinicians to read the patient journey as a sequential narrative. Unlike the categorical view, which emphasises relationships, the chronological view emphasises event order, time intervals between clinical decisions, and the progression of care across time [?].

Temporal Layout and Event Stacking

Events are distributed along a horizontal timeline scaled by a configurable `DATE_SCALE` constant, ensuring even temporal spacing regardless of event density. Each unique date is marked with a vertical tick and label rendered by the `drawDateMarkers` function. When multiple events occur on the same date within the same lane, they are stacked vertically using an alternating offset algorithm that places successive same-date nodes above and below the lane centreline, preventing overlap while preserving temporal accuracy.

The entire canvas supports pan and zoom via D3’s `zoom` behaviour, with a configurable scale range of $0.05\times$ to $3\times$. An automatic fit-to-view transform is computed and applied immediately after each render, ensuring the full timeline is visible on first load regardless of data density.

Branching Mechanism for Clinical Reasoning

A key feature of the chronological view is its branching mechanism. The main timeline represents the primary clinical pathway. When diagnostic uncertainty arises or alternative treatment strategies must be explored, clinicians can create independent sub-branches by activating the **branch creation button** (a blue code-branch icon in the bottom-right corner) and clicking any main-timeline node. This action opens the node creation form with the category preset to “Diagnosis”, and the resulting node is flagged with `isDiagnosis: true` and `isManualBranch: true`.

Sub-branches are rendered as distinct horizontal tracks below the main timeline, connected to their origin on the main line via smooth cubic Bézier divergence paths drawn by the `generateLinkPath` function. Each branch maintains its own horizontal baseline drawn at a fixed vertical offset determined by `BRANCH_VERTICAL_GAP`, ensuring consistent visual separation between parallel clinical reasoning paths.

Branching from an existing branch is explicitly prohibited; a contextual toast message is displayed if this is attempted, guiding the user to branch only from main-timeline nodes. This constraint reflects the clinical model that sub-branches represent direct hypotheses against the main care pathway.

Branch State Management

Every branch carries a **branch state** that governs its visual presentation and edit permissions:

- **In Progress** (blue): The branch is active. Nodes may be added, edited, and deleted freely.
- **Discontinued** (red): The branch has been ruled out. A prominent “DISCONTINUED” wall marker and pulsing wall animation are rendered at the branch terminus. All add, edit, and delete operations within the branch are blocked with an explanatory message. Figure 3.18 shows a zoomed view of a discontinued branch.
- **Completed** (green): The branch has been validated and merged back into the main timeline. A green curved merge arc is drawn from the branch’s last node to the nearest main-timeline node, and a “MERGED” label is rendered above the arc. If the branch’s last event post-dates all main-timeline events, the system automatically creates a *Linker* node on the main timeline at that date to serve as the merge target, ensuring geometric accuracy of the arc. Figure 3.17 shows a zoomed view of a completed and merged branch.

Branch state is changed by clicking the branch line itself to open a state-selection menu. The main timeline also carries a state that can be set to **Finish Case**, which renders a “CASE CLOSED” flag at the timeline terminus, displays a “Case Finished” banner in the header, and blocks all modifications to the entire graph until the case is reopened.

High-Priority Visualisation

High-priority events in the chronological view are highlighted using a red “HIGH PRIORITY” label rendered beneath the node’s date stamp. This visual hierarchy ensures that urgent clinical issues remain immediately noticeable within the temporal flow regardless of zoom level.

Stateful Mode Switching

A toggle control in the header allows instant switching between categorical and chronological views. This interaction is fully stateful: all filter selections, expansion state, and data mutations carry over seamlessly between modes. No page reload is required, and the `refreshGraph` function pointer is reassigned to the appropriate renderer on each mode switch, ensuring that subsequent filter changes or node operations automatically re-render in the correct mode.



Figure 3.16: Chronological Patient Journey view displaying medical events along a horizontal D3.js timeline with pan and zoom. Parallel sub-branches represent alternative diagnostic hypotheses, each on its own horizontal track. The completed branch is merged back into the main timeline via a green arc, and the discontinued branch is blocked with a red wall marker. A “CASE CLOSED” flag is rendered at the timeline terminus when the main timeline is set to the Finished state.

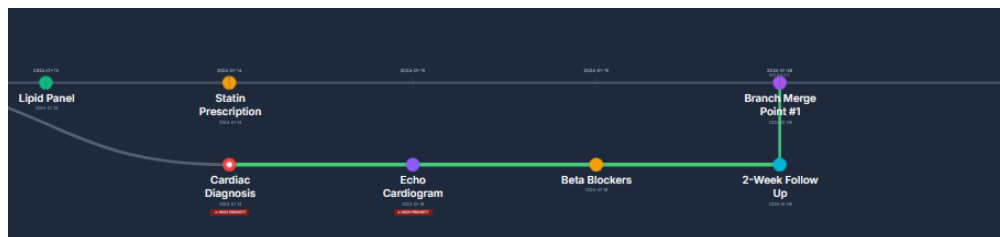


Figure 3.17: Zoomed view of a completed branch in the chronological timeline. The green curved merge arc connects the branch’s final node back to the main timeline, with a “MERGED” label rendered above the arc. The branch line is rendered in green to indicate the validated state.



Figure 3.18: Zoomed view of a discontinued branch in the chronological timeline. A pulsing red wall marker is rendered at the branch terminus with a “DISCONTINUED” label. The branch line is rendered in red, and all add, edit, and delete operations on nodes within this branch are blocked.

3.5.10 Complementary Roles of the Two Views

The categorical and chronological views address fundamentally different cognitive tasks. The categorical view focuses on **structural understanding**, enabling clinicians to reason about relationships, dependencies, and complete clinical paths. The chronological view focuses on **temporal understanding**, enabling clinicians to interpret the sequence and timing of events and to visualise parallel reasoning paths.

Providing both views allows clinicians to:

- Understand how events are connected (categorical).
- Understand when events occurred and how the case evolved (chronological).
- Reduce cognitive load by selecting the most appropriate representation for the current analytical task.

This dual-view approach follows established guidelines for multiple coordinated visualisations, which have been shown to improve usability and analytical effectiveness in complex information systems [?].

3.5.11 Interactive Tutorial System

A built-in interactive tutorial system guides new users through every major feature of the Patient Journey Dashboard on first login. The tutorial operates in two modes:

1. **Welcome Prompt:** On first visit (controlled via a `localStorage` flag), a modal dialog appears offering the clinician the choice to begin the tour immediately or skip it.
2. **Step-by-Step Tour:** The tutorial progresses through eighteen distinct steps covering categorical nodes, priority propagation, expand/collapse interaction, the node detail sidebar, add and delete modes, filters, the dashboard navigation button, chronological layout, diagnosis nodes, sub-branch tracks, the branch creation button, the completed-and-merged arc, the merge hint button, and both the main and sub-branch state menus.

Each step highlights the relevant UI element using a CSS outline pulse animation. Steps that require a direct user interaction (such as clicking the expand trigger, clicking a node, switching to chronological mode, or opening a branch state menu) display a “**Interact to continue**” badge in place of the Next button, and the tutorial advances automatically once the interaction is detected via targeted DOM event listeners.

To ensure the tutorial demonstrates the full feature set without depending on the live patient data, the system silently saves the clinician’s real data array, substitutes a rich pre-defined demo dataset covering all categories, branch types, and states, and restores the real data array when the tour ends or is dismissed. A help button (?) in the fixed bottom-right corner allows the tutorial to be replayed at any time.

Figure 3.19 illustrates tutorial Step 4, which introduces branch expansion. First-level nodes are shown in their collapsed default state, with each node carrying its SIBLING badge and a highlighted + button in the top-right corner of its dot. The tutorial card instructs the clinician to click the highlighted button to expand a branch, and the step advances automatically once the interaction is detected. Figure 3.20 shows Step 10, which guides the clinician to switch from the categorical view to the chronological timeline using the toggle control highlighted in the header. The step advances automatically when the toggle is activated, exposing the D3.js timeline without requiring a manual “Next” click. Figure 3.21 demonstrates Step 16, in which the clinician is instructed to click the main horizontal timeline line to open its branch state menu. The menu presents two options: In Progress (the current default) and Finish Case. The step is classified as an interaction step, so the tutorial waits for the clinician to open the menu before advancing, ensuring they observe the state-change mechanism in context.

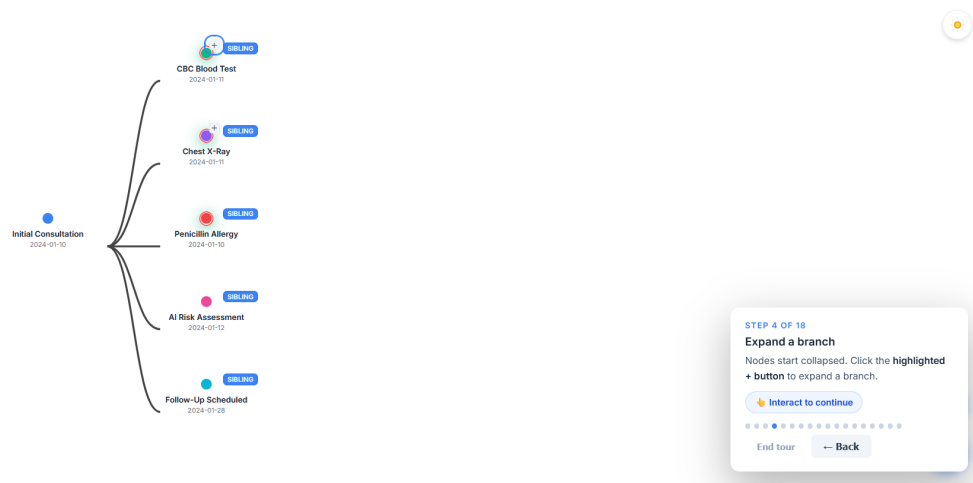


Figure 3.19: Tutorial Step 4 of 18: Expand a branch. The categorical view shows first-level nodes in their collapsed default state. The + button on the CBC Blood Test node is highlighted with a pulse animation, and the tutorial card instructs the clinician to click it to expand the sub-tree. The step advances automatically upon interaction.

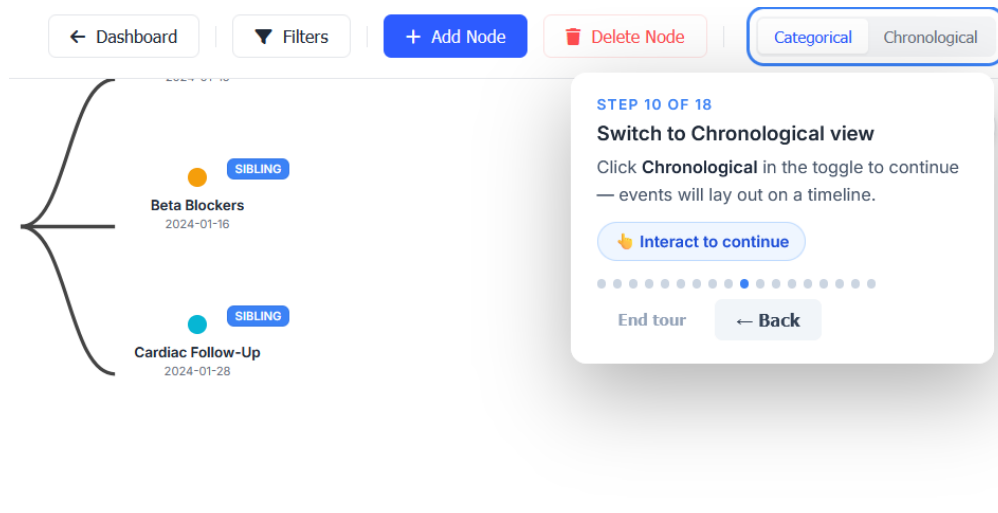


Figure 3.20: Tutorial Step 10 of 18: Switch to Chronological view. The view toggle in the header is highlighted, and the tutorial card instructs the clinician to click “Chronological” to transition the layout from the hierarchical tree to the D3.js timeline. The step advances automatically once the toggle is activated.

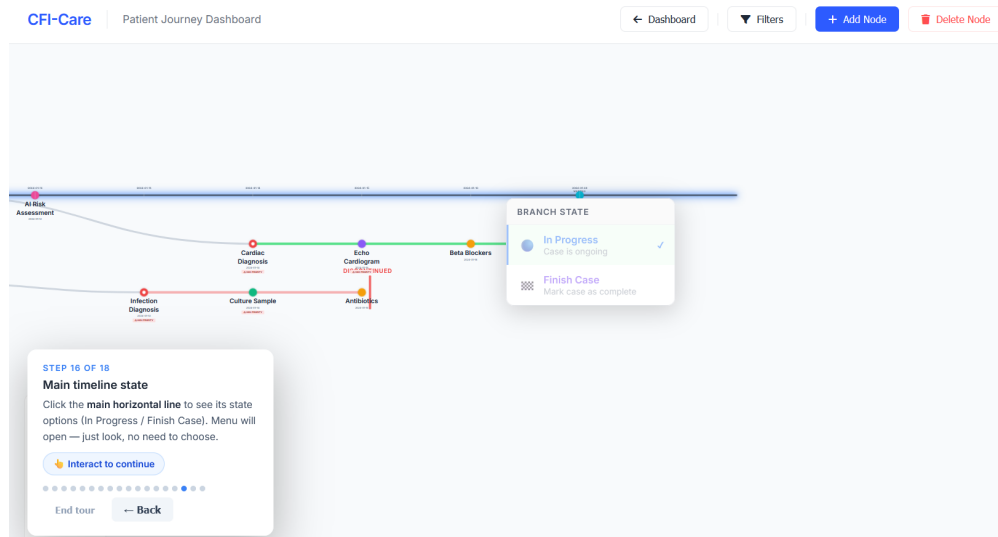


Figure 3.21: Tutorial Step 16 of 18: Main timeline state. The chronological view is shown with the branch state menu open on the main timeline line, presenting the In Progress and Finish Case options. The tutorial card instructs the clinician to click the main timeline to open this menu, advancing automatically once the interaction occurs.

3.5.12 AI-Assisted Clinical Chat

The `ChatSection` component provides an integrated AI chat interface accessible from within the MedGraph view, supporting real-time clinical reasoning alongside patient journey data. The component is designed for sustained, multi-turn clinical dialogue and incorporates a range of usability features tailored to medical workflows.

Conversation and Session Management

The chat interface maintains a list of named conversation sessions. A “New Chat” action saves the current session’s messages and initialises a blank session. Switching between sessions restores the full message history for the selected session. Sessions are auto-titled from the first user message (truncated to 27 characters for display), and titles can be manually edited inline via a double-click interaction on the session list item.

A search field filters the session list in real time, matching sessions by title prefix.

Message Composition and Input Controls

The message composer features a self-expanding textarea that grows vertically up to a maximum of 180 pixels as the user types, implemented via the `autoResize` event handler. File attachments (images and PDFs) can be added via a file picker; attached files are base64-encoded and displayed as inline previews in the outgoing message bubble, with individual removal controls. A voice input button activates the Web Speech API (`webkitSpeechRecognition`) for continuous speech-to-text input with real-time interim transcript display.

Response Mode Selection

Four response modes, Concise, Detailed, Bullet Points, and Differential, allow the clinician to specify the desired output format for AI responses, adapting the interface to different clinical contexts such as rapid triage versus comprehensive case review.

Message-Level Interactions

Each AI response bubble exposes a set of interaction controls:

- **Like / Dislike:** Binary feedback toggles allowing the clinician to rate response quality.
- **Copy:** Copies the full response text to the clipboard.
- **Retry:** Replaces the AI response in-place with a new generation, preserving conversation position.
- **Fork:** Opens a new browser tab pre-loaded with a snapshot of the conversation up to and including the selected response, encoded in URL parameters, allowing independent exploration of alternative follow-up threads from any prior point.
- **Delete:** Removes the AI response and its corresponding user message from the session.
- **Citations:** Opens an inline citations panel listing the reference sources associated with the AI response.

A confidence score is displayed with each AI response, giving the clinician a visible signal of response certainty.

Maximised View and Branch Tabs

The chat panel supports a maximised mode that expands the component to fill the available viewport, providing a focused environment for extended clinical dialogue. Sessions opened via the Fork action automatically launch in maximised mode and display a visual divider message indicating the branch origin session and message index, preserving chain-of-custody of the conversation thread.

Figure 3.22 shows the chat interface in its compact panel form, as it appears alongside the patient journey graph. Figure 3.23 shows the same interface in its maximised mode, expanding to fill the viewport for extended clinical dialogue.

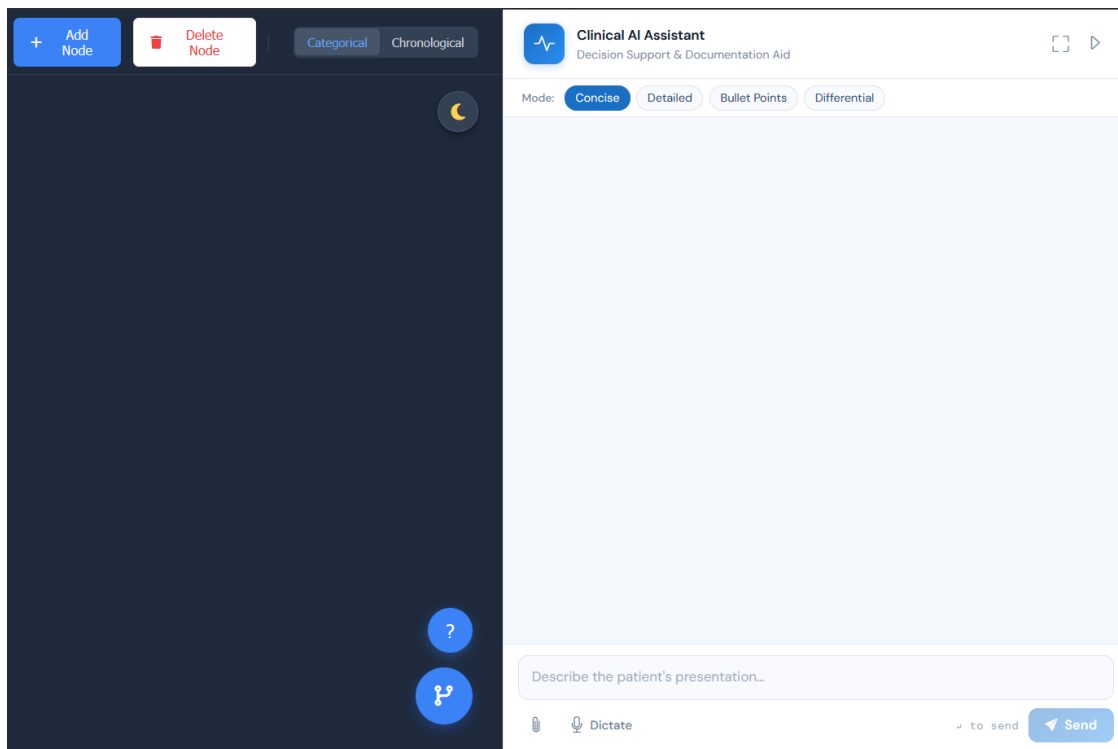


Figure 3.22: AI-assisted clinical chat in its compact panel view, rendered alongside the Patient Journey Dashboard. The session list, message history, response mode selector, and composer are all visible within the panel.

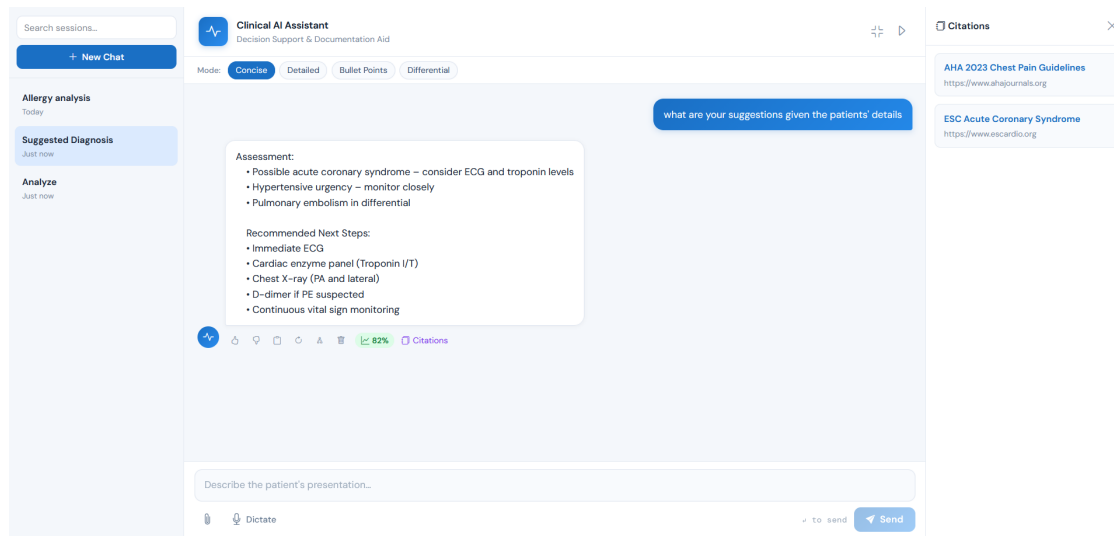


Figure 3.23: AI-assisted clinical chat in maximised mode, expanding to fill the full viewport. This mode is intended for extended clinical reasoning sessions and is also the default view for conversations opened via the Fork action.

3.5.13 Doctor Profile and Schedule Management

The `DoctorProfileComponent`, shown in Figure 3.24, provides each practitioner with a dedicated profile and availability management interface, structured as a two-panel layout.

Profile Panel

The left panel displays the practitioner’s initials avatar, name, title, specialty, contact details (phone and address), affiliated hospitals, and an “Active & Verified” status badge. An “Edit Profile” button at the bottom of the panel opens a modal dialog pre-populated with the current profile values, allowing inline editing of all fields. Hospital affiliations can be added and removed individually within the modal. On save, the component performs parallel FHIR updates to both the `Practitioner` resource and the `PractitionerRole` resource, with the role update preserving all existing FHIR metadata.

Schedule Manager

The right panel hosts the Schedule Manager, which presents schedules and slots in two separate tabular sections. A “Today’s Summary” sidebar on the far right provides an at-a-glance count of total slots broken down by status. A Glossary section beneath the summary defines the three slot states (Free, Busy, Unavailable) in plain language, reducing the need for external documentation.

Schedules represent planning horizons (date ranges during which the practitioner is available). New schedules are created via a collapsible form, and existing schedules can be edited in-place via an inline edit row or deleted with a confirmation guard. Each

schedule creation validates that the start date precedes the end date, surfacing inline error messages on failure.

Slots are bookable time units within a schedule. The slot creation form validates that the selected time range falls within the bounds of the chosen schedule, preventing orphaned slot creation. Slot statuses (Free, Busy, Unavailable) can be updated individually via an inline dropdown that persists the change to the backend immediately. A set of filter buttons (All / Free / Busy / Unavailable) controls which slots are displayed in the table, with counters for each status category visible in the summary panel.

The screenshot displays the 'Doctor Profile' page for Dr. Youssef, a Senior Cardiologist. The left sidebar contains contact details (phone: +20 1103686808, address: Cairo, Building C - Floor 3) and specialty (Interventional Cardiology). The main area is the 'Schedule Manager', which includes a 'Schedules' table with two entries (SCH-001 and SCH-002) and a 'Slots' table with three entries (SLT-001, SLT-002, SLT-003). The 'Slots' table shows status filters (All, Free, Busy, Unavailable) and a 'Create Slot' button. A 'Today's Summary' sidebar on the right shows 3 total slots (2 Free, 1 Busy, 0 Unavailable) and 2 schedules.

ID	ACTIVE	START	END
SCH-001	Yes	01 Apr. 08:00	30 Apr. 18:00
SCH-002	Yes	15 Apr. 09:00	20 Apr. 17:00

ID	SCHEDULE	STATUS	START	END
SLT-001	SCH-001	Free	25 Apr. 09:00	25 Apr. 09:30
SLT-002	SCH-001	Busy	25 Apr. 09:30	25 Apr. 10:00
SLT-003	SCH-002	Free	25 Apr. 10:00	25 Apr. 10:30

Figure 3.24: Doctor Profile page showing the practitioner information panel on the left and the Schedule Manager on the right, with slot creation form, status filters, and a today's summary sidebar.

3.5.14 Domain Models and State Services

Shared domain concepts are defined in the `models` directory. The `patient.model.ts` file formalises the patient entity, covering identifiers, metadata, episodes, encounters, and associated medical events, and provides all FHIR-to-DTO transformation functions as pure, stateless utilities.

Application-wide state is managed via injectable Angular services:

- **PatientApiService:** Responsible for all patient-related API communication, including FHIR resource retrieval, graph node operations, and binary resource resolution for lab PDF viewing.
- **SelectedPatientService:** Maintains the currently active patient context as a BehaviourSubject, enabling any component in the tree to subscribe to patient selection events without tight coupling.
- **HandshakeService:** Encapsulates the OTP verification and grant status polling API calls.

- **AppointmentService**: Manages all schedule and slot CRUD operations.
- **AuthService**: Wraps the Keycloak authentication status check used by both the auth guard and the doctor profile loader.

By centralising patient selection and API communication in services, multiple components can synchronise reactively without direct dependencies, reducing tight coupling and improving testability [?].

3.5.15 Error Handling and Resilience

The frontend implements a layered error-handling strategy. At the route level, a dedicated **GatewayErrorComponent** handles arbitrary HTTP error codes received as route parameters, rendering a contextual error page without exposing raw stack traces.

At the component level, all API subscriptions include **error** handlers that set typed error state variables rendered as user-facing messages. The MedGraph’s optimistic update pattern provides full rollback for all three node operation types: added nodes with temporary IDs are spliced from the data array on failure; edited nodes are restored from a **pendingEdits** snapshot map; deleted node trees are fully restored from a **pendingDeletes** collection. This ensures the visual state of the graph remains consistent with the backend at all times, even under intermittent network conditions.

3.5.16 Sequence Diagram

Figure 3.25 illustrates the complete message-passing sequence for MedGraph node operations, encompassing the iframe initialisation handshake, optimistic add/edit/delete operations, backend confirmation and rollback paths, and mode switching between categorical and chronological views.

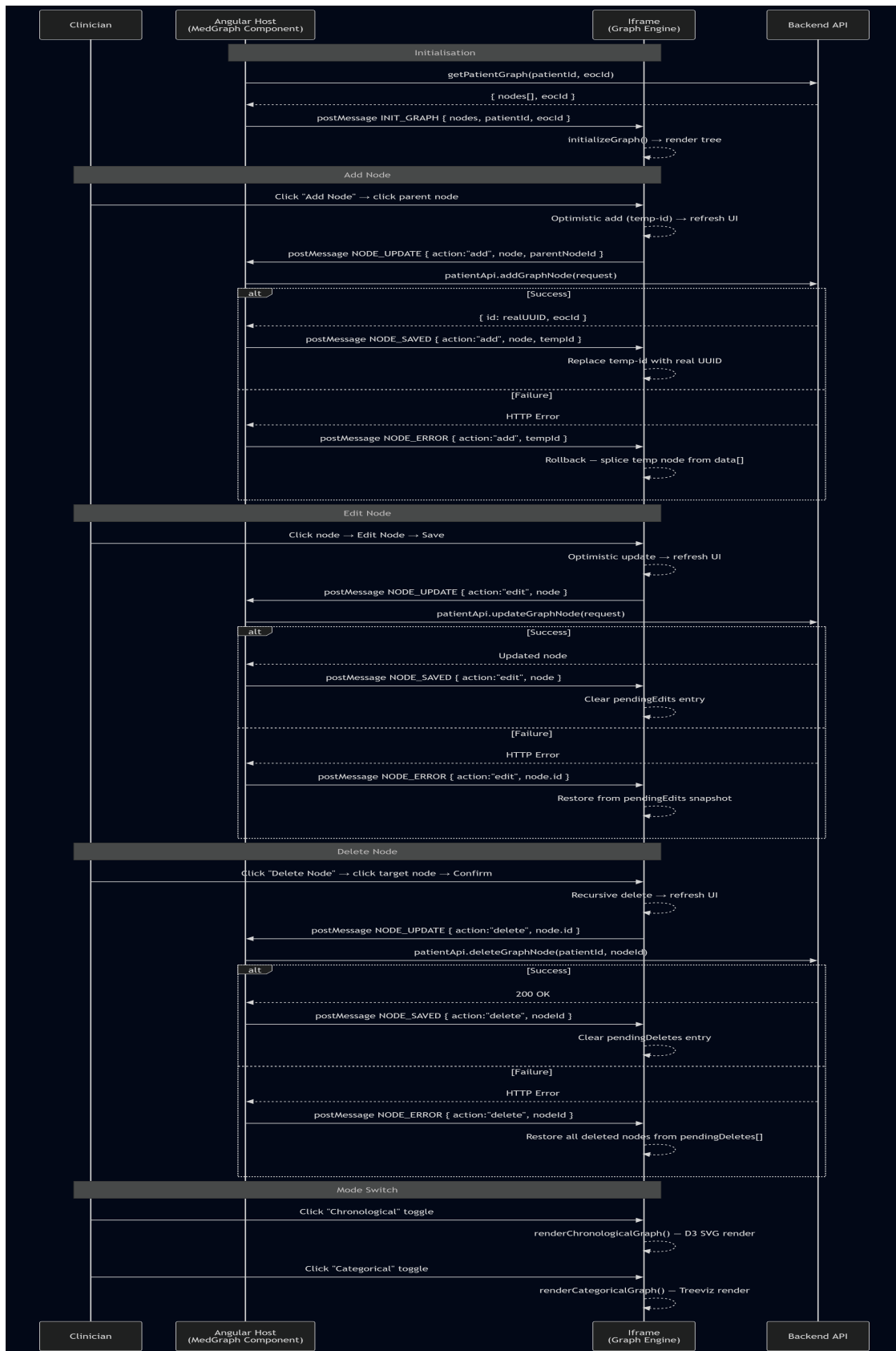


Figure 3.25: Sequence diagram illustrating the full MedGraph interaction lifecycle: iframe initialisation, postMessage-based node operations, backend confirmation, optimistic rollback on failure, and stateful mode switching.

3.5.17 Component Diagram

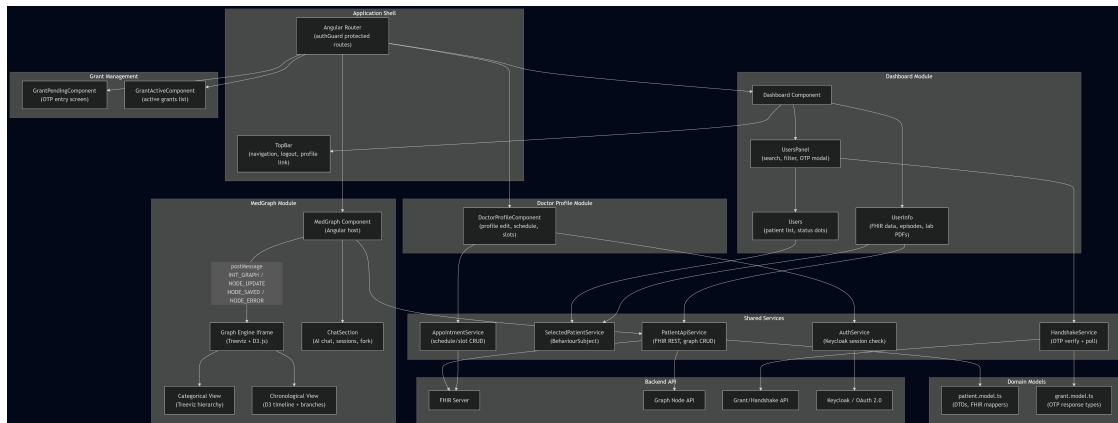


Figure 3.26: Component diagram illustrating the architecture of the dashboard, Med-Graph iframe boundary, AI chat module, application services, and backend integration points.

3.5.18 Data Flow Diagram

Figure 3.27 illustrates the complete data flow for a node CRUD operation, tracing the path from a user interaction inside the graph engine, through the `postMessage` boundary, into the Angular service layer, out to the FHIR backend, and back through the confirmation and rollback paths. This diagram complements the sequence diagram by focusing on data movement and state transitions rather than timing.

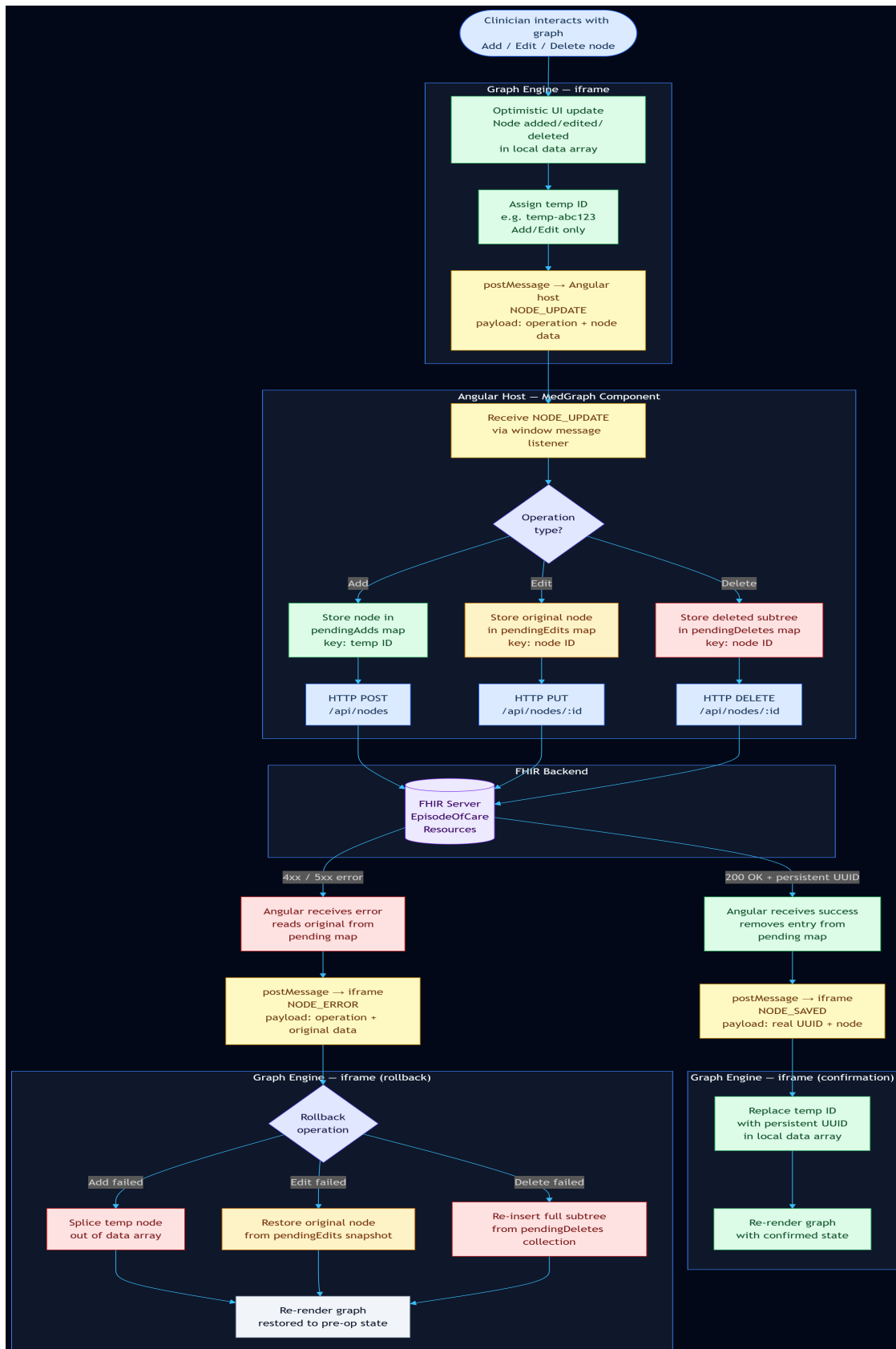


Figure 3.27: Data flow diagram for a MedGraph node CRUD operation. The flow begins with a user interaction in the graph engine (iframe), passes through the postMessage boundary into the Angular host, is forwarded to the FHIR backend, and returns via a confirmation or error path. On success, the temporary node ID is replaced with the persistent UUID. On failure, the pending operation is rolled back from the corresponding state map.

3.5.19 Architectural Rationale

The architectural decisions underlying the CFI-Care frontend were driven by four primary constraints:

1. **Visualisation Complexity:** Medical journey graphs require custom SVG layouts, zoom behaviours, and stateful interaction patterns that are not supported by standard Angular UI libraries.
2. **Performance:** D3.js performs intensive DOM mutations that would trigger excessive Angular change-detection cycles if executed within the component tree.
3. **Maintainability:** Clinical logic, routing logic, FHIR transformation logic, and graph rendering logic must evolve independently without cross-cutting refactoring.
4. **Data Integrity:** Optimistic updates require a principled rollback strategy at every operation type to prevent UI state from diverging from persisted backend state.

The iframe isolation boundary directly addresses constraints 1 and 2. The service layer and domain model separation address constraint 3. The `pendingEdits` and `pendingDeletes` maps address constraint 4. Together, these decisions produce a system that is both clinically capable and structurally sound for continued development [?].

3.5.20 Graph Logic Test Suite

To validate the correctness and robustness of the graph engine’s JavaScript logic, a dedicated browser-based test suite was developed for the CFI-Care patient journey graph. The suite targets the pure JavaScript functions that govern the graph’s data model and rendering pipeline, testing them directly in isolation from the graph’s visual layer. This approach ensures that the core clinical logic, including node hierarchy management, branch state transitions, chronological sorting, filter evaluation, and recursive tree operations, is verified independently and reproducibly.

Test Architecture

Rather than loading the full graph interface during testing, the suite reconstructs a controlled mock environment in JavaScript that mirrors the graph’s internal data structures and pure functions exactly. This allows every test to run deterministically in the browser without network dependencies, iframe loading times, or DOM rendering variability. The mock environment reproduces the twelve-node baseline dataset used throughout the system, covering all nine clinical categories, all three priority levels, all three normality values, and a representative branch with in-progress state.

Test Suites and Coverage

The suite is organised into five suites totalling 62 test cases, all of which pass with zero failures and zero warnings, completing in 22ms total. The five suites and their scope are as follows.

Functional Tests (21 cases) validate the core data model and node operations. Tests verify that the dataset maintains a single root node with a `null` father, that all non-root nodes reference existing parents, that no circular parent references exist, and that all required fields are present and valid. Node operations are tested end-to-end: adding a node confirms the correct parent-child relationship is established; deleting a node via `deleteRecursive` confirms the target is removed and all descendants are removed recursively, while siblings are preserved. Branch logic tests confirm that `isNodeInRejectedBranch` and `isNodeInCompletedBranch` correctly traverse the ancestor chain for both discontinued and completed branch states, and that linker node detection operates on both the `category` field and the `_linkedBranchId` flag.

Chronological View Tests (15 cases) validate the `getChronologicalData` function that drives the D3.js timeline. Tests confirm that the full dataset passes through with default filters, that output nodes are sorted in non-decreasing date order, that main-timeline nodes receive `laneId = 0` and branch nodes receive `laneId > 0`, and that all members of a branch share the correct `branchStartId`. Filter correctness is verified across all three dimensions (category, priority, normality) individually, as well as date-range filtering and `isDiagnosis` suppression. Mutation tests confirm that newly added nodes appear in output and that deleted nodes are absent. A multi-branch test confirms that two independent diagnosis branches receive distinct lane identifiers.

Categorical View Tests (7 cases) validate tree structure invariants required by the Treviz rendering layer. Tests confirm that exactly one root node exists, that node depth is computed correctly (node 10 sits at depth 3), that sibling detection correctly identifies all five children of the root, that the `MAX_EXPANDED = 2` concurrency limit evicts the oldest expanded branch when a third is activated, that linker nodes are excluded from the categorical view, that a virtual root is correctly inserted when multiple roots are present, and that the recursive collapse function removes all descendants from the visible set.

Boundary and Validation Tests (12 cases) probe edge conditions and guard logic. Tests cover single-node and empty datasets, 500-character node titles, epoch-dated and far-future nodes (sorting to the correct ends of the timeline), deletion of the root node (removing all 12 nodes), case-sensitivity rejection of a malformed category value, leaf-node collection, and both guard-logic scenarios in which attempting to add a child to a node inside a discontinued or completed branch is correctly blocked.

Performance Tests (7 cases) set explicit time budgets for all computationally intensive operations and warn if any budget is exceeded. `getChronologicalData` must complete in under 5ms averaged over 100 iterations on the 12-node dataset, and under 30ms on a synthetically generated 500-node dataset. `collectNodesRecursive` must average under 2ms over 200 iterations. `isNodeInRejectedBranch` must average under 1ms over 500 iterations on a 50-node deep chain. `deleteRecursive` on the full 12-node tree must average under 1ms over 100 iterations. The triple-filter pipeline (category,

priority, normality) on 500 nodes must average under 3ms over 50 iterations. A memory integrity test confirms that all read-only operations leave the original data array length unchanged across 50 iterations, detecting any inadvertent side effects.

Test Runner Interface

The test runner is implemented as a self-contained HTML application providing a live dashboard for executing and reviewing results, shown in Figure 3.28. The interface includes a scoreboard displaying total, passed, failed, warned, and skipped counts alongside total execution time; a progress bar that fills as tests complete; a collapsible suite-by-suite result view with per-test status icons, descriptions, assertion messages, and individual execution times; a performance bar chart plotting each measured function against its budget threshold; and a scrollable log console with colour-coded pass, fail, warn, and info entries. An export button generates a Markdown report containing the full results table and log.

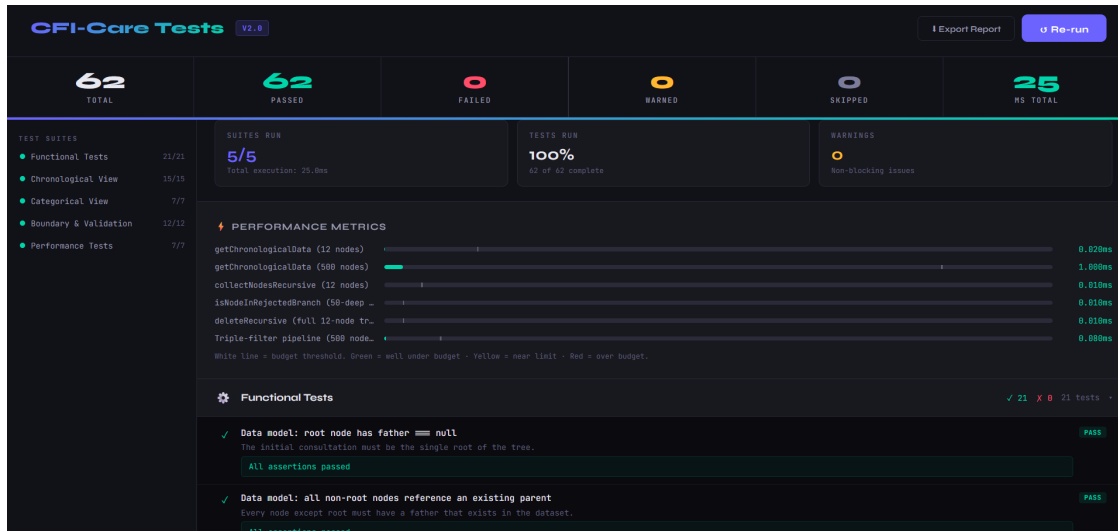


Figure 3.28: CFI-Care Graph Logic Test Suite dashboard after a full run. All 62 tests across five suites pass with zero failures and zero warnings in 22ms total. The scoreboard, per-suite result panels, performance bar chart, and log console are visible.

3.5.21 Summary

The CFI-Care frontend successfully transforms fragmented clinical data into a coherent, interactive medical information system. Through a carefully layered Angular architecture, a semantically rich visualisation engine combining Treviz and D3.js, an OTP-based patient access workflow, a full AI-assisted clinical chat interface, and a practitioner scheduling system, the frontend addresses the complete spectrum of clinical interaction requirements. The dual-view Patient Journey Dashboard, with its categorical tree and chronological timeline, provides clinicians with both structural and temporal analytical perspectives from a single, seamlessly navigable interface. The built-in tutorial system, optimistic update strategy, dark mode support, and branch state management

together reflect a commitment to usability, resilience, and clinical safety that underpins every design decision made during this phase of development. article listings xcolor [english]babel caption

[letterpaper,top=2cm,bottom=2cm,left=3cm,right=3cm,marginparwidth=1.75cm]geometry
amsmath graphicx float [colorlinks=true, allcolors=blue]hyperref

Your Paper

You

June 4, 2026

3.6 Mobile

3.7 Mobile Application Objectives & UI

The primary objective of developing the proposed mobile application is to provide a secure, efficient, and patient-centered platform for managing personal health information within the digital medical ecosystem integrated with the FHIR-compliant backend.

3.7.1 Authentication Flow & Account Security

Authentication in CFI-Care is delegated entirely to Keycloak, an open-source Identity and Access Management server that implements the OpenID Connect (OIDC) protocol on top of OAuth 2.0. This approach means the mobile app never handles or stores user credentials, it only deals with tokens issued by Keycloak after the user authenticates.

Login and Register

The Login and Register flow uses the Authorization Code Flow with **PKCE** (Proof Key for Code Exchange), implemented via the `flutter_appauth` package[?].

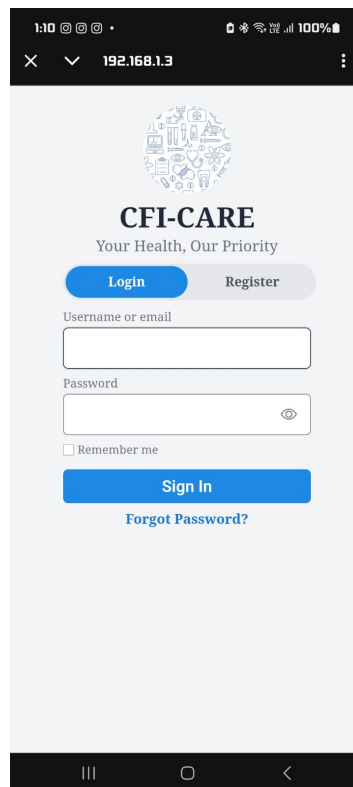


Figure 3.29: Login with KeyCloak

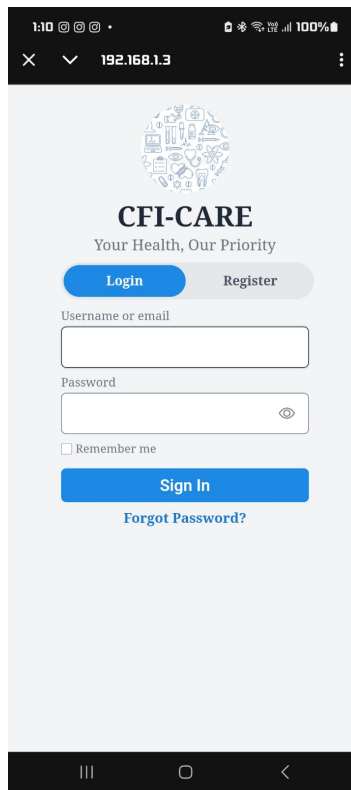


Figure 3.30: Register with KeyCloak

Email Verification Code

During the email verification process, a unique six-digit code is generated and saved within the Keycloak authentication session. This code is then delivered to the user's email address through Keycloak's email service using a customized HTML template. The user is presented with a verification page where the received code can be entered. Upon submission, the entered code is compared with the code stored in the session. If the codes match, the user's email address is successfully verified and the registration process proceeds. Otherwise, an error message is displayed, allowing the user to request a new verification code and try again.

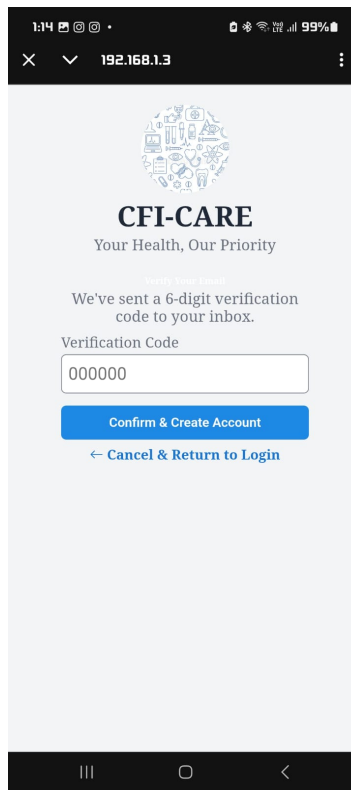


Figure 3.31: Verification Code

Recovery Codes

After completing registration, Keycloak presents the user with backup recovery codes. These are Keycloak's built-in recovery codes, displayed in a XXXX-XXXX-XXXX format in a list.

The page provides three options to save the codes:

- **Copy** : copies all codes to the clipboard
- **Save** : downloads them as a cfi-care-recovery-codes.txt file
- **Print** : opens a print dialog with a clean formatted view

The user must check a confirmation checkbox ("I have stored these backup recovery codes safely") before the submit button becomes enabled. The codes are then stored server-side in Keycloak and can be used at any future login as an alternative to the OTP authenticator if the user loses access to their device.

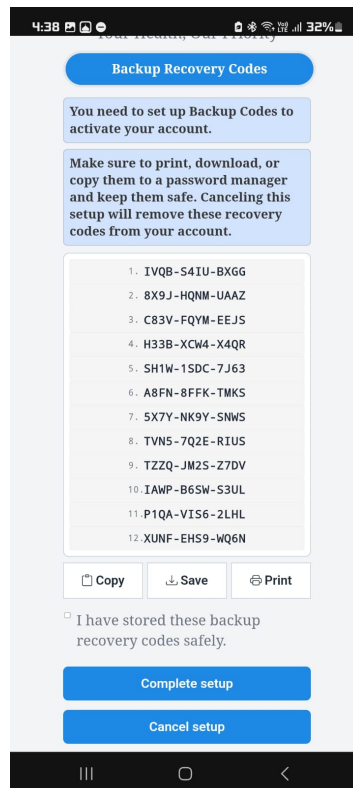


Figure 3.32: Recovery Codes

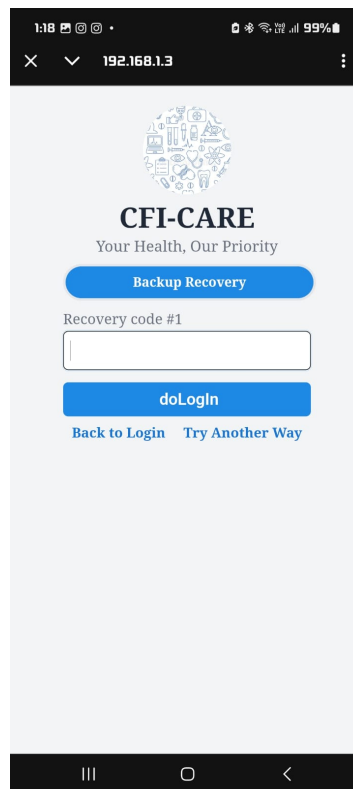


Figure 3.33: Enter Recovery Code #1

Authenticator Setup (TOTP)

Setting up a Time-based One-Time Password (TOTP) through three steps:

1. **Install an authenticator app** : the page lists supported apps (Google Authenticator, FreeOTP, etc.).
2. **Scan the QR code** : Keycloak generates a totpSecret and renders it as a base64-encoded QR code image. Users who cannot scan can switch to manual entry mode, which shows the raw secret key along with the algorithm (TOTP, HMAC-SHA1), digit count (6), and time interval (30 seconds).
3. **Enter a test code** : the user enters the 6-digit code from their authenticator app to prove the setup worked, and optionally gives the device a name (e.g., "My Phone").

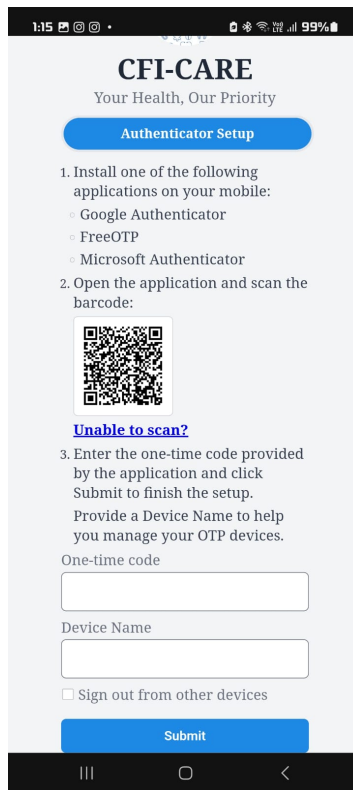


Figure 3.34: Authenticator Setup

Reset Password

1. **Unauthenticated Reset (Forgot Password)** On the login page(`login.ftl`), a "Forgot Password" link is rendered. Clicking it navigates to `login-reset-password.ftl`, which presents a simple form asking for the user's username or email.

On submission, Keycloak sends a password reset email to the user's registered address containing a time-limited link. Clicking the link in the email takes the user directly to login-update-password.ftl where they set a new password.

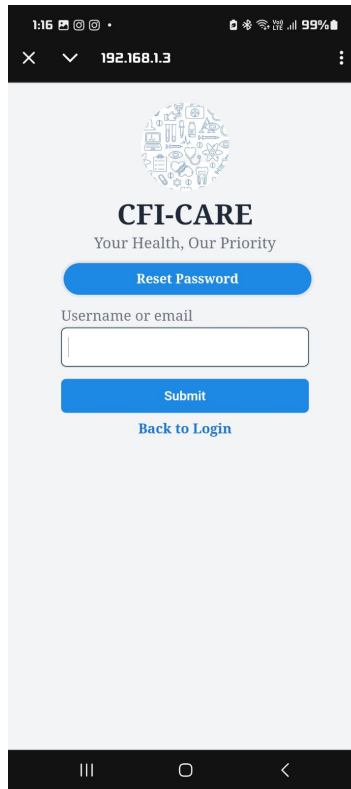
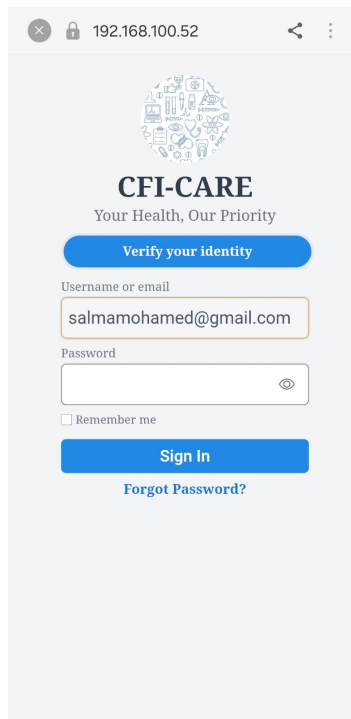


Figure 3.35: Unauthenticated Reset Password

2. **Authenticated Reset (Change Password from App)** A logged-in user can change their password from the Account & Security section of their profile. Tapping "Change Password" redirects to "Verify your identity" page, and the username field is pre-filled with the user's email and set to read-only. This prevents the user from changing identity mid-action.



The screenshot displays a mobile web interface for 'CFI-CARE' with the tagline 'Your Health, Our Priority'. At the top, a status bar shows a lock icon, the IP address '192.168.100.52', and navigation icons. The main content area features a circular logo composed of various medical icons. Below the logo, a blue button labeled 'Verify your identity' is present. Underneath, there are two input fields: 'Username or email' containing 'salmamohamed@gmail.com' and 'Password' which is currently empty and has an eye icon for toggling visibility. A 'Remember me' checkbox is located below the password field. At the bottom of the form, there is a blue 'Sign In' button and a link for 'Forgot Password?'.

Figure 3.36: Verify your Identity

After re-verifying credentials, Keycloak presents **login-update-password.ftl**, which shows a new password field, a confirm password field, and "Log out other sessions" checkbox, allowing the user to invalidate all other active sessions simultaneously. On successful submission, Keycloak returns a fresh token set and the app session remains active, the user does not need to log in again.

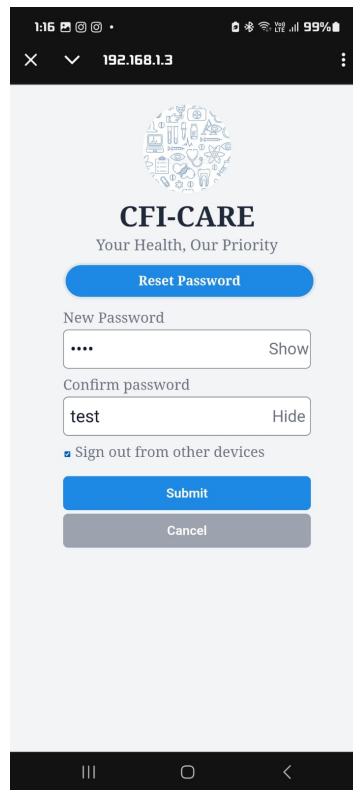


Figure 3.37: Enter & Confirm New Password

3.7.2 Episode of Care

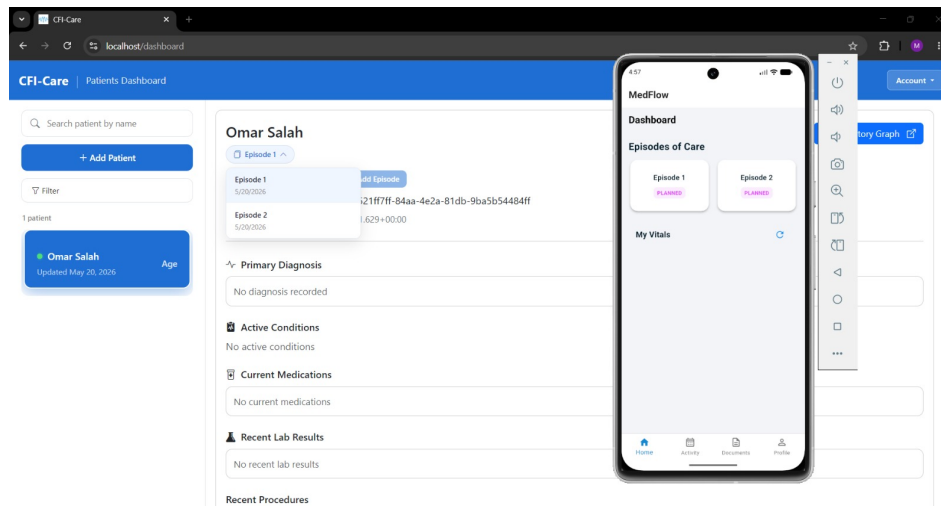


Figure 3.38: Episode of Care

1. **View Episode of Care (Dashboard):** The entry point where the patient views high-level summaries of their active medical contexts (e.g., Cardiac Evaluation).
2. **View Encounter Details:** Selecting a specific node allows the user to inspect the full clinical details of that visit.

3. **View Attached Documents:** Within the details view, the user can optionally retrieve and view specific medical files (PDFs/Images) associated with that encounter.

3.7.3 Digital Document Management & Elimination of Physical Records

The application enables patients to digitally capture, upload, and store medical reports, diagnostic scans, and other relevant health documents using their mobile devices. These documents are maintained within the application's local storage and synchronized with the backend system when required, thereby reducing reliance on physical medical records.

Document List

It displays all uploaded medical documents as a scrollable list, with a custom search header at the top for filtering by title and a blue **Floating Action Button** with a + icon anchored to the bottom-right corner. Tapping the FAB opens the Add Document bottom sheet. Each document is rendered as a card with:

- A type icon, red PDF icon or blue image icon, color-coded by file type.
- A one-line summary below it, and two tag chips showing the document's type (e.g. LAB, XRAY) and speciality (e.g. CARDIOLOGY, GENERAL)
- A time label on the right
- A **cloud sync indicator** green cloud_done if synced to the server, light blue cloud_off if pending, and grey if out of sync.
- When a document is added, it is immediately inserted into the local list (`_documents.add(newDoc)`) so it appears in the UI instantly, and then a sync is attempted. The `isSynced` flag on each `DocumentModel` tracks whether the backend has received the file, reflected visually by the cloud icon on each card.
- Documents support swipe-to-delete via Flutter's `Dismissible` widget [?], swiping left reveals a red delete background with a trash icon.
- Tapping a document opens it using `OpenFilex` [?], which launches the device's default app for PDFs or images. If the file no longer exists, a toast error is shown.

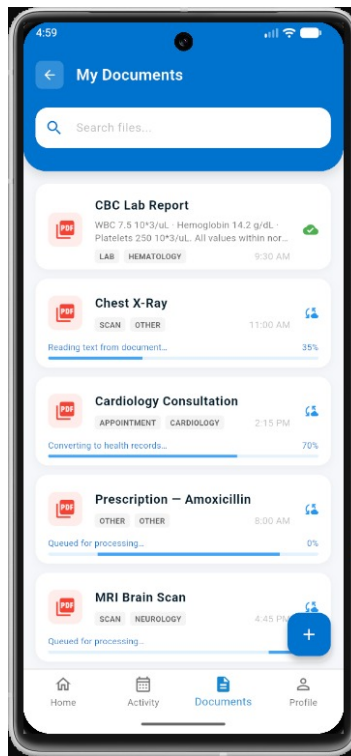


Figure 3.39: Documents Screen

Add Document Bottom Sheet

Tapping the Floating Action Button opens a modal bottom sheet (**DocumentAddSheet**) that slides up from the bottom as it can be seen in figure 3.40. . The form contains:

- **Title** required
- **Summary** one-line description
- **Details** multi-line notes
- **Time picker** tapping opens a time picker dialog
- **Two dropdowns side by side** : Type (Lab, Xray, Prescription, etc.) and Speciality (Cardiology, Neurology, General, etc.)
- **Save button** is disabled unless both a title and a file are provided (isValid check).

The screenshot shows a mobile application interface for adding a document. At the top, there's a dark blue header with a back arrow and the text 'My Documents'. Below this is a section titled 'Add Document'. It contains several input fields: 'Document Title' with a 'T' icon, 'Summary' with a '-' icon, 'Details' with a '≡' icon, 'Time' with a clock icon and the value '8:20 PM', 'Type' with a dropdown arrow and the value 'OTHER', and 'Speciality' with a dropdown arrow and the value 'OTHER'. Below these fields is an 'Attach File' section with three buttons: 'PDF' with a PDF icon, 'Gallery' with a gallery icon, and 'Scan' with a scanner icon. At the bottom of the form is a large blue button labeled 'Save Document'.

Figure 3.40: Add Document Form

File Attachments

The bottom sheet offers three ways to attach a file:

1. **PDF Picker** :opens the system file picker filtered to .pdf files only. The selected file is copied into the app's documents directory with a timestamp-prefixed filename to avoid collisions.
2. **Gallery** opens the image gallery via `image_picker[?]`. After selection, the image is copied to the app directory and an automatic blur check is run immediately.
3. **Scan** navigates to the MultiPageScanner screen. The scanner returns a path to a PDF of the scanned pages, which is then copied to the app directory.

Image Quality Detection — Laplacian Variance

After any image is picked from the gallery or camera, `ImageQualityService.computeBlurScore()` runs automatically. The algorithm to compute if the image or the scanned photo is blurry or not.

1. Convert to grayscale to reduce each pixel to a single value and removes colour noise that could interfere with the edge detection
2. Computes the variance of all Laplacian responses A high variance means many sharp edges (a clear image). A low variance means smooth, uniform regions (a blurry image).
3. The threshold is set at 80.0: any score below this triggers an orange warning banner in the UI: "Image looks blurry. Try retaking or enhance it."

Magic Filter

If the image is flagged as blurry (or the user simply wants better quality), the Magic Filter button applies a 6-step document enhancement pipeline :

Table 3.4: Image Preprocessing Pipeline

Step	Operation	Purpose
1	Grayscale	Removes colour noise and reduces the processing space.
2	Percentile histogram stretch (2%–98%)	Corrects uneven lighting while preserving image details without binarization.
3	Unsharp mask (radius 1, amount 2.5)	Enhances text edges while maintaining gradient information.
4	8-connected Laplacian convolution ([-1,-1,-1,-1,9,-1,-1,-1,-1])	Sharpens both horizontal, vertical, and diagonal text strokes.
5	S-curve contrast boost (factor 1.5)	Increases contrast by pushing the paper background toward white and the ink toward black.
6	Auto-invert check	If more than 50% of pixels are dark, the image is inverted to ensure black text on a white background.

The enhanced image is saved as a new file (enhanced_FileName) and the blur score is re-computed on the result so the warning clears if enhancement succeeded.

Spirit Level

One of the most common problems with phone-based document scanning is perspective distortion, when the phone is held at an angle over the document, the resulting image is trapezoid-shaped rather than rectangular, making text harder to read and OCR less accurate. To address this, the CFI-Care scanner includes a real-time spirit level built using the device’s accelerometer.

How It Works

The `sensors_plus [?]` package provides a continuous stream of accelerometer events. The scanner subscribes to this stream in `_initSensors()`.

- The accelerometer reports gravitational force in m/s^2 along three axes. When the phone is held perfectly flat and parallel to a surface, both x and y approach 0 and only z carries the full gravitational value (9.8). The threshold of 0.7 on both axes was chosen experimentally to give a comfortable tolerance — the phone does not need to be perfectly flat, just reasonably aligned.
- The bubble’s position is a direct mapping of the accelerometer’s x and y values . When the phone tilts right, the bubble moves right. When it tilts forward, it moves down, exactly like a physical spirit level. The bubble and its containing ring turn green when the phone is level and red when it is not.

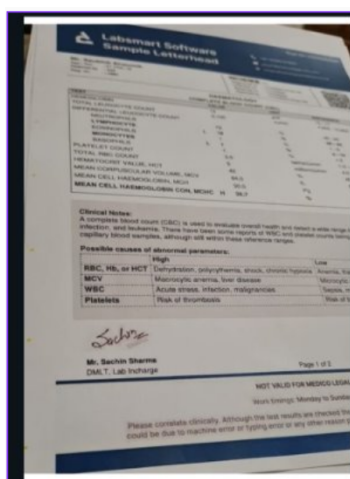


Figure 3.41: trapezoid-shaped Image

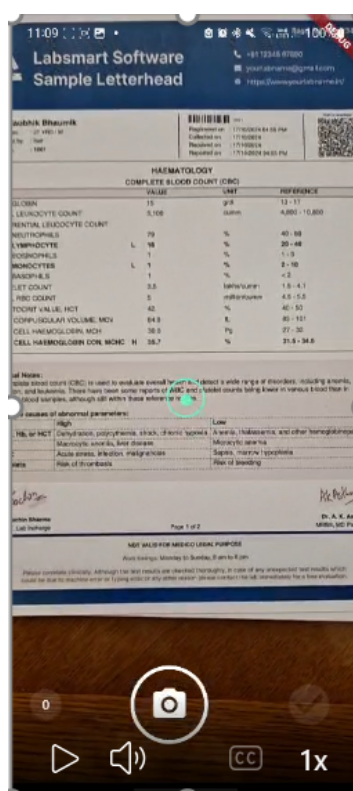


Figure 3.42: Green Spirit Level because parallel to the surface

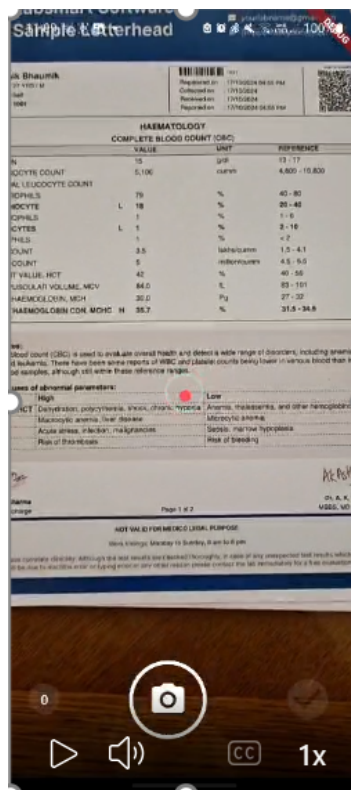


Figure 3.43: Red Spirit Level because tilted to the surface

Document Details & OCR Pipeline

Clicking on any document in the list shown in Figure 3.39 navigates the user to the Document Information screen. This screen displays the document details along with the status of the text extraction process, indicating whether extraction was successful, is still being processed, or has failed. Providing this information allows patients to verify whether the system was able to correctly interpret the uploaded document and identify cases where poor image quality or severe distortion prevented successful text extraction.

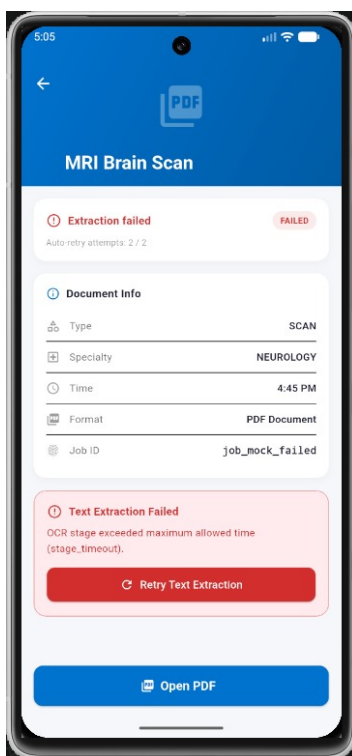


Figure 3.44: Text Extraction Failed

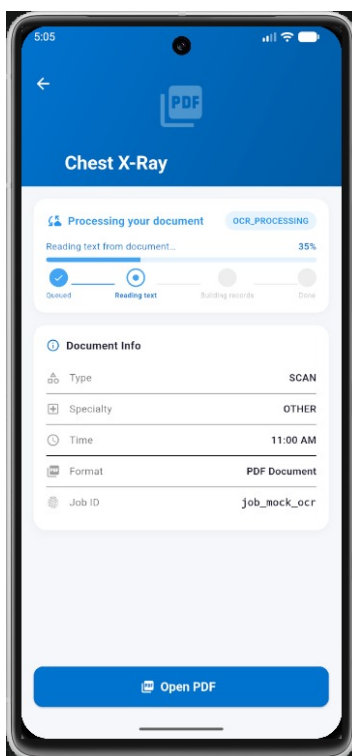


Figure 3.45: OCR processing

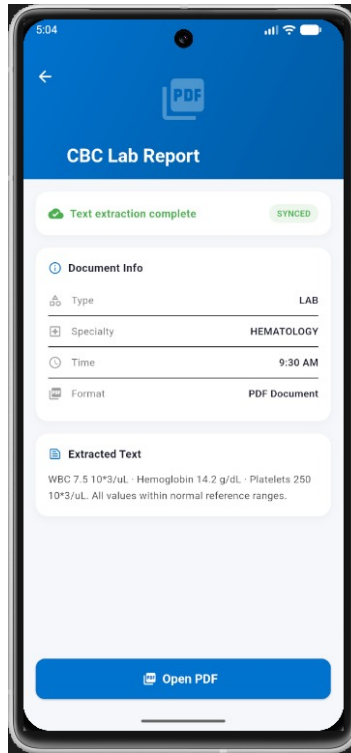


Figure 3.46: Text Extract successfully and synced with backend

3.7.4 Doctor-Patient Data Sharing

A core objective of the system is to support controlled and consent-based data sharing. Doctors may submit requests to access a patient's medical data, which the patient can explicitly approve or reject or revoke the access permission later. Upon approval, patients are able to define granular access permissions, such as read-only or read-write access, as well as specify a time-bound duration for which the access remains valid. This approach ensures patient data privacy while enabling physicians to effectively review and update medical records when authorized.

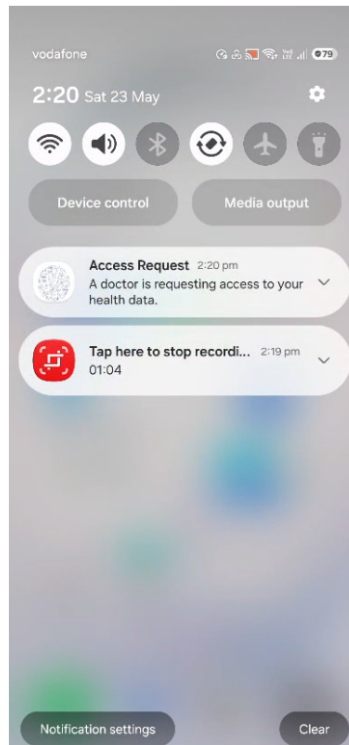


Figure 3.47: Notification with doctor requesting access to your data

Share Access Code with Doctor is a code for the doctor to enter. The patient decides who they want to give the code.

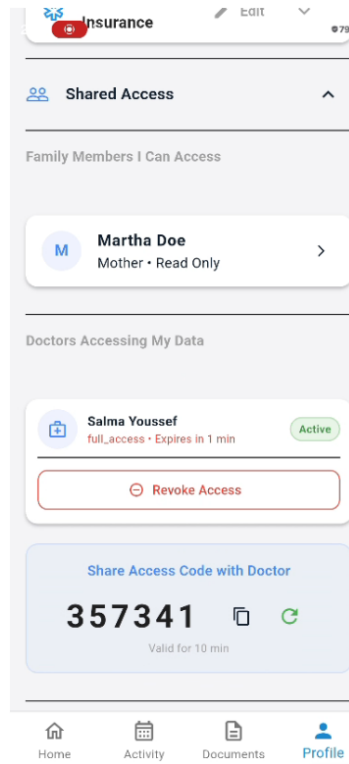


Figure 3.48: Doctor appearing in Doctors that can access your data list and user can revoke access

3.7.5 Caregiver and Family Involvement

In addition to doctor access, the application supports caregiver and family member involvement. Patients may grant designated caregivers permission to monitor their health status, subject to explicit consent.

Caregiver Access Flow

1. Patient generates a code

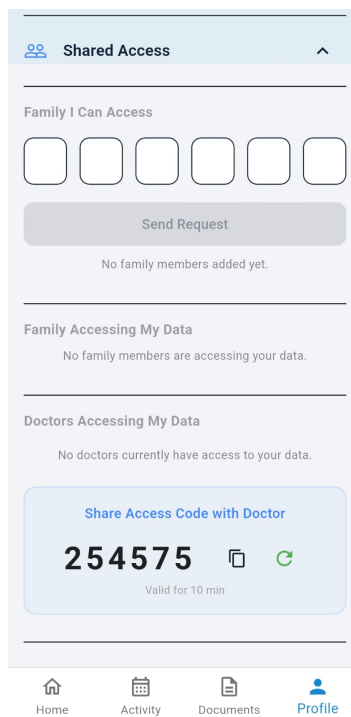


Figure 3.49: Patient Generating Code

2. Caregiver enters the code

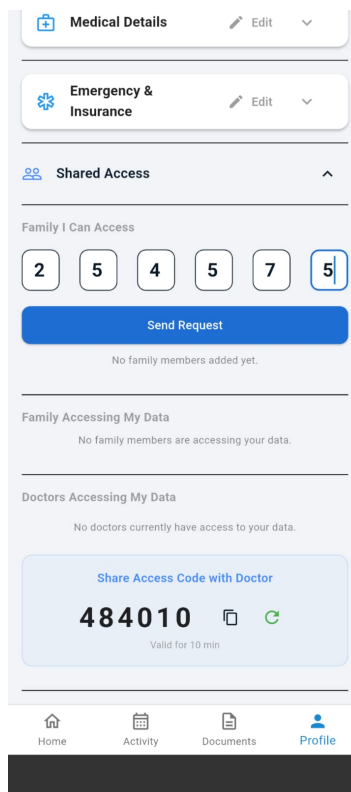


Figure 3.50: Caregiver entering Patient's code

3. Patient receives notification and approves/denies

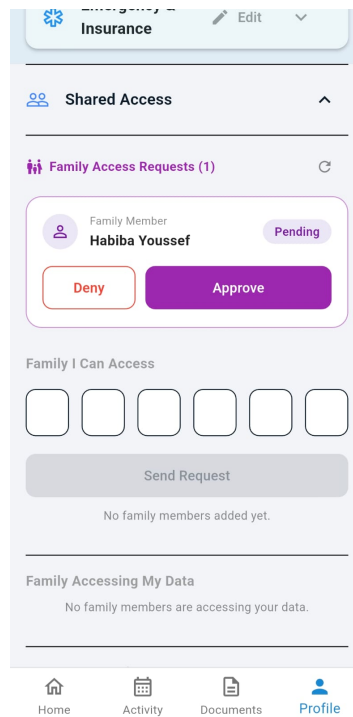


Figure 3.51: Caregiver request access (Patient view)

4. Caregiver s notified and their list updates

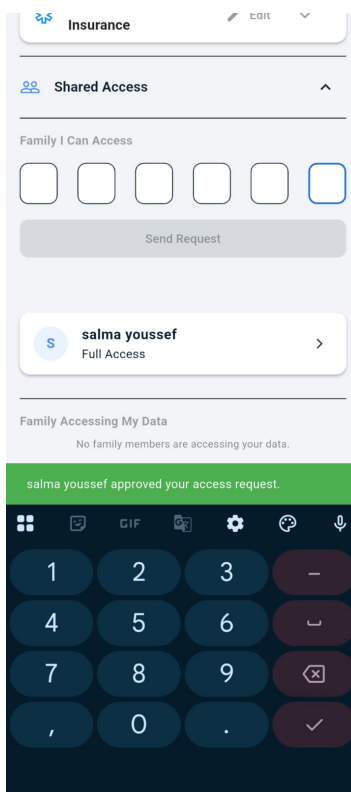


Figure 3.52: Caregiver is notified and their list updates

5. Patient sees Caregiver in their family accessors list

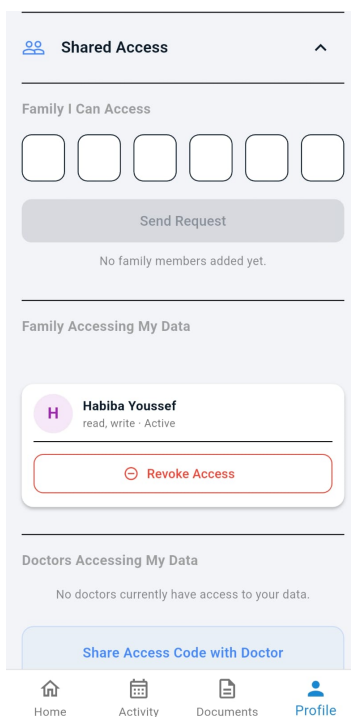


Figure 3.53: Patient sees Caregiver in their family accessors list and ability to revoke access

Proxy View (Caregiver viewing Patient's data)

Caregiver taps Patient's name in their "Family I Can Access" list → ProxySession-Provider.switchTo(Patient.patientId, Patient.name).

What changes immediately across the whole app:

1. A purple banner appears at the top of every screen: "Viewing (Patient name) s profile"
2. The home screen detects the proxy change, and loads Patient's episodes of care instead of Caregiver's own data.
3. In the Profile screen, all Shared Access sub-sections (pending requests, doctors, family lists, share code generator) are hidden Caregiver only sees Patient's profile data
4. Caregiver taps "Switch Back to My Profile" → everything reverts to Caregiver's own data.

3.7.6 Appointment Management

The application further facilitates appointment management by allowing patients to schedule medical appointments directly through the mobile interface. During the booking process, patients may optionally attach relevant medical documents to provide physicians with preliminary information prior to consultation.

Updating Data

- If caregiver updates patient data, patient is notified, and data is updated for both.

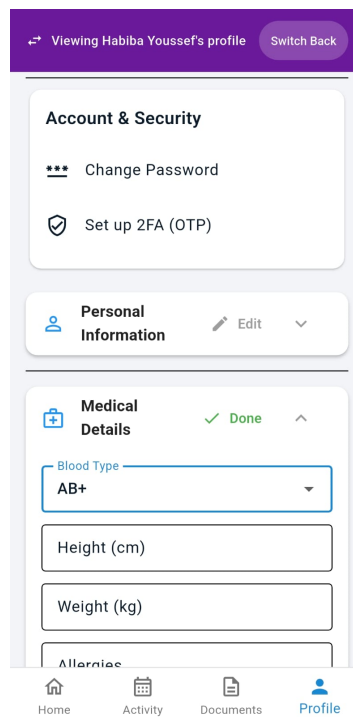


Figure 3.54: Caregiver updating Patient data

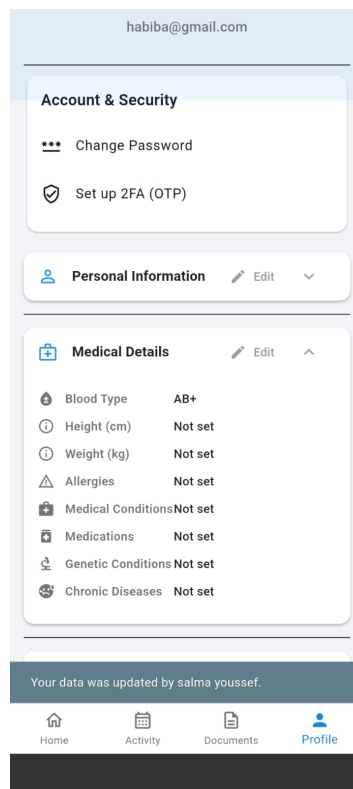


Figure 3.55: Patient receiving notification that Caregiver updated Patient data

- If patient updates their own data, caregiver is notified, and data is updated for both.

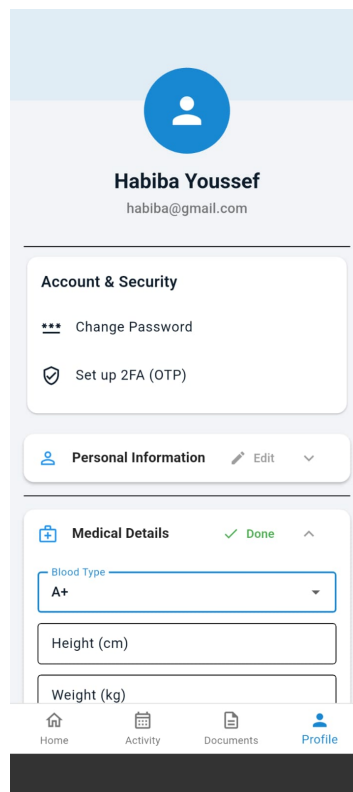


Figure 3.56: Patient update their own data

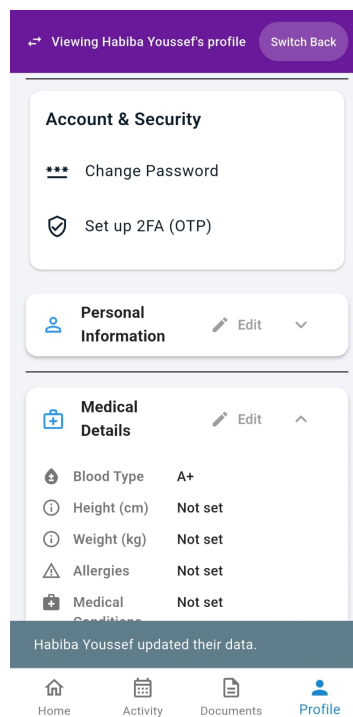


Figure 3.57: Caregiver receiving notification that Patient updated their own data

Booking Appointment Flow

1. Search Doctor Specialty

- Patient sees a grid of medical specialties (Cardiology, Dermatology, Psychiatry, etc.), and selects one speciality based on their need → navigates to Doctor List.

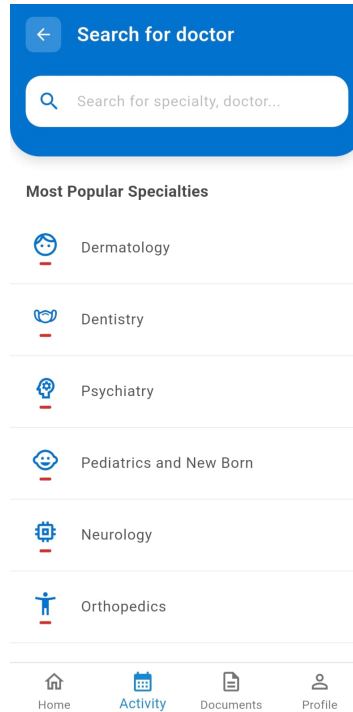


Figure 3.58: list of doctor speciality

2. Doctor List

- Load list of doctors with the speciality chosen in **Search Doctor Specialty**
- Displays cards with name, rating, fees, waiting time.
- Patient taps a doctor card → Doctor Profile

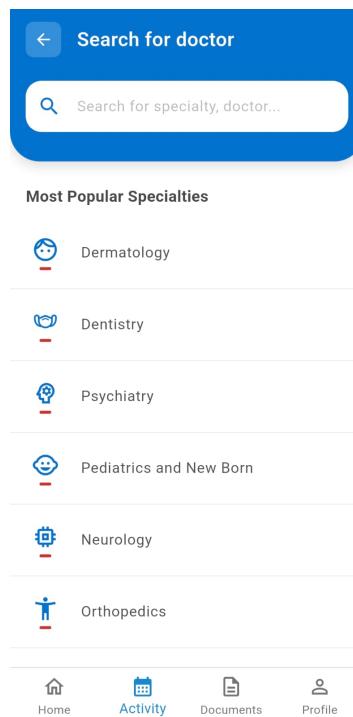


Figure 3.59: List of doctors with the speciality choosen (in this case Dermatology)

3. Doctor Profile

- Shows full doctor details (about, reviews, fees)
- Displays expandable schedules with available time slots
- Patient taps a "Book" button on a slot → Thank You Screen

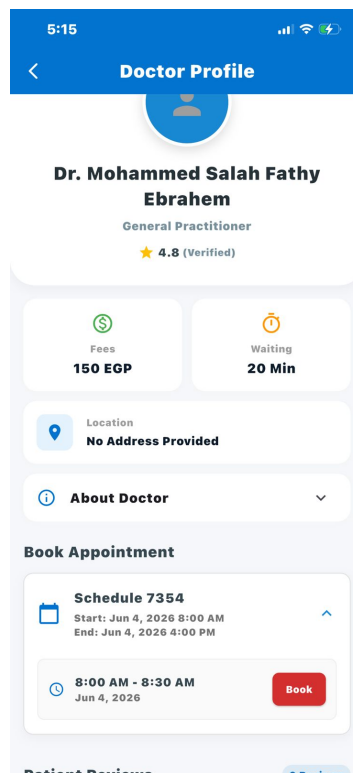


Figure 3.60: Doctor Profile with slots available

4. Confirmation

- Summary card: doctor info, date/time, location
- Patient form: name (pre-filled), phone number, optional "booking for someone else"
- checkbox "Confirm Booking"

5. Thank You Screen

- Shows confirmation with appointment ID, doctor, date/time, fees
- Optional: patient can add notes/symptoms
- Optional: attach documents Back to Home → resets all selections

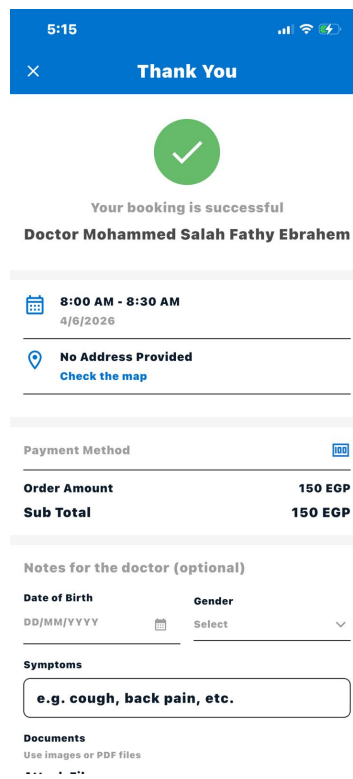


Figure 3.61: Thank You Screen

3.7.7 Wearable Device Integration

Finally, the system is designed to support integration with wearable and smart health devices, such as smartwatches, to collect physiological data including heart rate and other vital signs Using Google Health_Connect[?].

What is Health_Connect

Health Connect is a database that's stored on your device. This means that Google doesn't use the cloud to store data that's sent to Health Connect by connected applications.

Health Connect facilitates interoperability among Android health applications by providing a centralized platform for sharing wellness and fitness data without requiring direct integrations between applications. The platform offers APIs for both storing and retrieving medical records, while also providing users with granular control over data access permissions and a dedicated interface for viewing their medical information.

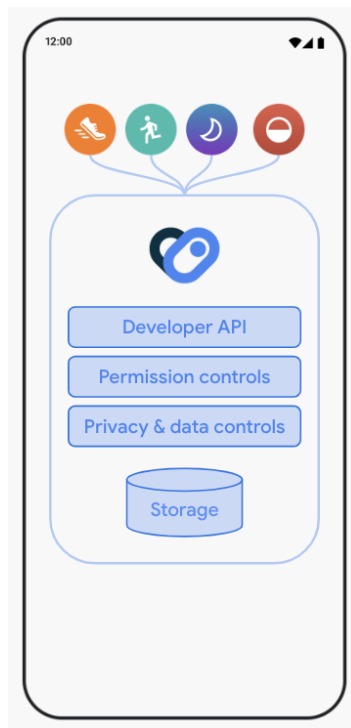


Figure 3.62: Health_Connect Diagram

How to handle Health Connect with several recent versions of Android

- Starting **Android 14** (API Level 34), Health Connect is part of the Android Framework. This version of Health Connect is a framework module. With that, there's no setup necessary.

Go to Settings > Security and Privacy > Privacy Controls > Health Connect, or search for Health Connect in Settings.

- **On Android 13 (API Level 33) and lower** versions, Health Connect is not part of the Android Framework. With that, you need to install the Health Connect app from the Google Play Store.

Go to Settings > Apps > Health Connect, or add Health Connect to your Quick Settings menu.

Health Connect Granular Permissions

One of the key benefits of Health Connect is its support for granular permissions. Instead of granting applications unrestricted access to all health information, users can authorize access to individual data categories. This fine-grained permission model strengthens privacy and security while allowing applications to access only the data necessary for their functionality.

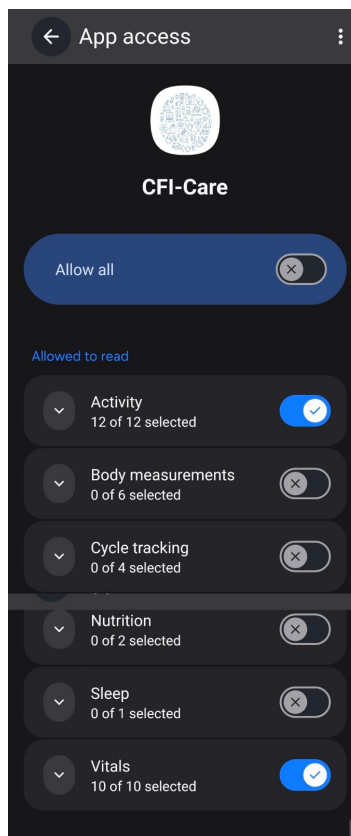


Figure 3.63: Granular Permissions

Data Sources & Priority

• Data Sources

Health Connect acts as a centralized repository that receives health data from different applications and wearable ecosystems, such as:

- CFI-Care
- Samsung Health
- OMRON Connect
- Withings
- Other Health Connect-compatible applications

Each record stored in Health Connect contains metadata identifying the source application that created it.

• Data Priority

Because multiple applications may contribute the same health metric (for example, step counts from both Samsung Health and Fitbit), Health Connect allows users to configure data source priority.

The priority setting determines which application's data should be preferred when conflicts or overlapping records occur. This helps avoid duplicate counting and improves data consistency.

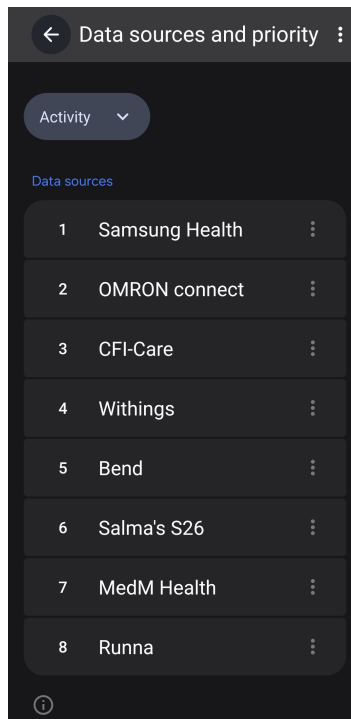


Figure 3.64: Data Sources & Priority

Collecting Medical Data from Various Medical Devices

By integrating with Health Connect, the application can access health and fitness data aggregated from a wide range of compatible wearable devices and health platforms. This interoperability eliminates the need for separate integrations with individual device vendors, allowing the system to obtain user health metrics from multiple sources through a unified interface.

1. Galaxy Watch :

- Both the smartphone and Android Smartphone have Samsung Health[?].
- Samsung Health sends data to Health Connect.
- CFI-Care then asks for data from Health.

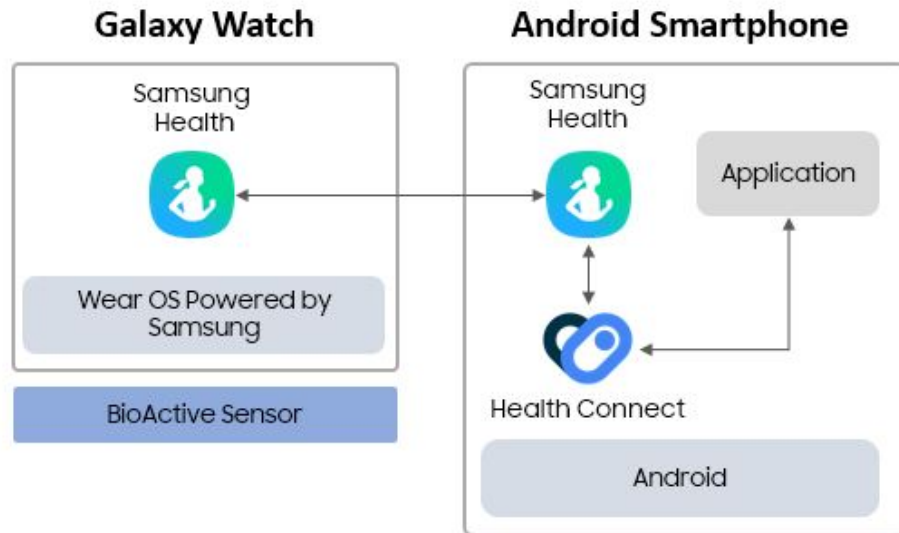


Figure 3.65: Health_Connect with GalaxyWatch & AndroidSmartphone

2. OMRON Connect

- User installs OMRON Connect on their smartphone.
- OMRON Connect gets data from one of OMRON medical devices (eg. Arm Blood Pressure).
- OMRON ask for permission to read and write data on Samsung health
- Samsung has sends data to Health Connect.
- CFI-Care then asks for data from Health.

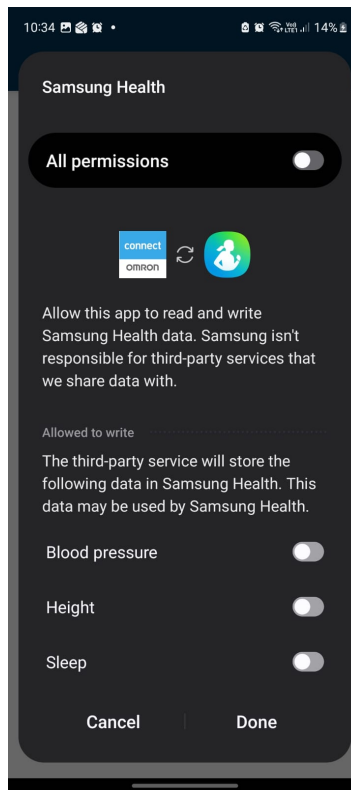


Figure 3.66: OMRON Connect requesting to read & write Samsung Health data

3. Withings App

- Withings ask for permission to read and write data on directly to Health Connect.
- CFI-Care then asks for data from Health.

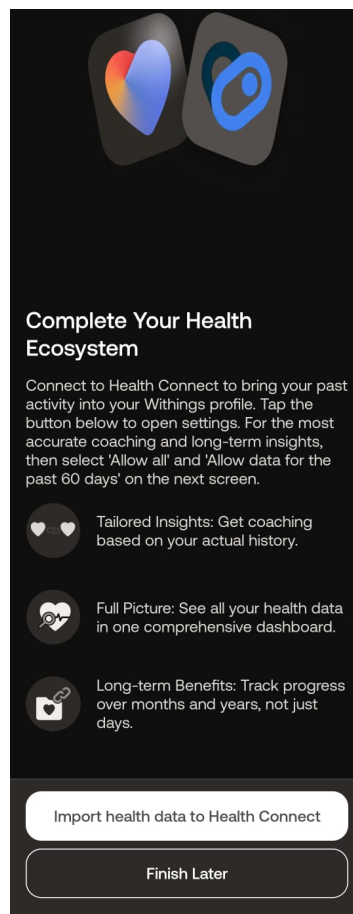


Figure 3.67: Withings requesting access to Health Connect

3.8 Flutter and Dart

3.8.1 Clean Architecture in the Mobile Application

To ensure a scalable, maintainable, and testable codebase for the patient mobile application, this project adopts principles of Clean Architecture.

Clean Architecture is a layered software design paradigm that promotes clear separation of concerns, dependency management, and modularization of application logic. The fundamental goal of this approach is to isolate core business rules from implementation details and platform-specific frameworks, thereby improving long-term maintainability and facilitating unit testing [?].

According to the Flutter documentation on application architecture, well-structured applications organize code into logical layers that separate the user interface, application logic, and data handling. The documentation emphasizes the importance of isolating state management, dependency injection, and data access concerns to support growth and complexity in larger applications [?].

As illustrated in Figure 3.68, the the application is structured into three principal layers: the presentation layer, the domain layer, and the data layer. Each layer

has a distinct responsibility and communicates with adjacent layers through abstract interfaces, ensuring that dependencies always point inward toward business logic rather than outward toward infrastructure or frameworks [?].

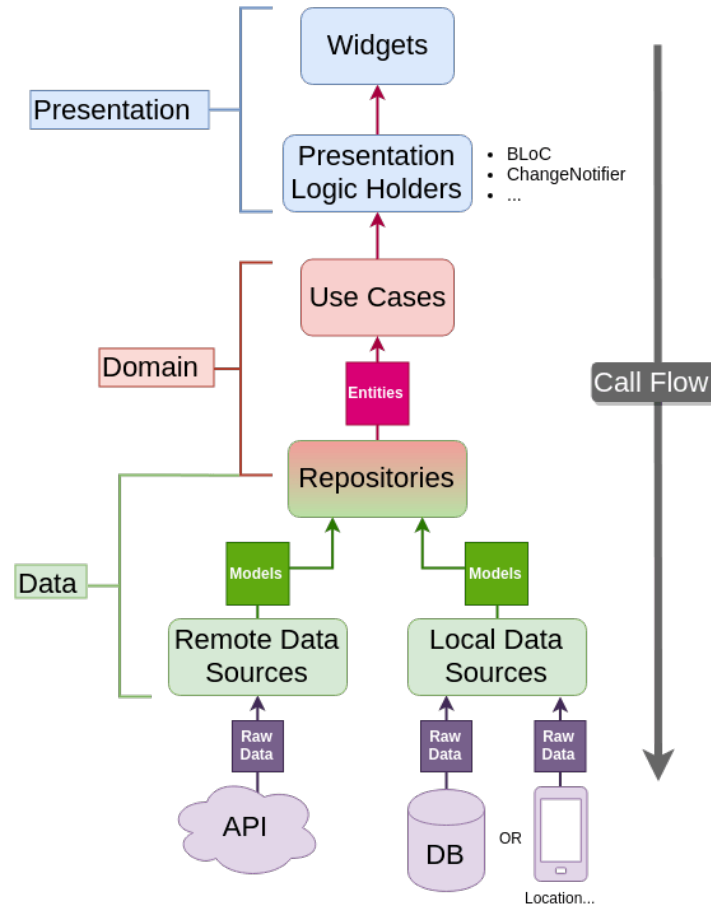


Figure 3.68: Layered Clean Architecture adopted in the mobile application

3.8.2 Layered Structure

Presentation Layer This outermost layer contains code that directly interacts with the user interface. In the context of this Flutter application, it includes widgets, screens, routing logic, and Riverpod providers to manage application state when implemented. The presentation layer orchestrates the flow of data from use cases in the domain layer to the UI and captures user interactions for processing. Presentation logic is kept distinct from business rules and data access to avoid tightly coupling UI changes with core system logic [?].

Domain Layer The domain layer encapsulates the core business logic and entities of the system. It is independent of Flutter or any external dependencies. This layer defines use cases that represent application-specific operations, such as requesting document access or scheduling appointments, and serves as the central coordinator of business rules. The domain layer's independence from external libraries and frameworks enhances testability and ensures that business logic can be validated in isolation [?].

Data Layer The data layer is responsible for integrating with external systems, including APIs, local storage, and the backend FHIR server. It provides implementations of repository interfaces defined in the domain layer and manages data transformation between network responses, local models, and domain entities. The layered separation ensures that external dependencies do not leak into the domain or presentation layers [?].

3.8.3 Centralized Runtime Permission Handling

To ensure a secure and seamless interaction with device hardware, the application implements a centralized asynchronous mechanism for managing runtime permissions. This approach abstracts the complexity of the `permission_handler` library and follows established patterns for robust permission management [?]. The function `handlePermission` provides a unified interface for requesting access to sensitive resources such as the camera, storage, or microphone.

The function operates as a `Future<bool>`, returning a boolean value to indicate the success or failure of the authorization process. Upon invocation, the function executes the following logic flow:

1. **Asynchronous Request:** It initiates a request to the underlying operating system using `permission.request()`. The execution yields until the user interacts with the system dialog or the operating system returns the current status.
2. **Status Evaluation:** The returned `PermissionStatus` is evaluated against standard authorization states:
 - **Granted:** If the user approves the request (`isGranted`), the function immediately returns `true`, allowing the calling process to proceed.
 - **Permanently Denied:** If the permission is flagged as `isPermanentlyDenied` (a state where the operating system suppresses further prompts due to repeated denials), the system intervenes to prevent a silent failure. It displays a toast notification instructing the user to enable the permission manually and invokes `openAppSettings()` to redirect the user to the device's settings menu.
 - **Denied:** In cases of a standard denial, a feedback message is displayed via `Fluttertoast` to inform the user of the restriction, and the function returns `false`.

By encapsulating this logic, the system ensures that permission failures are handled gracefully, guiding users through recovery steps when necessary rather than causing application crashes or unresponsive features. The implementation of this logic is demonstrated in Listing 3.1.

Listing 3.1: Asynchronous Permission Handling Function

```

import 'package:permission_handler/permission_handler.dart';
import 'package:fluttertoast/fluttertoast.dart';

Future<bool> handlePermission(
  Permission permission,
  String name,
) async {
  final status = await permission.request();

  // Case 1: Permission Granted
  if (status.isGranted) {
    return true;
  }

  // Case 2: Permanently Denied (Requires Settings)
  if (status.isPermanentlyDenied) {
    Fluttertoast.showToast(
      msg: '$name␣permission␣permanently␣denied.␣Enable␣it␣in␣settings
        .',
    );
    openAppSettings();
    return false;
  }

  // Case 3: Standard Denial
  Fluttertoast.showToast(msg: '$name␣permission␣denied');
  return false;
}

```



Figure 3.69: using handlePermission For Camera Access In code

Best Practices for handling Permissions in Flutter

- Contextual Permission Requests:** To avoid overwhelming the user and to establish context, the application rejects the traditional “up-front” permission model (where all permissions are requested at launch). Instead, a *Contextual Request* pattern is employed. Permissions are only requested at the specific moment a feature is invoked. For example, access to the **Camera** is requested solely when the user attempts to scan a medical report.
- Transparent Rationale and Trust Building:** In compliance with privacy best practices, the application explains the “Why” before the “How.” Prior to triggering the system permission dialog, the application presents a custom UI explanation detailing exactly why a specific permission is needed and how it benefits the user.

This transparency is critical in a healthcare context, increasing the likelihood of approval and reinforcing the patient’s understanding that their data is being used strictly for medical management.[?]

- **User Agency and Graceful Degradation:** The system is designed to respect user choices without compromising the core functionality of the application. If a user denies a non-critical permission, the app enters a state of *Graceful Degradation*. The application avoids repetitive prompting (“nagging”), ensuring a respectful user experience. [?]
- **Comprehensive State Handling:** The application logic accounts for the full lifecycle of permission states, ensuring stability across different scenarios[?][?]:
 - **Granted:** The feature proceeds immediately.
 - **Denied:** The feature is disabled, and the user is informed of the limitation.
 - **Permanently Denied:** In cases where the user has selected “Don’t ask again,” the application detects this state and provides a subtle option to guide the user to the system **App Settings** menu for manual enablement, should they choose to do so later.

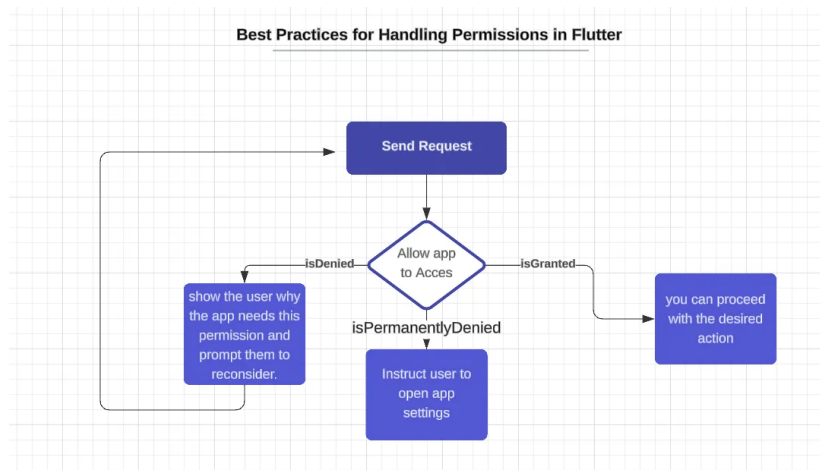


Figure 3.70: Best Practices for Handling Permissions in Flutter

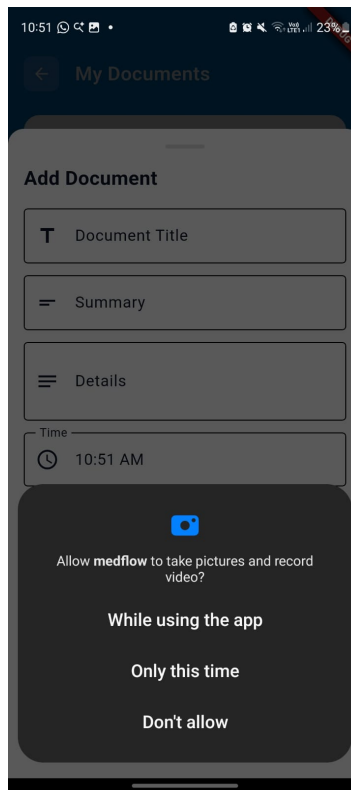


Figure 3.71: Dialog to allow the app to access the camera

3.9 FCM Push Notifications

The system uses Firebase Cloud Messaging (FCM) to deliver real-time push notifications from the Node.js backend to the Flutter mobile app. FCM acts as a broker: the backend never sends a message directly to a device — it sends a message to Google’s FCM servers, which deliver it to the target device using its unique FCM token.

3.9.1 How It Works

Part 1 — Token Registration

- Before any notification can be sent, the backend must know the device’s Firebase Cloud Messaging (FCM) token.
- On the Flutter side, the application calls `FirebaseMessaging.instance.getToken()` to obtain the device’s unique FCM token.
- The token is then sent to the backend through:

```
POST /FCM/fcm-token
{
  "fcmToken": "abc123"
}
```

- Token rotation is handled automatically. Whenever Firebase generates a new token, the updated value is sent again to the same endpoint.

Part 2 — The Shared Sender Utility

All notifications go through one private function :

- Before sending a notification, the backend retrieves the recipient’s FCM token from Redis using the user’s identifier.
- If no token is found, the notification is skipped because the user does not have a registered device.
- The backend then sends a Firebase Cloud Messaging (FCM) message containing both notification and data payloads.
- Notifications are sent with high priority to ensure timely delivery.
- Each notification message consists of two components:
 - **Notification Payload:** Contains the user-visible content, including the notification title and body displayed in the operating system’s notification tray.
 - **Data Payload:** Contains structured key–value pairs processed by the Flutter application. The most important field is `type`, which identifies the notification category and determines the action that the application should perform upon receiving the message.

Part 3 — The Four Notification Types

The system defines four distinct event types, each triggering a different UI response:

Table 3.5: Notification Events and Actions

Event Type	Trigger	Recipient	Action
grant_request	Doctor submits patient’s OTP	Patient	Orange snackbar and refreshing doctor requests
family_request	Caregiver submits patient’s OTP	Patient	Purple snackbar and refreshing family requests
grant_approved	Patient approves any request	Requester	Green snackbar, refresh members, and navigate to tab
data_updated	Patient or caregiver edits health data	The other party	Blue-grey snackbar notification

Access Request Notifications (`grant_request` / `family_request`)

When a doctor or caregiver submits a patient's OTP through the `/verify-practitioner-otp` or `/verify-caregiver-otp` endpoint, the backend first validates the OTP against Redis and retrieves the corresponding patient identifier. A pending handshake record is then created and an FCM notification is sent to the patient.

Depending on the requester type, the notification payload contains either the `grant_request` or `family_request` event type.

Upon receiving the notification, the Flutter application refreshes the appropriate provider and displays a visual notification to the user. Doctor requests trigger an orange snackbar and refresh the pending grants list, while caregiver requests trigger a purple snackbar and refresh the pending family requests list.

Approval Notification (`grant_approved`)

When a patient approves an access request through the `/create-grant` endpoint, the backend creates a grant record in Redis with a predefined time-to-live (TTL). The patient's display name is retrieved from the FHIR server and included in an approval notification sent to the requester.

The notification contains the event type `grant_approved`. Upon receipt, the Flutter application refreshes the accessible members list, navigates the requester to the profile tab, and displays a green snackbar indicating that access has been approved.

Data Update Notifications (`data_updated`)

Whenever a patient's health data is modified, the backend determines whether the update was performed by the patient or by a caregiver.

If a caregiver updates the patient's data, a notification is sent to the patient informing them of the change. Conversely, if the patient updates their own data, all active caregivers associated with that patient are notified.

This implementation follows a fan-out notification pattern, where a single patient update may generate notifications for multiple caregivers. The Flutter application displays an informational snackbar upon receiving a `data_updated` event. demand when needed.

Part 7 — Application State Handling

The notification subsystem supports three application states:

Background and terminated notification flows share a common message handler responsible for refreshing application state after the user opens the notification.

Table 3.6: Notification Handling by Application State

Application State	Handler	Purpose
Foreground	<code>FirebaseMessaging.onMessage.listen()</code>	Display snack-bars and refresh providers
Background	<code>FirebaseMessaging.onMessageOpenedApp.listen()</code>	Handle notification tap events
Terminated	<code>getInitialMessage()</code>	Process notifications after application launch

3.10 Requirements Engineering

To ensure the delivery of a viable and effective application, the development lifecycle begins with a stakeholder analysis to establish the system context. This is followed by a comprehensive definition of the functional, non-functional, and optional requirements. Synthesizing this information facilitates the creation of a domain model, which acts as the foundational blueprint for the subsequent database design.

3.10.1 Target Group and User Group

Stakeholders are categorized into Internal Stakeholders, who are directly involved in the development and operation of the system, and External Stakeholders, who utilize the system's end products.

External Stakeholders

External stakeholders are the primary users of the system. Their interaction drives the application's core functionality.

1. Patients (Primary Users) Interaction:

- **Data Management:** Patients interact with the mobile interface to upload medical documents (lab results, scans) and view their historical data presented in a structured "Major Events" timeline.
- **Access Control:** They possess full authority over their data privacy. Patients receive requests from doctors for data access and can grant permissions (Read-Only or Read-Write) for a specific duration. Crucially, they retain the ability to revoke this access at any time.
- **Family Management:** Patients can authorize family members to view their health records for monitoring purposes.

Benefits:

- **Centralized History:** Eliminates the need for physical paperwork by consolidating all medical history into a single, organized mobile repository.
- **Data Ownership:** Empowers patients with absolute control over who views their medical information and for how long.
- **Continuity of Care:** Ensures that any new doctor can immediately access a complete medical background, reducing diagnostic errors.

2. Family Members (Secondary Users) Interaction:

- **Monitoring:** Through shared access granted by the patient, family members can view health trends and documents to assist in care.
- **Administrative Support:** Unlike doctors, family members have the permission to book appointments, and change data on behalf of the patient, facilitating care for elderly or incapacitated users.

Benefits:

- **Remote Assurance:** Provides peace of mind by allowing real-time monitoring of a loved one's health status.
- **Care Coordination:** Simplifies the logistical burden of managing appointments and records for dependents.

3. Doctors (Service Providers) Interaction:

- **Session Initiation:** While doctors primarily use the Web Interface, their actions directly trigger updates on the mobile app. A doctor initiates a "New Encounter Node" in the patient's history tree during a visit.
- **Access Requests:** Doctors request permission to view a patient's historical records. If granted "Write" access, they can add new diagnoses and prescriptions to the encounter node.
- **Limitation:** Doctors strictly cannot book appointments; this function is reserved for patients and their families to prevent conflict of interest or scheduling errors.

Benefits:

- **Informed Decision Making:** Instant access to a comprehensive medical history allows for more accurate diagnoses.
- **Efficiency:** Reduces time spent interviewing patients for past medical details, allowing more focus on current treatment.

Internal Stakeholders

Internal stakeholders are responsible for the technical construction, maintenance, and security of the platform.

Bibliography